

目录

00_overview

- [WebRTC 音频子系统架构深度分析](#)
 - [1. 组件关系全图](#)
 - [2. 核心类职责一句话总结](#)
 - [3. 线程模型全图](#)
 - [3.1 线程边界关键约束](#)
 - [3.2 线程安全设计模式分析](#)
 - [4. 平台实现选择逻辑](#)
 - [4.1 选择流程](#)
 - [4.2 CreatePlatformSpecificObjects 平台分支](#)
 - [4.3 平台选择决策树](#)
 - [5. 初始化时序与数据流](#)
 - [5.1 初始化调用链](#)
 - [5.2 音频数据流 \(运行时\)](#)
 - [5.3 关键设计决策分析](#)

01_interfaces

- [WebRTC 音频设备层核心接口深度解析](#)
 - [目录](#)
 - [1. AudioDeviceModule — 公共 API 接口](#)
 - [1.1 AudioLayer 枚举 — 音频后端选择](#)
 - [1.2 WindowsDeviceType 枚举](#)
 - [1.3 方法组一：生命周期管理](#)
 - [1.4 方法组二：音频回调注册](#)
 - [1.5 方法组三：设备枚举](#)
 - [1.6 方法组四：设备选择](#)
 - [1.7 方法组五：音频传输初始化](#)
 - [1.8 方法组六：音频传输控制](#)
 - [1.9 方法组七：音频混音器初始化](#)
 - [1.10 方法组八：扬声器音量控制](#)
 - [1.11 方法组九：麦克风音量控制](#)
 - [1.12 方法组十：静音控制](#)
 - [1.13 方法组十一：立体声支持](#)

- [1.14 方法组十二：延迟信息](#)
 - [1.15 方法组十三：平台内置音频处理](#)
 - [1.16 方法组十四：iOS 专有](#)
 - [1.17 Stats 结构体](#)
- [2. AudioTransport — 音频数据传输回调](#)
 - [2.1 设计哲学](#)
 - [2.2 RecordedDatalsAvailable — 录音数据上报](#)
 - [2.3 NeedMorePlayData — 播放数据拉取](#)
 - [2.4 PullRenderData — 渲染数据拉取 \(辅助方法\)](#)
- [3. AudioDeviceGeneric — 平台实现基类](#)
 - [3.1 与 AudioDeviceModule 的关键差异](#)
 - [3.2 AttachAudioBuffer — 核心连接点](#)
 - [3.3 BuiltInAEC/AGC/NS — 平台音频处理能力声明](#)
 - [3.4 GetPlayoutUnderrunCount](#)
 - [3.5 iOS 专有方法](#)
 - [3.6 平台实现的典型生命周期](#)
- [4. AudioDeviceBuffer — 中间缓冲层](#)
 - [4.1 录音路径 API](#)
 - [4.2 播放路径 API](#)
 - [4.3 关键内部状态](#)
- [5. 三者关系与数据流](#)
 - [5.1 架构层次](#)
 - [5.2 录音数据流 \(完整路径\)](#)
 - [5.3 播放数据流 \(完整路径\)](#)
 - [5.4 线程模型](#)
 - [5.5 AudioDeviceModuleDefault 模板](#)
- [附录：关键常量](#)
- [附录：源码文件索引](#)

02_voice_engine

- [WebRtcVoiceEngine 深度源码分析](#)
 - [概述](#)
 - [1. 线程模型与 SequenceChecker 设计](#)
 - [1.1 worker_thread_checker_ 的 kDetached 模式](#)
 - [1.2 线程归属总结](#)

- [2. ADM 创建时机: EnsureAudioDeviceModule](#)
 - [2.1 源码分析](#)
 - [2.2 ADM 创建时机的设计决策](#)
- [3. 构造函数逐行分析](#)
 - [3.1 完整构造函数](#)
 - [3.2 逐行深度解析](#)
 - [3.3 构造函数的设计约束](#)
- [4. Init\(\) 逐行分析](#)
 - [4.1 完整 Init\(\) 实现](#)
 - [4.2 逐行深度解析](#)
- [5. adm_helpers::Init\(\) 逐行分析](#)
 - [5.1 完整实现](#)
 - [5.2 逐行深度解析](#)
 - [5.3 设计决策: 为什么分离 Init 和硬件初始化?](#)
- [6. Terminate\(\) 逐行分析](#)
 - [6.1 完整实现](#)
 - [6.2 逐行深度解析](#)
 - [6.3 Terminate\(\) 的顺序设计](#)
- [7. 完整生命周期总结](#)
 - [7.1 时序图](#)
 - [7.2 关键设计原则](#)
 - [7.3 源码引用索引](#)

03_audio_state

- [AudioState 完整调度机制分析](#)
 - [1. 概述](#)
 - [1.1 线程模型](#)
 - [2. 构造函数分析](#)
 - [源码](#)
 - [逐行分析](#)
 - [关键设计决策](#)
 - [3. SetPlayout 方法分析](#)
 - [源码](#)
 - [逐行分析](#)
 - [边界条件](#)

- [4. AddReceivingStream 方法分析](#)
 - [源码](#)
 - [逐行分析](#)
 - [边界条件](#)
 - [与 SetPlayout 的协作关系](#)
- [5. RemoveReceivingStream 方法分析](#)
 - [源码](#)
 - [逐行分析](#)
 - [边界条件](#)
 - [与 AddReceivingStream 的对称性](#)
- [6. AddSendingStream / RemoveSendingStream 分析](#)
 - [设计对称性](#)
 - [播放 vs 录音对比](#)
- [7. UpdateNullAudioPollerState 方法分析](#)
 - [源码](#)
 - [逐行分析](#)
 - [NullAudioPoller 的存在意义](#)
 - [启动/停止条件真值表](#)
- [8. 完整状态矩阵](#)
 - [8.1 播放方向状态矩阵](#)
 - [8.2 状态转换图](#)
 - [8.3 状态转换触发事件](#)
- [9. AudioTransport 桥接机制](#)
 - [构造时的绑定](#)
 - [NullPoller 对 AudioTransport 的复用](#)
- [10. 线程安全与生命周期](#)
 - [10.1 线程约束](#)
 - [10.2 生命周期管理](#)
 - [10.3 析构顺序](#)
- [11. 设计模式总结](#)
 - [11.1 双条件守卫模式](#)
 - [11.2 幂等操作模式](#)
 - [11.3 补偿机制模式 \(NullPoller\)](#)
 - [11.4 最终一致性模式](#)

- [12. 常见问题与边界场景](#)
 - [Q1: 为什么 RemoveReceivingStream 在流为空时无条件停止 ADM?](#)
 - [Q2: SetPlayout\(false\) 后再 SetPlayout\(true\), ADM 会重新初始化吗?](#)
 - [Q3: NullPoller 的 10ms 周期是否精确?](#)
 - [Q4: 多个 PeerConnection 共享同一个 AudioState 会怎样?](#)
 - [Q5: InitPlayout\(\) 失败会怎样?](#)
- [13. 完整调用时序图](#)
 - [场景: 正常通话建立与结束](#)
 - [场景: 静音模式 \(禁用播放但保持解码\)](#)
- [14. 源码文件索引](#)
- [15. 总结](#)

04_device_buffer

- [AudioDeviceBuffer 深度解析](#)
 - [概述](#)
 - [架构位置](#)
 - [录音路径完整分析](#)
 - [Step 1: SetRecordedBuffer — 原始 PCM 数据接收](#)
 - [Step 2: SetVQEData — 设置 VQE 延迟参数](#)
 - [Step 3: DeliverRecordedData — 数据交付给上层](#)
 - [录音路径时序图](#)
 - [播放路径完整分析](#)
 - [Step 1: RequestPlayoutData — 向上层拉取混音数据](#)
 - [Step 2: GetPlayoutData — 数据拷贝到硬件缓冲区](#)
 - [播放路径时序图](#)
 - [录音 vs 播放路径对比](#)
 - [线程安全设计深度分析](#)
 - [线程模型](#)
 - [同步策略总览](#)
 - [策略 1: std::atomic — 采样率和通道数](#)
 - [策略 2: 协议约束 — audio_transport_cb](#)
 - [策略 3: 无锁 — 音频线程独占数据](#)
 - [策略 4: Mutex — 统计数据](#)
 - [策略 5: SequenceChecker — 状态标志](#)
 - [FineAudioBuffer 原理](#)

- [问题背景](#)
- [解决方案: FineAudioBuffer 适配层](#)
- [录音方向: 积累后推送](#)
- [播放方向: 预取后切片](#)
- [FineAudioBuffer 的设计决策](#)
- [完整数据流总结](#)
 - [录音完整路径 \(从麦克风到网络\)](#)
 - [播放完整路径 \(从网络到扬声器\)](#)
- [关键设计原则总结](#)
 - [1. 最小同步原则](#)
 - [2. 音频线程零阻塞](#)
 - [3. 固定帧长简化设计](#)
 - [4. Push/Pull 混合模型](#)
- [源码文件索引](#)
- [附录: 常见问题](#)
 - [Q: 为什么 AudioDeviceBuffer 不直接使用环形缓冲区?](#)
 - [Q: 如果 AudioTransport::NeedMorePlayData 执行太慢怎么办?](#)
 - [Q: capture_timestamp_ns 有什么用?](#)
 - [Q: typing_status_ 是什么?](#)

05_data_pipeline

- [AudioTransportImpl 数据管线深度解析](#)
 - [概述](#)
 - [架构位置](#)
 - [类定义与构造](#)
 - [头文件结构 \(audio/audio_transport_impl.h\)](#)
 - [构造函数分析](#)
 - [录音路径完整分析: RecordedDataIsAvailable](#)
 - [函数签名](#)
 - [Step 1: 帧长度校验](#)
 - [Step 2: 加锁读取发送参数](#)
 - [Step 3: 初始化采集帧 \(选择处理采样率\)](#)
 - [Step 4: Remix + Resample](#)
 - [Step 5: APM 处理 \(音频前处理\)](#)
 - [Step 6: 设置采集时间戳](#)

- [Step 7: 分发给 AudioSender](#)
- [SendProcessedData: 多流分发机制](#)
 - [逐行分析](#)
- [播放路径完整分析: NeedMorePlayData](#)
 - [函数签名](#)
 - [Step 1: AudioMixer 混合所有接收流](#)
 - [Step 2: APM 反向流处理 \(AEC 参考信号\)](#)
 - [Step 3: Resample 到 ADM 请求的采样率](#)
- [PullRenderData: Chromium 专用播放路径](#)
 - [与 NeedMorePlayData 的区别](#)
- [UpdateAudioSenders: 动态流管理](#)
 - [调用时机](#)
 - [参数计算逻辑 \(AudioState 中\)](#)
- [AsyncAudioProcessing: 异步处理路径](#)
 - [创建机制](#)
 - [工作流程](#)
 - [设计考量](#)
- [AudioMixer 的 Source 注册机制](#)
 - [接口定义 \(api/audio/audio_mixer.h\)](#)
 - [AudioMixerImpl 实现 \(modules/audio_mixer/audio_mixer_impl.h\)](#)
 - [AddSource 实现](#)
 - [RemoveSource 实现](#)
 - [Mix 实现核心流程](#)
 - [Source 注册的调用链](#)
 - [OutputRateCalculator: 混合采样率决策](#)
- [线程模型与锁设计](#)
 - [涉及的线程](#)
 - [锁分析](#)
- [完整数据流总结](#)
 - [录音路径 \(Capture Path\)](#)
 - [播放路径 \(Render Path\)](#)
- [APM 在整体管线中的位置总结](#)
- [关键设计模式总结](#)
 - [1. Push vs Pull 模型](#)

- [2. 10ms 帧统一](#)
- [3. 采样率协商](#)
- [4. 零拷贝优化](#)
- [5. 可选组件](#)
- [源码文件索引](#)

06_ios_platform

- [第六章：iOS 平台音频实现](#)
 - [概述](#)
 - [1. AudioDeviceIOS 类架构](#)
 - [1.1 继承关系](#)
 - [1.2 关键成员变量](#)
 - [2. Voice Processing I/O Audio Unit 架构](#)
 - [2.1 Audio Unit 类型选择](#)
 - [2.2 Bus 架构详解](#)
 - [2.3 回调注册](#)
 - [3. 完整回调链分析](#)
 - [3.1 播放回调链 \(Playout\)](#)
 - [3.2 录音回调链 \(Recording\)](#)
 - [3.3 回调时序关系](#)
 - [4. FineAudioBuffer：帧大小适配层](#)
 - [4.1 问题背景](#)
 - [4.2 FineAudioBuffer 的解决方案](#)
 - [4.3 播放方向工作原理](#)
 - [4.4 录音方向工作原理](#)
 - [4.5 为什么不直接修改 WebRTC 的帧大小？](#)
 - [5. AVAudioSession 中断处理](#)
 - [5.1 iOS 音频中断机制](#)
 - [5.2 WebRTC 的中断处理架构](#)
 - [5.3 中断开始处理](#)
 - [5.4 中断结束处理](#)
 - [5.5 UpdateAudioUnit 状态机](#)
 - [5.6 路由变化处理](#)
 - [6. 为何禁用 WebRTC 软件 AEC](#)
 - [6.1 双重 AEC 问题](#)

- [6.2 WebRTC 的 AEC 策略选择](#)
- [6.3 bypass_voice_processing_ 模式](#)
- [6.4 硬件 AEC 的优势](#)
- [6.5 BuiltInAECIsAvailable 的实现](#)
- [7. 线程模型总结](#)
 - [7.1 线程职责划分](#)
 - [7.2 线程安全机制](#)
 - [7.3 原子变量的使用模式](#)
- [8. 完整生命周期](#)
 - [8.1 初始化流程](#)
 - [8.2 销毁流程](#)
- [9. 总结与设计哲学](#)
 - [9.1 核心设计原则](#)
 - [9.2 与其他平台的对比](#)
 - [9.3 源码文件索引](#)

07_mac_platform

- [macOS 平台音频实现深度分析：AudioDeviceMac](#)
 - [1. 架构概述](#)
 - [核心设计特征](#)
 - [2. 设计参数常量分析](#)
 - [2.1 采样率选择：为什么是 48kHz?](#)
 - [2.2 通道配置：录音单声道 vs 播放立体声](#)
 - [2.3 缓冲策略：10ms + 双缓冲/三缓冲](#)
 - [3. CoreAudio HAL 回调架构](#)
 - [3.1 双回调设计](#)
 - [3.2 回调执行环境](#)
 - [3.3 deviceIOProc 详细分析](#)
 - [3.4 inDeviceIOProc 详细分析](#)
 - [4. 双工作线程设计](#)
 - [4.1 线程声明](#)
 - [4.2 播放工作线程：RenderWorkerThread\(\)](#)
 - [4.3 录音工作线程：CaptureWorkerThread\(\)](#)
 - [4.4 线程同步机制](#)
 - [5. AudioConverter 格式转换机制](#)

- [5.1 为什么需要格式转换?](#)
- [5.2 格式描述符](#)
- [5.3 转换器创建逻辑](#)
- [5.4 转换回调函数](#)
- [5.5 设计决策: 为什么用 AudioConverter 而不是自己实现?](#)
- [6. PaUtilRingBuffer 无锁环形缓冲](#)
 - [6.1 为什么选择 PaUtilRingBuffer?](#)
 - [6.2 环形缓冲区工作原理](#)
 - [6.3 录音方向的 RingBuffer 使用](#)
 - [6.4 播放方向的 RingBuffer 使用](#)
 - [6.5 缓冲区大小与延迟的关系](#)
 - [6.6 Underrun 和 Overrun 处理](#)
- [7. 设备变更监听机制](#)
 - [7.1 CoreAudio 属性监听器](#)
 - [7.2 监听的事件类型](#)
 - [7.3 事件处理函数](#)
 - [7.4 各事件的处理策略](#)
 - [7.5 监听器的生命周期管理](#)
- [8. _twoDevices 模式详解](#)
 - [8.1 模式定义](#)
 - [8.2 为什么需要区分单设备和双设备?](#)
 - [8.3 模式判断逻辑](#)
 - [8.4 单设备模式的数据流](#)
 - [8.5 双设备模式的数据流](#)
 - [8.6 停止控制的差异](#)
 - [8.7 IOProc 注册与启停](#)
- [9. 完整数据流总结](#)
 - [9.1 录音完整路径 \(从硬件到网络\)](#)
 - [9.2 播放完整路径 \(从网络到硬件\)](#)
 - [9.3 关键设计决策总结](#)
 - [9.4 与其他平台的对比](#)
- [10. 源码文件索引](#)

08_windows_platform

- [Windows 平台音频实现深度分析](#)

- [一、两套实现概览](#)
- [二、旧实现：AudioDeviceWindowsCore](#)
 - [2.1 依赖的 Windows API](#)
 - [2.2 静态检测方法](#)
 - [2.3 事件驱动线程模型](#)
- [三、新实现：AudioDeviceModuleWin](#)
 - [3.1 Input/Output 分离架构](#)
 - [3.2 工厂方法与 COM 要求](#)
 - [3.3 CoreAudioInput/Output 自动重启机制](#)
- [四、WASAPI Shared Mode vs Exclusive Mode](#)
- [五、设备选择：kDefaultCommunicationDevice](#)
- [六、线程模型](#)
- [七、常见问题](#)
- [八、源码文件索引](#)

09_linux_platform

- [Linux 平台音频实现深度分析](#)
 - [一、两套实现概览](#)
 - [二、PulseAudio 实现：AudioDeviceLinuxPulse](#)
 - [2.1 延迟常量设计](#)
 - [2.2 异步回调架构](#)
 - [2.3 双工作线程](#)
 - [2.4 播放数据流](#)
 - [2.5 录音数据流](#)
 - [2.6 Underflow 自适应延迟](#)
 - [2.7 动态符号加载](#)
 - [三、ALSA 实现：AudioDeviceLinuxALSA](#)
 - [3.1 同步 PCM 接口](#)
 - [3.2 双工作线程（与 PulseAudio 对称）](#)
 - [3.3 动态符号加载](#)
 - [四、两套实现对比](#)
 - [五、线程模型](#)
 - [六、PaLock / PaUnLock](#)
 - [七、源码文件索引](#)

10_android_platform

- [Android 平台音频实现深度分析](#)
 - [一、三套实现概览](#)
 - [二、架构：Input/Output 分离](#)
 - [三、Java 实现：AudioRecordJni / AudioTrackJni](#)
 - [3.1 Java/Native 边界](#)
 - [3.2 录音数据回调路径](#)
 - [3.3 线程检查器设计](#)
 - [3.4 内置 AEC/NS](#)
 - [四、OpenSL ES 实现](#)
 - [4.1 Buffer Queue 机制](#)
 - [4.2 播放数据流](#)
 - [4.3 低延迟条件](#)
 - [五、AAudio 实现](#)
 - [5.1 AAudio 特性](#)
 - [5.2 数据回调接口](#)
 - [5.3 低延迟性能模式](#)
 - [5.4 自动重启](#)
 - [六、三套实现延迟对比](#)
 - [七、线程模型](#)
 - [八、源码文件索引](#)

11_extension_points

- [音频扩展点概览](#)
 - [一、三个扩展点总览](#)
 - [二、扩展点 1: AudioFrameProcessor](#)
 - [注入方式](#)
 - [在管线中的位置](#)
 - [三、扩展点 2: AudioSinkInterface](#)
 - [注入方式](#)
 - [在管线中的位置](#)
 - [四、扩展点 3: 自定义 AudioMixer](#)
 - [注入方式](#)
 - [五、三个扩展点对比](#)
 - [六、源码文件索引](#)

12_custom_adm

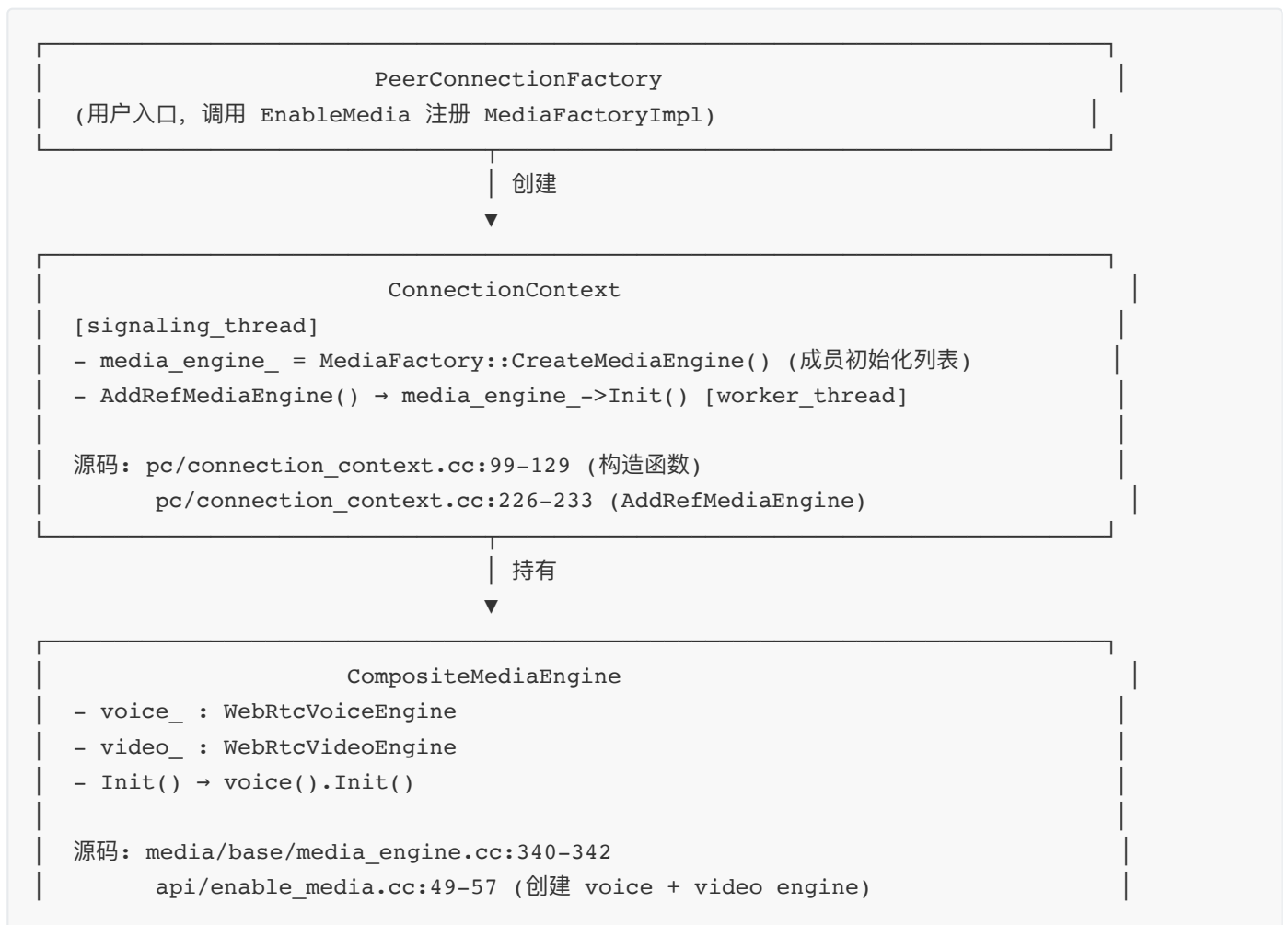
- [自定义 ADM 实现指南](#)
 - [一、为什么需要自定义 ADM](#)
 - [二、最简实现：继承 AudioDeviceModuleDefault](#)
 - [三、推送录音数据（采集侧）](#)
 - [四、拉取播放数据（播放侧）](#)
 - [五、FineAudioBuffer 适配非 10ms buffer](#)
 - [六、必须实现 vs 可用默认值](#)
 - [七、常见陷阱](#)
 - [八、参考实现](#)

WebRTC 音频子系统架构深度分析

基于 WebRTC 源码，所有结论附源码文件及行号引用。

源码路径： `/Volumes/AFreeDisk/peters_projects/webrtc/src`

1. 组件关系全图



包含

WebRtcVoiceEngine

```
[worker_thread_checker_]
- adm_ : AudioDeviceModule (引用计数持有)
- audio_state_ : AudioState
- apm_ : AudioProcessing
- Init(): 初始化 ADM 硬件、设置音频选项、注册 AudioCallback
```

源码: media/engine/webrtc_voice_engine.cc:466-510 (构造)
media/engine/webrtc_voice_engine.cc:522-557 (Init)
media/engine/webrtc_voice_engine.h:149-150 (线程检查器声明)

创建并持有

创建并持有

AudioState

```
[thread_checker_ kDetached]
- config_.audio_device_
  module (ADM引用)
- audio_transport_
- SetPlayout/SetRecording
```

源码: audio/audio_state.cc
:38-45 (构造)
audio/audio_state.h:70-71

AudioDeviceModule (ADM) (AudioDeviceModuleImpl)

```
- audio_device_ : AudioDeviceGeneric
- audio_device_buffer_ : AudioDeviceBuffer
```

源码: modules/audio_device/
audio_device_impl.cc:65-100

平台选择

AudioDeviceGeneric (平台实现)

```
iOS:    AudioDeviceIOS
macOS:  AudioDeviceMac
Win:    AudioDeviceWindowsCore
Linux:  AudioDeviceLinuxPulse (默认)
        AudioDeviceLinuxALSA
Android: 独立工厂方法
```

源码: audio_device_impl.cc:157-260

使用

AudioDeviceBuffer

```
- main_thread_checker_ (kDetached)
- 录音/播放数据中转
- audio_transport_cb_ → AudioTransportImpl
```

源码: modules/audio_device/
audio_device_buffer.cc:51-82
audio_device_buffer.h:158-159

2. 核心类职责一句话总结

类名	职责	源码位置
ConnectionContext	PeerConnectionFactory 的内部上下文，在 signaling_thread 构造，持有 MediaEngine 生命周期	pc/connection_context.cc:99
MediaFactoryImpl	工厂类，实现 CreateMediaEngine() 创建 Voice/Video Engine	api/enable_media.cc:30-59
CompositeMediaEngine	组合模式容器，持有 VoiceEngine + VideoEngine，Init() 委托给 voice	media/base/media_engine.cc:340
WebRtcVoiceEngine	音频引擎核心，管理 ADM、AudioState、APM，负责音频初始化和 codec 管理	media/engine/webrtc_voice_engine.cc:466
AudioState	音频状态管理器，控制播放/录音启停，管理收发流集合，持有 AudioTransport	audio/audio_state.cc:38
AudioTransportImpl	音频数据传输桥梁，实现 RecordedDataIsAvailable / NeedMorePlayData 回调	audio/audio_transport_impl.cc:116,214
AudioDeviceModuleImpl	ADM 实现，平台无关层，管理平台特定设备对象和 AudioDeviceBuffer	modules/audio_device/audio_device_impl.cc:65
AudioDeviceBuffer	音频数据缓冲区，在平台音频线程和 WebRTC 音频处理管线之间传递 PCM 数据	modules/audio_device/audio_device_buffer.cc:51
AudioDeviceGeneric	平台特定音频设备的抽象基类，定义录音/播放硬件操作接口	modules/audio_device/audio_device_generic.h
AudioDeviceIOS	iOS 平台 ADM 实现，基于 Audio Unit (VPIO)	modules/audio_device/ios/
AudioDeviceMac	macOS 平台 ADM 实现，基于 Core Audio HAL	modules/audio_device/mac/
AudioDeviceWindowsCore	Windows 平台 ADM 实现，基于 WASAPI (Core Audio)	modules/audio_device/win/
AudioDeviceLinuxPulse	Linux PulseAudio 实现（默认）	modules/audio_device/linux/
AudioDeviceLinuxALSA	Linux ALSA 实现（备选）	modules/audio_device/linux/
AudioProcessing (APM)	音频处理模块，提供 AEC、AGC、NS 等音频增强算法	modules/audio_processing/
AudioMixer	音频混音器，将多路接收流混合为单路输出	modules/audio_mixer/

3. 线程模型全图



RTC_DCHECK_RUN_ON(worker_thread())



WORKER THREAD

WebRtcVoiceEngine::Init()

- RTC_DCHECK_RUN_ON(&worker_thread_checker_)
[webrtc_voice_engine.cc:523]
- adm_helpers::Init(adm()) → ADM.Init(), 设置设备
[webrtc_voice_engine.cc:531, adm_helpers.cc:36-80]
- ApplyOptions() 配置 AEC/AGC/NS
- adm()->RegisterAudioCallback(audio_state()->audio_transport())
[webrtc_voice_engine.cc:555]

AudioState 所有方法

- RTC_DCHECK_RUN_ON(&thread_checker_)
[audio_state.cc:48,64,84,107,119,139,159]
- thread_checker_ 声明为 kDetached, 绑定到首次调用线程
[audio_state.h:70-71]
- SetPlayout() / SetRecording() / AddReceivingStream() 等
- 通过 ADM 控制硬件启停

AudioDeviceBuffer main_thread_checker_

- 声明为 kDetached [audio_device_buffer.h:158-159]
- 构造时 Detach() [audio_device_buffer.cc:80]
- 绑定到第一次调用的线程 (即 worker_thread 调用 Init 时)
- RegisterAudioCallback / StartPlayout / StartRecording 等
[audio_device_buffer.cc:85,104,115,137,159,174]

ADM 启动硬件后, 平台创建独立音频线程



PLATFORM AUDIO I/O THREADS

(实时线程, 由操作系统音频子系统创建)

Recording Thread (采集)

- 平台实现调用 AudioDeviceBuffer::SetRecordedBuffer()
[audio_device_buffer.cc:260]
- 然后调用 DeliverRecordedData()
[audio_device_buffer.cc:317]
- → AudioTransportImpl::RecordedDataIsAvailable()
[audio_transport_impl.cc:135]
- → APM 处理 → 编码 → 发送

Playout Thread (播放)

- 平台实现调用 AudioDeviceBuffer::GetPlayoutData()
[audio_device_buffer.cc:383]

- → `AudioTransportImpl::NeedMorePlayData()`
[`audio_transport_impl.cc:214`]
- → `AudioMixer` 混音 → `APM` 后处理 → 返回 `PCM` 给硬件

注意：这些线程不受 `WebRTC` 线程模型管理，是 `OS` 级实时线程
`audio_device_buffer.h:149-155` 注释明确说明此设计

3.1 线程边界关键约束

操作	线程	源码依据
<code>ConnectionContext</code> 构造	<code>signaling_thread</code>	<code>connection_context.cc:129</code> <code>RTC_DCHECK_RUN_ON(signaling_thread_)</code>
<code>MediaEngine</code> 对象创建	<code>signaling_thread</code>	<code>connection_context.cc:115-118</code> 成员初始化列表
<code>WebRtcVoiceEngine</code> 对象构造	<code>signaling_thread</code>	同上, <code>enable_media.cc:49</code> 在 <code>CreateMediaEngine</code> 内
<code>MediaEngine::Init()</code>	<code>worker_thread</code>	<code>connection_context.cc:227</code> <code>RTC_DCHECK_RUN_ON(worker_thread())</code>
<code>WebRtcVoiceEngine::Init()</code>	<code>worker_thread</code>	<code>webrtc_voice_engine.cc:523</code> <code>RTC_DCHECK_RUN_ON(&worker_thread_checker_)</code>
<code>AudioState</code> 所有公开方法	<code>worker_thread</code>	<code>audio_state.cc:48,64,84...</code> <code>RTC_DCHECK_RUN_ON(&thread_checker_)</code>
<code>ADM.Init()</code> / <code>SetPlayoutDevice</code>	<code>worker_thread</code>	通过 <code>adm_helpers::Init()</code> 在 <code>VoiceEngine::Init()</code> 中调用
<code>AudioDeviceBuffer</code> 管理方法	<code>worker_thread</code> (首次绑定)	<code>audio_device_buffer.h:158-159</code> <code>kDetached</code>
<code>SetRecordedBuffer</code> / <code>GetPlayoutData</code>	平台音频线程	<code>audio_device_buffer.h:149-155</code> 注释
<code>MediaEngine</code> 析构	<code>worker_thread</code>	<code>connection_context.cc:197-209</code> <code>BlockingCall/PostTask</code>

3.2 线程安全设计模式分析

kDetached 模式： `AudioState::thread_checker_` 和 `AudioDeviceBuffer::main_thread_checker_` 都使用 `SequenceChecker::kDetached` 初始化。这意味着：

1. 对象可以在任意线程构造（如 `signaling_thread`）
2. 首次被某线程调用时，`checker` 绑定到该线程
3. 后续所有调用必须在同一线程

源码依据：

- `audio_state.h:70-71`: `SequenceChecker thread_checker_{SequenceChecker::kDetached};`
- `audio_device_buffer.h:158-159`: `SequenceChecker main_thread_checker_{SequenceChecker::kDetached};`
- `audio_device_buffer.cc:80`: 构造函数中 `main_thread_checker_.Detach();`

这种设计允许对象在 `signaling_thread` 创建，但实际使用绑定到 `worker_thread`。

4. 平台实现选择逻辑

源码位置： `modules/audio_device/audio_device_impl.cc:157-260`

4.1 选择流程

```
AudioDeviceModuleImpl::Create(env, audio_layer)    [line 65]
├─ audio_layer == kAndroid* ?
│   → 返回错误, 提示使用 CreateAndroidAudioDeviceModule() [line 75-83]
├─ audio_layer == kWindowsCoreAudio2 ?
│   → 返回错误, 提示使用 CreateWindowsCoreAudioAudioDeviceModule() [line 70-73]
└─ 正常流程:
    ├─ new AudioDeviceModuleImpl(env, audio_layer) [line 86]
    ├─ CheckPlatform() [line 89]
    ├─ CreatePlatformSpecificObjects(env) [line 94]
    └─ AttachAudioBuffer() [line 100]
```

4.2 CreatePlatformSpecificObjects 平台分支

```
// modules/audio_device/audio_device_impl.cc:157
int32_t AudioDeviceModuleImpl::CreatePlatformSpecificObjects(const Environment& env)
```

Windows (WEBRTC_WIN) — line 181-189

```
条件: audio_layer == kWindowsCoreAudio || kPlatformDefaultAudio
检查: AudioDeviceWindowsCore::CoreAudioIsSupported()
实现: new AudioDeviceWindowsCore()
```

Windows 只支持 WASAPI (Core Audio), 不再支持旧的 Wave Audio API。

Linux (WEBRTC_LINUX, 非 Android) — line 199-228

无 PulseAudio 编译 (!WEBRTC_ENABLE_LINUX_PULSE):

```
kPlatformDefaultAudio → AudioDeviceLinuxALSA    [line 208]
kLinuxAlsaAudio        → AudioDeviceLinuxALSA    [line 207]
kLinuxPulseAudio       → 无效选择
```

有 PulseAudio 编译 (WEBRTC_ENABLE_LINUX_PULSE, 默认):

```
kPlatformDefaultAudio → AudioDeviceLinuxPulse [line 221]
kLinuxPulseAudio      → AudioDeviceLinuxPulse [line 220]
kLinuxAlsaAudio       → AudioDeviceLinuxALSA [line 224]
```

关键注释 (line 195-196):

```
WEBRTC_ENABLE_LINUX_ALSA is always defined by default when WEBRTC_LINUX is defined.
WEBRTC_ENABLE_LINUX_PULSE depends on the 'rtc_include_pulse_audio' build flag.
```

iOS (WEBRTC_IOS) — line 231-239

条件: `audio_layer == kPlatformDefaultAudio`

实现: `std::make_unique<ios_adm::AudioDeviceIOS>(env, false, nullptr, nullptr)`

iOS 实现基于 Audio Unit, 使用 Voice Processing I/O (VPIO) 单元, 内置硬件 AEC 和 NS。

构造参数: `bypass_voice_processing=false` (默认使用 VPIO)。

macOS (WEBRTC_MAC) — line 243-247

条件: `audio_layer == kPlatformDefaultAudio`

实现: `new AudioDeviceMac()`

macOS 使用 Core Audio HAL (Hardware Abstraction Layer) API。

Android — line 75-83

Android 不走 `CreatePlatformSpecificObjects` 流程。

`audio_layer` 为 Android 相关值时, `Create()` 直接返回 `nullptr` 并提示:

"Use the `CreateAndroidAudioDeviceModule()` factory method instead for this option."

Android 使用独立的工厂方法 `CreateAndroidAudioDeviceModule()`, 支持:

- Java AudioTrack/AudioRecord (`kAndroidJavaAudio`)
- OpenSL ES (`kAndroidOpenSLESAudio`)
- AAudio (`kAndroidAAudioAudio`)
- 混合模式 (Java 输入 + OpenSL/AAudio 输出)

Dummy 实现 — line 250-252

条件: `audio_layer == kDummyAudio`

实现: `new AudioDeviceDummy()`

用途: 测试环境, 无真实硬件

4.3 平台选择决策树

```
CreatePlatformSpecificObjects(env)
|
├─ WEBRTC_DUMMY_AUDIO_BUILD defined?
|   └─ YES → AudioDeviceDummy [line 166]
|
├─ WEBRTC_DUMMY_FILE_DEVICES defined?
|   └─ YES → FileAudioDeviceFactory::CreateFileAudioDevice [line 170]
|
└─ 正常平台选择:
    |
    └─ WEBRTC_WIN?
```

```

└─ CoreAudioIsSupported() → AudioDeviceWindowsCore [line 187]
└─ WEBRTC_LINUX (非 Android)?
    └─ WEBRTC_ENABLE_LINUX_PULSE?
        └─ default/pulse → AudioDeviceLinuxPulse [line 221]
        └─ alsa → AudioDeviceLinuxALSA [line 224]
    └─ !PULSE?
        └─ default/alsa → AudioDeviceLinuxALSA [line 208]
└─ WEBRTC_IOS?
    └─ default → AudioDeviceIOS [line 233]
└─ WEBRTC_MAC?
    └─ default → AudioDeviceMac [line 245]
└─ kDummyAudio?
    └─ AudioDeviceDummy [line 251]

```

5. 初始化时序与数据流

5.1 初始化调用链

```

[signaling_thread]
PeerConnectionFactory::Create()
└─ ConnectionContext::Create(env, dependencies) [connection_context.cc:82-86]
    └─ new ConnectionContext(env, dependencies) [connection_context.cc:99]
        └─ 成员初始化列表中:
            └─ media_engine_ = MediaFactory::CreateMediaEngine(env, deps) [line 115-118]
                └─ MediaFactoryImpl::CreateMediaEngine() [enable_media.cc:41-58]
                    └─ new WebRtcVoiceEngine(env, adm, ...) [enable_media.cc:49-52]
                        └─ 构造函数中:
                            └─ adm_ = EnsureAudioDeviceModule(env, adm) [webrtc_voice_engine.cc:480]
                                └─ 若 adm 为空: CreateAudioDeviceModule(env,
kPlatformDefaultAudio) [webrtc_voice_engine.cc:457-458]
                                    └─ audio_state_ = AudioState::Create(config) [webrtc_voice_engine.cc:510]
                                        └─ new WebRtcVideoEngine(...) [enable_media.cc:53-55]
└─ 构造函数体: RTC_DCHECK_RUN_ON(signaling_thread_) [line 129]

```

... 后续创建 PeerConnection 时 ...

```

[worker_thread]
ConnectionContext::AddRefMediaEngine() [connection_context.cc:226]
└─ RTC_DCHECK_RUN_ON(worker_thread_) [line 227]
└─ media_engine_->Init() [line 232]
    └─ CompositeMediaEngine::Init() [media_engine.cc:340]

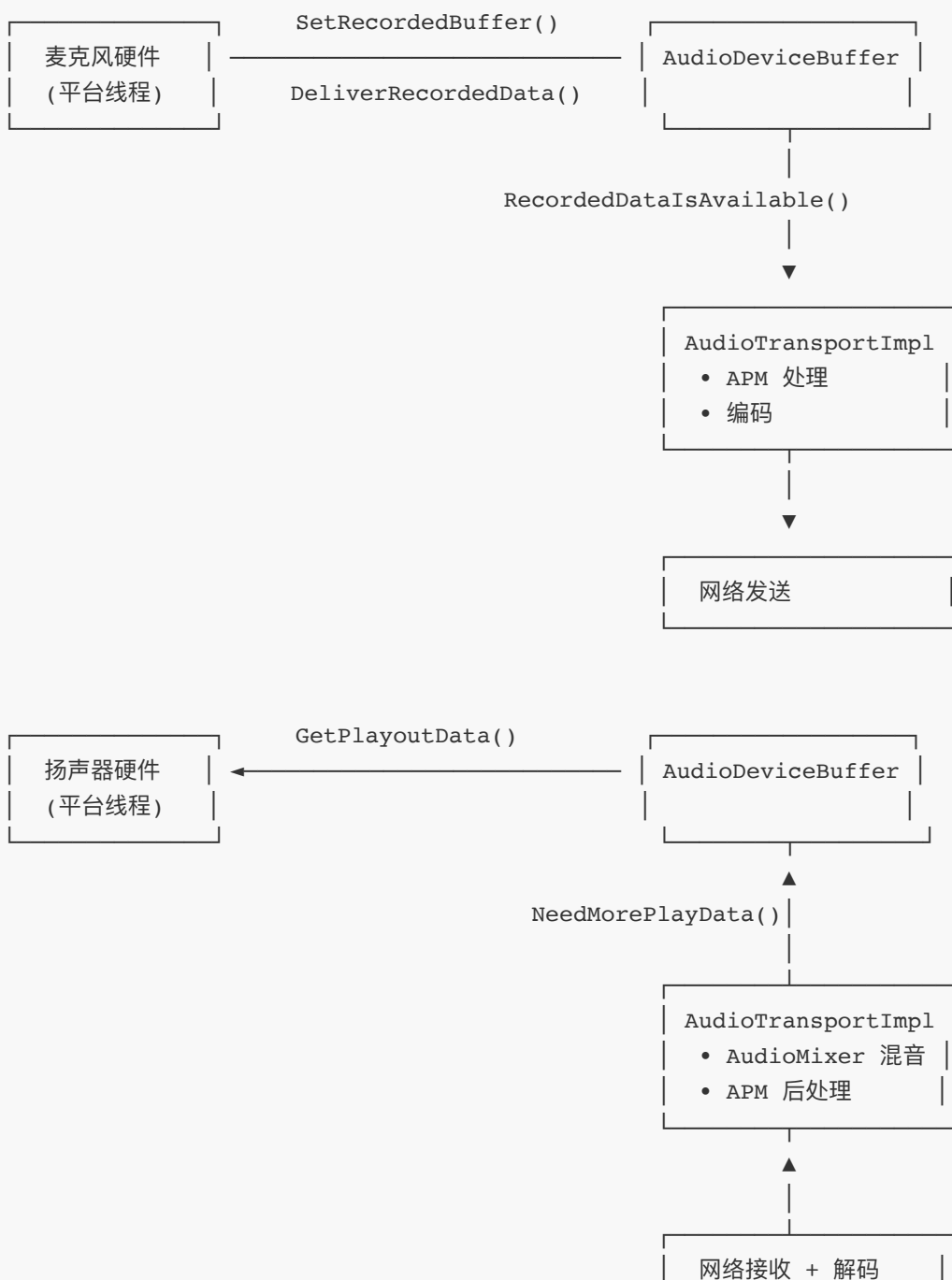
```

```

└─ voice().Init() [media_engine.cc:341]
  └─ WebRtcVoiceEngine::Init() [webrtc_voice_engine.cc:522]
    │ RTC_DCHECK_RUN_ON(&worker_thread_checker_) [line 523]
    └─ adm_helpers::Init(adm()) [line 531]
      │ adm->Init() [adm_helpers.cc:39]
      │ └─ SetPlayoutDevice() [adm_helpers.cc:43]
      │ └─ InitSpeaker() [adm_helpers.cc:47]
      │ └─ SetRecordingDevice() [adm_helpers.cc:63]
      │ └─ InitMicrophone() [adm_helpers.cc:67]
      └─ ApplyOptions(options) [line 549]
        └─ adm()->RegisterAudioCallback( [line 555]
          audio_state()->audio_transport())

```

5.2 音频数据流（运行时）



5.3 关键设计决策分析

为什么对象创建和初始化分离在不同线程？

`ConnectionContext` 构造在 `signaling_thread` (line 129) ，但 `MediaEngine::Init()` 在 `worker_thread` (line 227) 。这是因为：

1. **PeerConnectionFactory 创建是同步的**：用户在应用主线程（通常就是 `signaling_thread`）调用 `Create()`，需要快速返回
2. **ADM 硬件初始化耗时**：`adm->Init()` 涉及系统调用（打开音频设备），不应阻塞 `signaling_thread`
3. **延迟初始化**：`AddRefMediaEngine()` 只在第一个 `PeerConnection` 创建时调用（引用计数从 0 到 1），实现按需初始化

源码证据：

```
// connection_context.cc:231-232
if (media_engine_reference_count_ == 1) {
    media_engine_>Init();
}
```

AudioDeviceBuffer 的 kDetached 设计

`AudioDeviceBuffer::main_thread_checker_` 使用 `kDetached` 是因为：

- 对象随 `AudioDeviceModuleImpl` 在 `signaling_thread` 构造
- 但 `RegisterAudioCallback`、`StartPlayout` 等管理方法在 `worker_thread` 调用
- `kDetached` 允许 checker 绑定到首次调用线程（`worker_thread`）

而录音/播放数据方法（`SetRecordedBuffer`/`GetPlayoutData`）则运行在完全独立的平台音频线程上，这些方法没有 `RTC_DCHECK_RUN_ON` 注解，如 `audio_device_buffer.h:149-155` 注释所述：

"When audio has started some methods will be called on either a native audio thread for playout or a native thread for recording."

WebRTC 音频设备层核心接口深度解析

基于 WebRTC 源码（branch 7680），逐接口、逐参数解析音频设备抽象层的设计意图。

目录

1. [AudioDeviceModule — 公共 API 接口](#)
2. [AudioTransport — 音频数据传输回调](#)

3. [AudioDeviceGeneric — 平台实现基类](#)
4. [AudioDeviceBuffer — 中间缓冲层](#)
5. [三者关系与数据流](#)

1. AudioDeviceModule — 公共 API 接口

源码位置: `api/audio/audio_device.h`

`AudioDeviceModule` (简称 ADM) 是 WebRTC 对外暴露的音频硬件抽象接口。它继承自 `RefCountInterface`, 使用引用计数管理生命周期。所有上层代码 (`WebRtcVoiceEngine`、`PeerConnectionFactory`) 只通过此接口与音频硬件交互。

1.1 AudioLayer 枚举 — 音频后端选择

```
enum AudioLayer {
    kPlatformDefaultAudio = 0,
    kWindowsCoreAudio,
    kWindowsCoreAudio2,
    kLinuxAlsaAudio,
    kLinuxPulseAudio,
    kAndroidJavaAudio,
    kAndroidOpenSLESAudio,
    kAndroidJavaInputAndOpenSLESOutputAudio,
    kAndroidAAudioAudio,
    kAndroidJavaInputAndAAudioOutputAudio,
    kDummyAudio,
};
```

设计意图: 同一平台可能有多种音频后端 (如 Android 上 Java AudioTrack vs OpenSL ES vs AAudio), 此枚举让调用者在创建 ADM 时显式选择。 `kPlatformDefaultAudio` 表示由运行时自动选择最优后端。

调用时机: 在 `CreateAudioDeviceModule(env, audio_layer)` 工厂函数中传入, 决定内部创建哪个 `AudioDeviceGeneric` 子类。

1.2 WindowsDeviceType 枚举

```
enum WindowsDeviceType {
    kDefaultCommunicationDevice = -1,
    kDefaultDevice = -2
};
```

Windows 特有的设备选择方式。 `kDefaultCommunicationDevice` 对应系统"通信设备" (通常是耳机), `kDefaultDevice` 对应系统默认播放设备。非 Windows 平台的实现会忽略此参数。

1.3 方法组一: 生命周期管理

方法	语义	调用时机
<code>Init()</code>	初始化音频子系统，探测可用设备	<code>WebRtcVoiceEngine::Init()</code> 中调用，在 worker thread 上
<code>Terminate()</code>	释放所有音频资源，停止所有流	引擎销毁前调用
<code>Initialized()</code>	查询是否已初始化	各方法前置检查

`Init()` 内部会调用 `AudioDeviceGeneric::Init()`（返回 `InitStatus` 枚举），然后调用 `AttachAudioBuffer()` 将 `AudioDeviceBuffer` 传给平台实现。这是整个音频子系统启动的入口。

1.4 方法组二：音频回调注册

```
virtual int32_t RegisterAudioCallback(AudioTransport* audioCallback) = 0;
```

语义： 注册双向音频数据传输回调。录音数据通过此回调上报，播放数据通过此回调拉取。

调用时机： 在 `Init()` 之后、`StartPlayout()` / `StartRecording()` 之前调用。`AudioState` 在构造时将自身（实现了 `AudioTransport`）注册到 ADM。

设计约束： 只能注册一个回调。如果传 `nullptr`，则音频流仍然运行但数据被丢弃。回调对象的生命周期必须覆盖整个音频活跃期。

1.5 方法组三：设备枚举

```
virtual int16_t PlayoutDevices() = 0;
virtual int16_t RecordingDevices() = 0;
virtual int32_t PlayoutDeviceName(uint16_t index,
                                   char name[kAdmMaxDeviceNameSize],
                                   char guid[kAdmMaxGuidSize]) = 0;
virtual int32_t RecordingDeviceName(uint16_t index,
                                     char name[kAdmMaxDeviceNameSize],
                                     char guid[kAdmMaxGuidSize]) = 0;
```

语义：

- `PlayoutDevices()` / `RecordingDevices()`：返回当前系统可用的播放/录音设备数量。
- `PlayoutDeviceName()` / `RecordingDeviceName()`：根据索引获取设备名称和 GUID。

参数解析：

- `index`：设备索引，从 0 开始，最大值为 `PlayoutDevices() - 1`。
- `name[kAdmMaxDeviceNameSize]`：输出缓冲区，128 字节，存放人类可读的设备名。
- `guid[kAdmMaxGuidSize]`：输出缓冲区，128 字节，存放设备唯一标识符（Windows 为 endpoint ID，macOS 为 UID 字符串）。

调用时机： 通常在 UI 层展示设备列表时调用，或在 `Init()` 后自动选择设备前调用。

1.6 方法组四：设备选择

```
virtual int32_t SetPlayoutDevice(uint16_t index) = 0;
virtual int32_t SetPlayoutDevice(WindowsDeviceType device) = 0;
virtual int32_t SetRecordingDevice(uint16_t index) = 0;
virtual int32_t SetRecordingDevice(WindowsDeviceType device) = 0;
```

语义：选定要使用的播放/录音设备。有两个重载：按索引选择（跨平台）和按 Windows 设备类型选择（Windows 专用）。

调用时机：必须在 `InitPlayout()` / `InitRecording()` 之前调用。一旦音频流启动，切换设备需要先 Stop → 重新 Set → 重新 Init → 重新 Start。

返回值：0 表示成功，-1 表示失败（设备不存在或已被占用）。

1.7 方法组五：音频传输初始化

```
virtual int32_t PlayoutIsAvailable(bool* available) = 0;
virtual int32_t InitPlayout() = 0;
virtual bool PlayoutIsInitialized() const = 0;
virtual int32_t RecordingIsAvailable(bool* available) = 0;
virtual int32_t InitRecording() = 0;
virtual bool RecordingIsInitialized() const = 0;
```

语义：

- `PlayoutIsAvailable()` / `RecordingIsAvailable()`：探测当前选定设备是否可用于播放/录音。
- `InitPlayout()` / `InitRecording()`：为选定设备分配音频缓冲区、配置采样率和通道数。这一步不启动音频流，只是准备资源。
- `PlayoutIsInitialized()` / `RecordingIsInitialized()`：查询初始化状态。

调用时机：在 `SetPlayoutDevice()` 之后、`StartPlayout()` 之前。`InitPlayout()` 内部会配置 CoreAudio (macOS) 或 AudioTrack (Android) 的音频格式参数。

设计意图：将"资源分配"和"流启动"分离，允许上层在启动前检查配置是否合法。

1.8 方法组六：音频传输控制

```
virtual int32_t StartPlayout() = 0;
virtual int32_t StopPlayout() = 0;
virtual bool Playing() const = 0;
virtual int32_t StartRecording() = 0;
virtual int32_t StopRecording() = 0;
virtual bool Recording() const = 0;
```

语义：启动/停止实际的音频 I/O。`StartPlayout()` 后，平台层开始周期性调用 `NeedMorePlayData()` 拉取数据；`StartRecording()` 后，平台层开始周期性调用 `RecordedDataIsAvailable()` 推送数据。

调用时机：

- `StartPlayout()`：当第一个远端音频轨道被添加到 `PeerConnection` 时。
- `StartRecording()`：当本地音频轨道开始采集时。
- `StopPlayout()` / `StopRecording()`：对应的轨道移除或 `PeerConnection` 关闭时。

线程模型：这些方法在 worker thread 上调用，但启动后的回调发生在平台原生音频线程上。

1.9 方法组七：音频混音器初始化

```
virtual int32_t InitSpeaker() = 0;
virtual bool SpeakerIsInitialized() const = 0;
virtual int32_t InitMicrophone() = 0;
virtual bool MicrophoneIsInitialized() const = 0;
```

语义：初始化扬声器/麦克风的音量控制通道。这不是初始化音频流，而是初始化对硬件混音器的访问权限。

调用时机：在需要调节音量之前调用。macOS 实现中，`InitSpeaker()` 会获取 `CoreAudio` 的 `kAudioDevicePropertyVolumeScalar` 属性句柄。

设计意图：音量控制和音频流是独立的子系统。即使没有启动播放，也可以预先初始化扬声器以查询音量范围。

1.10 方法组八：扬声器音量控制

```
virtual int32_t SpeakerVolumeIsAvailable(bool* available) = 0;
virtual int32_t SetSpeakerVolume(uint32_t volume) = 0;
virtual int32_t SpeakerVolume(uint32_t* volume) const = 0;
virtual int32_t MaxSpeakerVolume(uint32_t* maxVolume) const = 0;
virtual int32_t MinSpeakerVolume(uint32_t* minVolume) const = 0;
```

语义：控制系统扬声器硬件音量。音量值的范围由 `MaxSpeakerVolume()` 和 `MinSpeakerVolume()` 定义，不同平台范围不同。

参数解析：

- `volume`：无符号 32 位整数，表示绝对音量值（不是百分比）。macOS 上通常映射到 0-255。
- `available`：输出参数，指示当前设备是否支持音量控制。某些虚拟设备或蓝牙设备可能不支持。

1.11 方法组九：麦克风音量控制

```
virtual int32_t MicrophoneVolumeIsAvailable(bool* available) = 0;
virtual int32_t SetMicrophoneVolume(uint32_t volume) = 0;
virtual int32_t MicrophoneVolume(uint32_t* volume) const = 0;
virtual int32_t MaxMicrophoneVolume(uint32_t* maxVolume) const = 0;
virtual int32_t MinMicrophoneVolume(uint32_t* minVolume) const = 0;
```

语义：与扬声器音量对称，控制麦克风输入增益。AGC（自动增益控制）模块会通过 `AudioTransport::RecordedDataIsAvailable()` 的 `newMicLevel` 输出参数建议新的麦克风音量，上层再调用 `SetMicrophoneVolume()` 应用。

调用时机： AGC 每 10ms 回调一次，但实际调节音量的频率通常被限制（macOS 实现中 `kGetMicVolumeIntervalMs = 1000`，即每秒最多调一次）。

1.12 方法组十：静音控制

```
virtual int32_t SpeakerMuteIsAvailable(bool* available) = 0;
virtual int32_t SetSpeakerMute(bool enable) = 0;
virtual int32_t SpeakerMute(bool* enabled) const = 0;
virtual int32_t MicrophoneMuteIsAvailable(bool* available) = 0;
virtual int32_t SetMicrophoneMute(bool enable) = 0;
virtual int32_t MicrophoneMute(bool* enabled) const = 0;
```

语义： 硬件级静音控制。与将音量设为 0 不同，硬件静音通常会在驱动层切断信号路径。

注意： 不是所有平台都支持硬件静音。`AudioDeviceModuleDefault` 模板中这些方法默认返回 0（成功）但不做任何事。

1.13 方法组十一：立体声支持

```
virtual int32_t StereoPlayOutIsAvailable(bool* available) const = 0;
virtual int32_t SetStereoPlayOut(bool enable) = 0;
virtual int32_t StereoPlayOut(bool* enabled) const = 0;
virtual int32_t StereoRecordingIsAvailable(bool* available) const = 0;
virtual int32_t SetStereoRecording(bool enable) = 0;
virtual int32_t StereoRecording(bool* enabled) const = 0;
```

语义： 查询和配置立体声模式。

调用时机： 必须在 `InitPlayOut()` / `InitRecording()` 之前调用 `SetStereoPlayOut()` / `SetStereoRecording()`。一旦初始化完成，通道数就固定了。

设计意图： WebRTC 默认使用单声道录音（节省带宽），但播放默认立体声。macOS 实现中 `N_REC_CHANNELS = 1`，`N_PLAY_CHANNELS = 2`。

1.14 方法组十二：延迟信息

```
virtual int32_t PlayOutDelay(uint16_t* delayMS) const = 0;
```

语义： 获取当前播放延迟（毫秒）。这个值加上录音延迟，构成 `totalDelayMS` 传给 AEC。

调用时机： 每次录音回调时，平台层会查询当前播放延迟，与录音延迟相加后传给 `RecordedDataIsAvailable()`。

精度要求： AEC 对延迟估计非常敏感。误差超过 $\pm 20\text{ms}$ 会显著降低回声消除效果。macOS 实现通过 CoreAudio 的 `kAudioDevicePropertyLatency` + `kAudioDevicePropertySafetyOffset` + buffer duration 计算。

1.15 方法组十三：平台内置音频处理

```
virtual bool BuiltInAECIsAvailable() const = 0;
virtual bool BuiltInAGCIsAvailable() const = 0;
virtual bool BuiltInNSIsAvailable() const = 0;
virtual int32_t EnableBuiltInAEC(bool enable) = 0;
virtual int32_t EnableBuiltInAGC(bool enable) = 0;
virtual int32_t EnableBuiltInNS(bool enable) = 0;
```

语义：查询和启用平台硬件/系统级的回声消除（AEC）、自动增益控制（AGC）、噪声抑制（NS）。

平台差异：

- **Android**：支持硬件 AEC/AGC/NS（通过 `android.media.audiofx` API）。
- **iOS**：通过 Audio Unit 的 `kAudioUnitSubType_VoiceProcessingIO` 提供。
- **macOS/Windows/Linux**：默认返回 `false`，使用 WebRTC 软件实现。

设计意图：当平台内置处理可用且质量足够时，可以关闭 WebRTC 的软件 APM，减少 CPU 开销和延迟。

1.16 方法组十四：iOS 专有

```
#if defined(WEBRTC_IOS)
virtual int GetPlayoutAudioParameters(AudioParameters* params) const = 0;
virtual int GetRecordAudioParameters(AudioParameters* params) const = 0;
#endif
```

语义：获取 iOS 音频会话协商后的实际音频参数（采样率、通道数、缓冲区大小）。

设计意图：iOS 的 Audio Session 会根据当前路由（扬声器/耳机/蓝牙）动态调整参数。上层需要知道实际参数以正确配置编解码器。

1.17 Stats 结构体

```
struct Stats {
    double synthesized_samples_duration_s = 0;
    uint64_t synthesized_samples_events = 0;
    double total_samples_duration_s = 0;
    double total_playout_delay_s = 0;
    uint64_t total_samples_count = 0;
};
```

语义：用于生成 WebRTC Stats API 中的 `RTCAudioPlayoutStats`。`synthesized_samples` 表示因数据不足而合成（静音或重复）的样本，是衡量播放质量的关键指标。

2. AudioTransport — 音频数据传输回调

源码位置：`api/audio/audio_device_defines.h`

`AudioTransport` 是连接平台音频层和 WebRTC 引擎的桥梁。它定义了两个核心回调方法：一个用于录音数据上报（push 模型），一个用于播放数据拉取（pull 模型）。

2.1 设计哲学

WebRTC 的音频数据流采用 **10ms 帧** 为基本单位。无论平台原生缓冲区多大（macOS CoreAudio 通常 128/256/512 samples），`AudioDeviceBuffer` 都会将其切分或拼接为 10ms 帧后再调用 `AudioTransport`。

这个设计有几个原因：

1. **AEC 需要对齐的帧**：回声消除算法要求录音和播放数据严格按 10ms 对齐。
2. **编码器输入**：Opus 编码器接受 10ms/20ms/40ms/60ms 帧，10ms 是最小公倍数。
3. **延迟可控**：10ms 是实时通信可接受的最小处理延迟。

2.2 RecordedDataIsAvailable — 录音数据上报

```
virtual int32_t RecordedDataIsAvailable(  
    const void* audioSamples,  
    size_t nSamples,  
    size_t nBytesPerSample,  
    size_t nChannels,  
    uint32_t samplesPerSec,  
    uint32_t totalDelayMS,  
    int32_t clockDrift,  
    uint32_t currentMicLevel,  
    bool keyPressed,  
    uint32_t& newMicLevel,  
    std::optional<int64_t> estimatedCaptureTimeNS) = 0;
```

参数逐字段解析

`const void* audioSamples`

- **类型**：指向原始 PCM 数据的指针
- **格式**：16-bit 有符号整数（`int16_t`），交错（interleaved）排列
- **交错含义**：对于立体声，数据排列为 `[L0, R0, L1, R1, L2, R2, ...]`
- **为什么用 `void*`**：历史原因，早期设计考虑过支持其他位深（如 32-bit float），但实际上 WebRTC 内部始终使用 16-bit PCM
- **生命周期**：指针仅在回调期间有效，回调返回后数据可能被覆盖

`size_t nSamples`

- **含义**：每通道的采样数（不是总采样数）
- **典型值**：`samplesPerSec / 100`，即 10ms 对应的采样数
 - 48000 Hz → 480 samples
 - 44100 Hz → 441 samples
 - 16000 Hz → 160 samples
 - 8000 Hz → 80 samples
- **为什么是 10ms**：WebRTC 音频处理管线的基本时间单位。`AudioDeviceBuffer` 保证每次回调恰好 10ms

数据

- 数据总字节数: `nSamples * nBytesPerSample`

`size_t nBytesPerSample`

- 含义: 每个采样点的字节数, 包含所有通道
- 计算公式: `nChannels * sizeof(int16_t) = nChannels * 2`
- 典型值: 单声道 = 2, 立体声 = 4
- 注意: 这个命名容易误导。它不是"每个通道每个采样的字节数", 而是"一个采样时刻所有通道的总字节数"。这是 WebRTC 的历史命名约定
- 用途: 计算缓冲区总大小 = `nSamples * nBytesPerSample`

`size_t nChannels`

- 含义: 音频通道数
- 取值: 1 (单声道) 或 2 (立体声)
- 默认: WebRTC 录音默认单声道 (`N_REC_CHANNELS = 1`), 除非显式调用 `SetStereoRecording(true)`
- 为什么默认单声道: 语音通信场景下, 单声道足够且节省 50% 带宽

`uint32_t samplesPerSec`

- 含义: 采样率, 单位 Hz
- 支持的值: 8000, 16000, 32000, 44100, 48000
- 典型值: macOS/iOS 通常 48000, Android 因设备而异 (44100 或 48000)
- 为什么不统一: 不同平台的原生采样率不同, 强制重采样会引入额外延迟和失真。WebRTC 在 APM 内部处理重采样

`uint32_t totalDelayMS`

- 含义: 录音延迟 + 播放延迟的总和, 单位毫秒
- 计算: `play_delay_ms + rec_delay_ms` (在 `AudioDeviceBuffer::SetVQEData()` 中设置)
- 用途: 提供给 AEC (回声消除) 算法。AEC 需要知道从扬声器播出到麦克风采集到的总延迟, 才能在录音信号中找到对应的回声位置
- 精度要求: $\pm 20\text{ms}$ 以内。误差过大会导致 AEC 收敛慢或完全失效
- macOS 实现: 通过 `_renderDelayUs + _captureDelayUs` 计算, 包含硬件延迟 + 安全偏移 + 缓冲区延迟

`int32_t clockDrift`

- 含义: 录音和播放时钟之间的漂移量
- 当前状态: 已废弃, 始终传 0
- 历史背景: 早期设计中, 当录音和播放使用不同硬件设备时 (如 USB 麦克风 + 内置扬声器), 两者的时钟可能有微小漂移。现代 AEC 算法已内置漂移补偿, 不再需要外部提供

`uint32_t currentMicLevel`

- **含义：** 当前麦克风硬件音量级别
- **范围：** 0-255（归一化到 8-bit 范围）
- **用途：** 提供给 AGC（自动增益控制）算法作为输入。AGC 需要知道当前硬件增益才能计算需要多少软件增益补偿
- **获取方式：** 平台层定期调用 `MicrophoneVolume()` 获取（macOS 限制为每秒一次）

`bool keyPressed`

- **含义：** 当前是否检测到键盘按键
- **用途：** 用于 **键盘噪音检测**。当用户打字时，机械振动会被麦克风采集到。APM 可以利用此信息增强噪声抑制
- **macOS 实现：** 通过 `CGEventSourceKeyState()` 检测按键状态，扫描所有按键码（0x00-0x5C）
- **隐私考虑：** 只检测"是否有按键"，不记录具体按了哪个键

`uint32_t& newMicLevel` [输出参数]

- **含义：** AGC 建议的新麦克风音量
- **范围：** 0-255
- **行为：** 如果 AGC 认为需要调整音量，会设置此值。调用者（`AudioDeviceBuffer`）读取后调用 `SetMicrophoneVolume()` 应用
- **为什么是引用：** 这是一个 in-out 参数。输入时可能携带当前值，输出时携带建议值
- **节流：** 实际音量调整会被节流（如每秒一次），避免频繁调节导致音量抖动

`std::optional<int64_t> estimatedCaptureTimeNS` [可选参数]

- **含义：** 音频帧的估计采集时间戳，单位纳秒
- **时钟基准：** 系统单调时钟（`rtc::TimeNanos()`）
- **用途：** 用于 **音视频同步（A/V Sync）**。视频帧有 NTP 时间戳，音频帧也需要精确时间戳才能对齐
- **为什么是 optional：** 不是所有平台都能提供精确的采集时间戳。Android 的 `AudioRecord` 可以提供，macOS 的 CoreAudio 通过 `AudioTimeStamp` 提供
- **新旧 API：** 这是新版 API（带 `estimatedCaptureTimeNS`），旧版不含此参数。新版默认调用旧版，直到所有实现迁移完成

返回值

- `0`：成功处理
- 非零：错误（实际上当前实现总是返回 0）

2.3 NeedMorePlayData — 播放数据拉取


```
virtual int32_t NeedMorePlayData(size_t nSamples,
                                size_t nBytesPerSample,
                                size_t nChannels,
                                uint32_t samplesPerSec,
                                void* audioSamples,
                                size_t& nSamplesOut,
                                int64_t* elapsed_time_ms,
                                int64_t* ntp_time_ms) = 0;
```

参数逐字段解析

`size_t nSamples`

- 含义：请求的每通道采样数
- 典型值：`samplesPerSec / 100` (10ms)
- 约束：调用者保证 `audioSamples` 缓冲区至少有 `nSamples * nBytesPerSample` 字节空间
- 与录音的对称性：播放和录音使用相同的 10ms 帧大小，确保 AEC 能正确对齐

`size_t nBytesPerSample`

- 含义：每采样点字节数（包含所有通道）
- 计算：`nChannels * 2` (16-bit PCM)
- 用途：与 `nSamples` 一起确定输出缓冲区大小

`size_t nChannels`

- 含义：请求的通道数
- 典型值：2（立体声播放）
- 注意：如果混音器输出是单声道但设备要求立体声，`AudioTransport` 实现会做上混（mono → stereo 复制）

`uint32_t samplesPerSec`

- 含义：请求的采样率
- 约束：必须与 `InitPlayOut()` 时配置的采样率一致
- 重采样：如果混音器输出采样率与设备不同，`AudioTransport` 内部会做重采样

`void* audioSamples` [输出参数]

- 含义：输出缓冲区，调用者分配，被调用者填充
- 格式：16-bit PCM 交错数据，与录音格式相同
- 大小：至少 `nSamples * nBytesPerSample` 字节
- 填充要求：实现必须填充完整的 `nSamples` 数据。如果没有足够数据，应填充静音（零）

`size_t& nSamplesOut` [输出参数]

- 含义：实际填充的每通道采样数
- 正常情况：等于 `nSamples`

- **异常情况：** 如果内部没有数据可播放，可能返回 0 或小于请求值
- **为什么需要：** 允许实现在特殊情况下（如静音检测、DTX）返回不同长度的数据

`int64_t* elapsed_time_ms` [输出参数]

- **含义：** 音频帧的 elapsed time（经过时间），单位毫秒
- **时钟基准：** 从某个固定起点开始的单调递增时间
- **用途：** 用于 jitter buffer 的时间管理和统计
- **实现：** 由 `AudioMixer` 在混音时设置

`int64_t* ntp_time_ms` [输出参数]

- **含义：** 音频帧对应的 NTP 时间戳，单位毫秒
- **用途：** 音视频同步的关键。视频渲染器通过比较音频和视频的 NTP 时间戳来决定何时渲染视频帧
- **来源：** 从 RTP 包的 RTCP SR（Sender Report）中推算得到
- **精度：** 毫秒级，对于 A/V 同步足够（人类感知阈值约 $\pm 80\text{ms}$ ）

返回值

- `0`：成功
- 非零：错误

调用时序

平台音频线程（每10ms）：

```
AudioDeviceBuffer::RequestPlayoutData(480)
  → AudioTransport::NeedMorePlayData(480, 4, 2, 48000, buf, ...)
  → AudioMixer::Mix() → 混合所有活跃音频源
  → 重采样（如需要）
  → 填充 buf
  ← 返回
AudioDeviceBuffer::GetPlayoutData(outputBuf)
  → 将数据拷贝到平台输出缓冲区
```

2.4 PullRenderData — 渲染数据拉取（辅助方法）

```
virtual void PullRenderData(int bits_per_sample,
                           int sample_rate,
                           size_t number_of_channels,
                           size_t number_of_frames,
                           void* audio_data,
                           int64_t* elapsed_time_ms,
                           int64_t* ntp_time_ms) = 0;
```

语义： 拉取混合后的渲染数据，但不将其传给音频处理模块作为参考信号。

用途： 当需要获取远端音频数据但不希望影响 AEC 参考信号时使用（如外部录制场景）。

与 `NeedMorePlayData` 的区别：`NeedMorePlayData` 拉取的数据会同时作为 AEC 的远端参考信号；`PullRenderData` 只拉取数据，不更新 AEC 状态。

3. AudioDeviceGeneric — 平台实现基类

源码位置：`modules/audio_device/audio_device_generic.h`

`AudioDeviceGeneric` 是所有平台音频设备实现的基类。它不是 `AudioDeviceModule` 的子类，而是被 `AudioDeviceModuleImpl` 内部持有（组合模式）。每个平台提供一个具体实现：

平台	实现类	源码位置
macOS	<code>AudioDeviceMac</code>	<code>modules/audio_device/mac/audio_device_mac.h</code>
iOS	<code>AudioDeviceIOS</code>	<code>modules/audio_device/ios/audio_device_ios.h</code>
Android	<code>AudioDeviceTemplate<InputType, OutputType></code>	<code>modules/audio_device/android/</code>
Windows	<code>AudioDeviceWindowsCore</code>	<code>modules/audio_device/win/</code>
Linux	<code>AudioDeviceLinuxALSA</code> / <code>AudioDeviceLinuxPulse</code>	<code>modules/audio_device/linux/</code>

3.1 与 AudioDeviceModule 的关键差异

虽然 `AudioDeviceGeneric` 的方法签名与 `AudioDeviceModule` 几乎一一对应，但有几个重要区别：

3.1.1 Init() 返回 InitStatus 枚举

```
enum class InitStatus {
    OK = 0,
    PLAYOUT_ERROR = 1,
    RECORDING_ERROR = 2,
    OTHER_ERROR = 3,
    NUM_STATUSES = 4
};

virtual InitStatus Init() = 0;
```

与 ADM 的区别：`AudioDeviceModule::Init()` 返回 `int32_t`（0 成功，-1 失败），而 `AudioDeviceGeneric::Init()` 返回 `InitStatus` 枚举。

设计意图：`InitStatus` 用于 **UMA (User Metrics Analysis)** 统计。Chrome 通过 `UMA_HISTOGRAM_ENUMERATION` 记录初始化失败的具体原因，帮助开发团队了解不同平台的失败分布。`AudioDeviceModuleImpl::Init()` 内部调用 `AudioDeviceGeneric::Init()`，将枚举结果转换为 `int32_t` 返回给上层。

枚举值含义：

- `OK`：初始化成功
- `PLAYOUT_ERROR`：播放设备初始化失败（如设备被占用）
- `RECORDING_ERROR`：录音设备初始化失败（如权限被拒绝）
- `OTHER_ERROR`：其他错误（如内存分配失败）

- `NUM_STATUSES`：枚举计数，用于 UMA 直方图的 bucket 数

3.1.2 参数传递方式不同

`AudioDeviceGeneric` 使用 引用 作为输出参数：

```
// AudioDeviceGeneric (引用)
virtual int32_t PlayoutIsAvailable(bool& available) = 0;
virtual int32_t SpeakerVolume(uint32_t& volume) const = 0;

// AudioDeviceModule (指针)
virtual int32_t PlayoutIsAvailable(bool* available) = 0;
virtual int32_t SpeakerVolume(uint32_t* volume) const = 0;
```

原因：`AudioDeviceGeneric` 是内部接口，使用引用更简洁且不需要空指针检查。`AudioDeviceModule` 是公共 API，使用指针符合 Google C++ 风格指南（输出参数用指针）。`AudioDeviceModuleImpl` 在两者之间做转换。

3.2 AttachAudioBuffer — 核心连接点

```
virtual void AttachAudioBuffer(AudioDeviceBuffer* audioBuffer) = 0;
```

语义：将 `AudioDeviceBuffer` 指针传给平台实现。这是平台层获取缓冲区对象的唯一方式。

调用时机：在 `AudioDeviceModuleImpl::Init()` 中，创建完平台实现后立即调用：

```
// AudioDeviceModuleImpl::Init() 内部逻辑（简化）
int32_t AudioDeviceModuleImpl::Init() {
    // 1. 创建平台实现
    CreatePlatformSpecificObjects(env); // 创建 audio_device_

    // 2. 连接缓冲区
    AttachAudioBuffer(); // 调用 audio_device_->AttachAudioBuffer(&audio_device_buffer_)

    // 3. 初始化平台实现
    auto status = audio_device_->Init();
    // ...
}
```

平台实现中的使用：平台实现保存此指针，在音频回调中使用：

```
// macOS 实现示例（简化）
void AudioDeviceMac::AttachAudioBuffer(AudioDeviceBuffer* audioBuffer) {
    _ptrAudioBuffer = audioBuffer;
    // 配置采样率和通道数
    _ptrAudioBuffer->SetRecordingSampleRate(N_REC_SAMPLES_PER_SEC);
    _ptrAudioBuffer->SetPlayoutSampleRate(N_PLAY_SAMPLES_PER_SEC);
    _ptrAudioBuffer->SetRecordingChannels(N_REC_CHANNELS);
    _ptrAudioBuffer->SetPlayoutChannels(N_PLAY_CHANNELS);
}
```

生命周期约束： `AudioDeviceBuffer` 由 `AudioDeviceModuleImpl` 拥有，其生命周期覆盖整个 ADM 生命周期。平台实现不需要管理此指针的释放。

3.3 BuiltInAEC/AGC/NS — 平台音频处理能力声明

```
// audio_device_generic.cc 中的默认实现
bool AudioDeviceGeneric::BuiltInAECIsAvailable() const { return false; }
int32_t AudioDeviceGeneric::EnableBuiltInAEC(bool enable) { return -1; }
bool AudioDeviceGeneric::BuiltInAGCIsAvailable() const { return false; }
int32_t AudioDeviceGeneric::EnableBuiltInAGC(bool enable) { return -1; }
bool AudioDeviceGeneric::BuiltInNSIsAvailable() const { return false; }
int32_t AudioDeviceGeneric::EnableBuiltInNS(bool enable) { return -1; }
```

设计模式： 基类提供默认实现（返回 false / -1），表示“不支持”。只有特定平台覆盖这些方法。

Android 覆盖示例：

- `OpenSLESRecorder` 会检查 `SLAndroidConfigurationItf` 是否支持 AEC
- `AAudioRecorder` 通过 `AAudioStreamBuilder_setPerformanceMode()` 启用

为什么不是纯虚函数： 大多数平台不支持内置音频处理，如果是纯虚函数，每个平台都要写一个空实现。使用默认实现减少样板代码。

3.4 GetPlayoutUnderrunCount

```
int32_t AudioDeviceGeneric::GetPlayoutUnderrunCount() const { return -1; }
```

语义： 获取播放欠载（underrun）次数。欠载表示播放缓冲区为空，硬件没有数据可播放。

平台支持： 主要在 Android 上有意义（通过 `AudioTrack.getUnderrunCount()`）。其他平台返回 -1 表示不支持。

3.5 iOS 专有方法

```
#if defined(WEBRTC_IOS)
int AudioDeviceGeneric::GetPlayoutAudioParameters(AudioParameters* params) const {
    return -1;
}
int AudioDeviceGeneric::GetRecordAudioParameters(AudioParameters* params) const {
    return -1;
}
#endif
```

条件编译： 这些方法只在 iOS 构建中存在。基类提供返回 -1 的默认实现，`AudioDeviceIOS` 覆盖它们返回实际参数。

AudioParameters 类的关键字段：

- `sample_rate_`：采样率（iOS 通常 48000 或 44100）
- `channels_`：通道数

- `frames_per_buffer_`: 原生缓冲区大小 (iOS 通常 256 或 512 frames)
- `frames_per_10ms_buffer_`: 10ms 对应的帧数 (`sample_rate / 100`)

3.6 平台实现的典型生命周期

以 macOS `AudioDeviceMac` 为例:

```

1. 构造 AudioDeviceMac()
2. AttachAudioBuffer(buffer) ← 保存 buffer 指针, 配置采样率/通道数
3. Init() ← 枚举 CoreAudio 设备, 注册设备变更监听
4. SetPlayoutDevice(0) ← 选择默认输出设备
5. SetRecordingDevice(0) ← 选择默认输入设备
6. InitPlayout() ← 创建 AudioConverter, 配置输出格式
7. InitRecording() ← 创建 AudioConverter, 配置输入格式
8. StartPlayout() ← 注册 IOProc, 启动渲染线程
9. StartRecording() ← 注册 IOProc, 启动采集线程
   ... 音频流运行中 ...
   每 10ms:
       采集线程: buffer->SetRecordedBuffer() → buffer->DeliverRecordedData()
       渲染线程: buffer->RequestPlayoutData() → buffer->GetPlayoutData()
10. StopRecording() ← 停止采集线程, 注销 IOProc
11. StopPlayout() ← 停止渲染线程, 注销 IOProc
12. Terminate() ← 释放 AudioConverter, 注销设备监听

```

4. AudioDeviceBuffer — 中间缓冲层

源码位置: `modules/audio_device/audio_device_buffer.h`

`AudioDeviceBuffer` 是平台音频层和 `AudioTransport` 之间的中间人。它负责:

1. 将平台原生缓冲区大小转换为 10ms 帧
2. 收集 VQE (Voice Quality Enhancement) 参数 (延迟、音量)
3. 调用 `AudioTransport` 的回调方法
4. 统计音频质量指标

4.1 录音路径 API

```

virtual int32_t SetRecordedBuffer(const void* audio_buffer,
                                size_t samples_per_channel,
                                std::optional<int64_t> capture_timestamp_ns);
virtual void SetVQEData(int play_delay_ms, int rec_delay_ms);
virtual int32_t DeliverRecordedData();
uint32_t NewMicLevel() const;

```

调用顺序 (平台层每次录音回调):

1. `SetRecordedBuffer(data, 480, timestamp)` — 存储原始 PCM 数据

2. `SetVQEData(play_delay, rec_delay)` — 设置延迟信息
3. `DeliverRecordedData()` — 触发 `AudioTransport::RecordedDataIsAvailable()`
4. `NewMicLevel()` — 读取 AGC 建议的新音量

设计意图：将"数据准备"和"数据投递"分离。平台层可以在不同时机获取延迟信息和音频数据，最后统一投递。

4.2 播放路径 API

```
virtual int32_t RequestPlayoutData(size_t samples_per_channel);  
virtual int32_t GetPlayoutData(void* audio_buffer);
```

调用顺序（平台层每次播放回调）：

1. `RequestPlayoutData(480)` — 触发 `AudioTransport::NeedMorePlayData()`，数据存入内部 `play_buffer_`
2. `GetPlayoutData(output)` — 将 `play_buffer_` 拷贝到平台输出缓冲区

为什么分两步：某些平台（如 macOS CoreAudio 的 converter proc）需要在回调中多次拉取小块数据。分两步允许先请求再按需获取。

4.3 关键内部状态

```
AudioTransport* audio_transport_cb_; // 注册的回调对象  
std::atomic<uint32_t> rec_sample_rate_; // 录音采样率  
std::atomic<uint32_t> play_sample_rate_; // 播放采样率  
std::atomic<size_t> rec_channels_; // 录音通道数  
std::atomic<size_t> play_channels_; // 播放通道数  
int play_delay_ms_; // 播放延迟  
int rec_delay_ms_; // 录音延迟  
bool typing_status_; // 键盘输入状态
```

原子变量的使用：采样率和通道数可能在线程设置，在音频线程读取，使用 `std::atomic` 避免数据竞争。

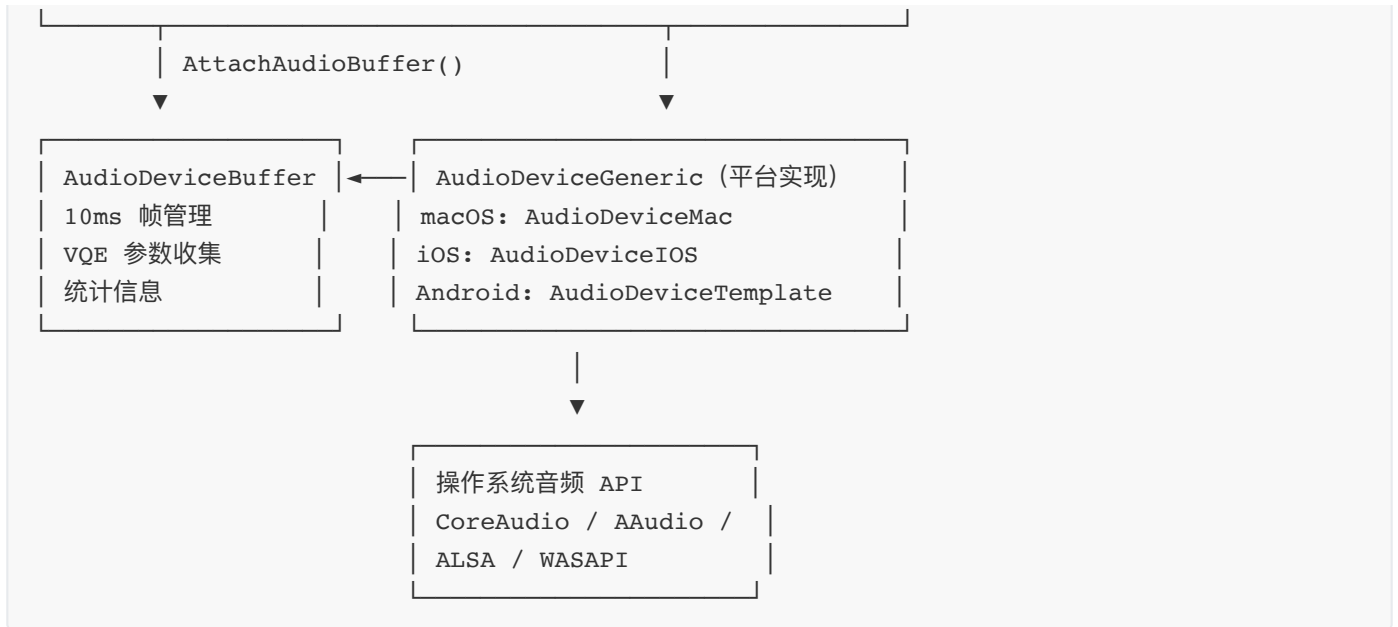
5. 三者关系与数据流

5.1 架构层次

上层 (WebRtcVoiceEngine / AudioState)
实现 AudioTransport 接口

RegisterAudioCallback()

AudioDeviceModuleImpl
持有 AudioDeviceBuffer + AudioDeviceGeneric
对外暴露 AudioDeviceModule 接口



5.2 录音数据流（完整路径）

1. 硬件麦克风采集模拟信号
2. ADC 转换为数字信号
3. OS 音频驱动将数据送入应用缓冲区
4. CoreAudio IOProc 回调触发 (macOS, 可能不是 10ms 对齐)
5. AudioDeviceMac::implInDeviceIOProc()
 - 格式转换 (float32 → int16, 重采样)
 - 写入 ring buffer
6. AudioDeviceMac::CaptureWorkerThread() (每 10ms 唤醒)
 - 从 ring buffer 读取 10ms 数据
 - `_ptrAudioBuffer->SetRecordedBuffer(data, 480, timestamp)`
 - `_ptrAudioBuffer->SetVQEData(play_delay, rec_delay)`
 - `_ptrAudioBuffer->DeliverRecordedData()`
7. AudioDeviceBuffer::DeliverRecordedData()
 - `audio_transport_cb_->RecordedDataIsAvailable(audioSamples, 480, 2, 1, 48000, totalDelayMS, 0, currentMicLevel, keyPressed, newMicLevel, captureTimestamp)`
8. AudioTransport 实现 (AudioState/AudioTransportImpl)
 - APM 处理 (AEC、AGC、NS)
 - 编码器输入队列

5.3 播放数据流（完整路径）

1. 网络接收 RTP 包
2. 解码器输出 PCM 到 jitter buffer
3. CoreAudio IOProc 回调触发 (需要输出数据)
4. AudioDeviceMac::RenderWorkerThread() (每 10ms 唤醒)
 - `_ptrAudioBuffer->RequestPayoutData(480)`
5. AudioDeviceBuffer::RequestPayoutData()
 - `audio_transport_cb_->NeedMorePlayData(480, 4, 2, 48000, play_buffer_,`

```

        nSamplesOut, &elapsed_time_ms, &ntp_time_ms)
6. AudioTransport 实现
    → AudioMixer::Mix() 混合所有远端音频源
    → 重采样到设备采样率
    → 填充输出缓冲区
7. AudioDeviceMac::RenderWorkerThread()
    → _ptrAudioBuffer->GetPlayOutData(renderBuf)
    → 写入 ring buffer
8. CoreAudio IOProc
    → 从 ring buffer 读取数据
    → 格式转换 (int16 → float32)
    → 送入 DAC → 扬声器播放

```

5.4 线程模型

线程	职责	涉及的类
Worker Thread	ADM 生命周期管理 (Init/Start/Stop)	AudioDeviceModuleImpl
Capture Thread	10ms 录音数据投递	AudioDeviceMac, AudioDeviceBuffer
Render Thread	10ms 播放数据拉取	AudioDeviceMac, AudioDeviceBuffer
CoreAudio IOProc Thread	硬件中断回调, 数据搬运	AudioDeviceMac (ring buffer 读写)

关键约束:

- `AudioTransport` 回调发生在 Capture/Render 线程上, 不是 Worker Thread
- 回调中不能做阻塞操作 (如锁等待、内存分配), 否则会导致音频卡顿
- `AudioDeviceBuffer` 的 `play_buffer_` 和 `rec_buffer_` 在不同线程访问, 通过原子操作和设计约束 (单写单读) 保证安全

5.5 AudioDeviceModuleDefault 模板

源码位置: `modules/audio_device/include/audio_device_default.h`

```

template <typename T>
class AudioDeviceModuleDefault : public T {
    // 所有方法返回 0 (成功) 或 false (不支持)
};

```

用途: 为测试和 mock 提供便利。继承此模板后只需覆盖关心的方法:

```

class MyTestADM : public AudioDeviceModuleDefault<AudioDeviceModule> {
    // 只覆盖需要测试的方法
    int32_t StartRecording() override { /* test logic */ }
};

```

与 `AudioDeviceGeneric` 的区别:

- `AudioDeviceModuleDefault` 是 `AudioDeviceModule` 的默认实现（公共 API 层）
- `AudioDeviceGeneric` 是内部平台实现基类（内部 API 层）
- 两者不在同一继承链上

附录：关键常量

源码位置： `api/audio/audio_device_defines.h`

常量	值	含义
<code>kAdmMaxDeviceNameSize</code>	128	设备名最大长度
<code>kAdmMaxFileNameSize</code>	512	文件名最大长度
<code>kAdmMaxGuidSize</code>	128	设备 GUID 最大长度
<code>kAdmMinPlayoutBufferSizeMs</code>	10	最小播放缓冲区（ms）
<code>kAdmMaxPlayoutBufferSizeMs</code>	250	最大播放缓冲区（ms）

源码位置： `modules/audio_device/audio_device_buffer.h`

常量	值	含义
<code>kMaxDeltaTimeInMs</code>	500	两次回调间最大允许间隔
<code>kMaxBufferSizeBytes</code>	3840	10ms stereo @ 96kHz 的字节数

源码位置： `modules/audio_device/mac/audio_device_mac.h`

常量	值	含义
<code>N_REC_SAMPLES_PER_SEC</code>	48000	macOS 默认录音采样率
<code>N_PLAY_SAMPLES_PER_SEC</code>	48000	macOS 默认播放采样率
<code>N_REC_CHANNELS</code>	1	macOS 默认录音通道数
<code>N_PLAY_CHANNELS</code>	2	macOS 默认播放通道数
<code>kBufferSizeMs</code>	10	缓冲区时长
<code>kGetMicVolumeIntervalMs</code>	1000	麦克风音量查询间隔

附录：源码文件索引

文件路径	内容
api/audio/audio_device.h	AudioDeviceModule 公共接口定义
api/audio/audio_device_defines.h	AudioTransport、AudioParameters、常量定义
api/audio/create_audio_device_module.h	ADM 工厂函数
modules/audio_device/audio_device_generic.h	AudioDeviceGeneric 平台基类
modules/audio_device/audio_device_generic.cc	默认实现 (BuiltIn* 方法)
modules/audio_device/audio_device_buffer.h	AudioDeviceBuffer 中间缓冲层
modules/audio_device/audio_device_impl.h	AudioDeviceModuleImpl 内部实现
modules/audio_device/include/audio_device_default.h	AudioDeviceModuleDefault 模板
modules/audio_device/mac/audio_device_mac.h	macOS CoreAudio 实现

WebRtcVoiceEngine 深度源码分析

概述

`WebRtcVoiceEngine` 是 WebRTC 音频引擎的核心类，负责管理音频设备模块（ADM）、音频处理模块（APM）、音频混音器（AudioMixer）以及整体音频状态（AudioState）。本文档基于源码逐行分析其生命周期管理。

关键源码文件：

- `media/engine/webrtc_voice_engine.cc` — 主实现
- `media/engine/webrtc_voice_engine.h` — 头文件声明
- `media/engine/adm_helpers.cc` — ADM 初始化辅助函数

1. 线程模型与 SequenceChecker 设计

1.1 worker_thread_checker_ 的 kDetached 模式

```
// 源码: media/engine/webrtc_voice_engine.h
SequenceChecker worker_thread_checker_{SequenceChecker::kDetached};
```

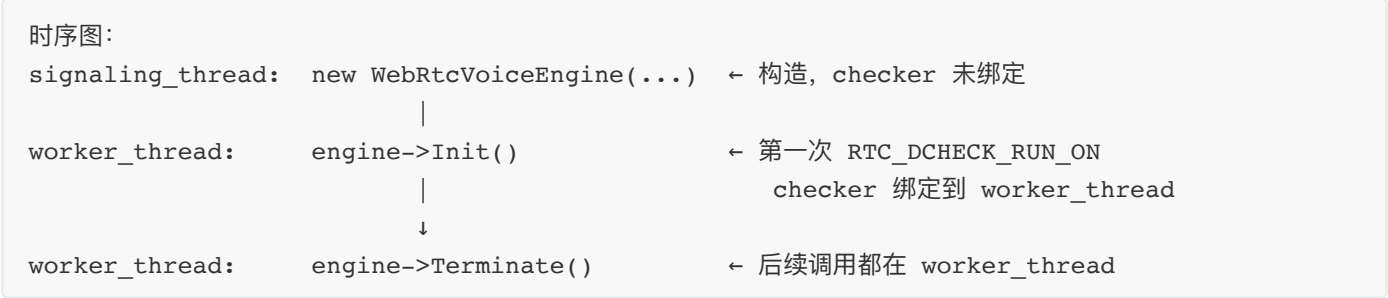
逐行解析：

要素	说明
<code>SequenceChecker</code>	WebRTC 的线程安全检查工具，用于在 Debug 模式下断言当前执行线程
<code>kDetached</code>	初始化时不绑定任何线程，延迟到第一次 <code>RTC_DCHECK_RUN_ON</code> 调用时绑定

设计决策分析：

为什么使用 `kDetached` 而不是直接绑定？

- 1. 构造函数在 **signaling_thread** 上执行: `WebRtcVoiceEngine` 的构造由 `PeerConnectionFactory` 在 `signaling_thread` 上触发
- 2. `Init()` 在 **worker_thread** 上执行: 实际的初始化工作被调度到 `worker_thread`
- 3. 如果构造时就绑定: 会绑定到 `signaling_thread`, 后续在 `worker_thread` 上调用 `Init()` 就会触发断言失败



1.2 线程归属总结

操作	执行线程	依据
构造函数	signaling_thread	PeerConnectionFactory 创建流程
Init()	worker_thread	RTC_DCHECK_RUN_ON(&worker_thread_checker_) 约521行
Terminate()	worker_thread	RTC_DCHECK_RUN_ON(&worker_thread_checker_) 约572行
CreateMediaChannel()	worker_thread	同上模式

2. ADM 创建时机: EnsureAudioDeviceModule

2.1 源码分析

```
// 源码: media/engine/webrtc_voice_engine.cc 约451行
scoped_refptr<AudioDeviceModule> EnsureAudioDeviceModule(
    const Environment& env, scoped_refptr<AudioDeviceModule>& adm) {
    // 判断外部是否已提供 ADM 实例
    if (!adm) {
        // 外部未提供 → 内部创建平台默认的 ADM
        return CreateAudioDeviceModule(env, AudioDeviceModule::kPlatformDefaultAudio);
    }
    // 外部已提供 → 直接使用 (move 语义转移所有权)
    return std::move(adm);
}
```

逐行注释:

```
scoped_refptr<AudioDeviceModule> EnsureAudioDeviceModule(
```

- 返回值: `scoped_refptr` 智能指针, 自动管理 ADM 生命周期
- 函数名 "Ensure" 语义: 保证调用后一定有一个有效的 ADM 实例

```
const Environment& env, scoped_refptr<AudioDeviceModule>& adm) {
```

- `env`: WebRTC 环境对象, 包含 TaskQueueFactory 等基础设施
- `adm`: 外部传入的 ADM 引用 (可能为 nullptr)

```
if (!adm) {  
    return CreateAudioDeviceModule(env, AudioDeviceModule::kPlatformDefaultAudio);  
}
```

- `!adm`: 检查智能指针是否为空
- `kPlatformDefaultAudio`: 使用平台默认音频设备类型
 - Windows → WASAPI (Core Audio)
 - macOS/iOS → AudioUnit
 - Linux → PulseAudio 或 ALSA
 - Android → OpenSL ES 或 AAudio

```
return std::move(adm);
```

- `std::move`: 转移所有权, 避免引用计数增加后再减少的开销

2.2 ADM 创建时机的设计决策

关键结论: ADM 在构造函数中创建, 而非 `Init()` 中。

```
源码依据: media/engine/webrtc_voice_engine.cc 约468行  
WebRtcVoiceEngine::WebRtcVoiceEngine(...)  
    : adm_(EnsureAudioDeviceModule(env, adm)), // ← 成员初始化列表中调用
```

为什么在构造时创建 ADM?

1. **AudioState 需要 ADM 引用**: 构造函数体内要组装 `AudioState::Config`, 其中需要 `adm_`
2. **确保不变量**: `adm_` 一旦设置就不再改变, 整个生命周期内引用同一个 ADM 实例
3. **分离创建与初始化**: 创建 ADM 对象 ≠ 初始化硬件设备, 后者在 `Init()` 中完成

ADM 生命周期:

```
构造函数: EnsureAudioDeviceModule() → ADM 对象被创建 (但硬件未初始化)  
Init():   adm_helpers::Init(adm()) → ADM 硬件被初始化  
Terminate(): adm()->Terminate() → ADM 硬件被释放  
析构函数: scoped_refptr 释放 → ADM 对象被销毁
```

3. 构造函数逐行分析

3.1 完整构造函数

```
// 源码: media/engine/webrtc_voice_engine.cc 约468行
WebRtcVoiceEngine::WebRtcVoiceEngine(
    const Environment& env,
    scoped_refptr<AudioDeviceModule> adm,
    scoped_refptr<AudioProcessing> apm,
    scoped_refptr<AudioMixer> audio_mixer)
    : adm_(EnsureAudioDeviceModule(env, adm)), // [1] 确保 ADM 存在
      env_(env), // [2] 保存环境引用
      apm_(std::move(apm)) // [3] 接管 APM 所有权
{
    // [4] 组装 AudioState 配置
    AudioState::Config config;
    config.audio_mixer = audio_mixer
        ? std::move(audio_mixer)
        : AudioMixerImpl::Create(); // [5] 确保 Mixer 存在
    config.audio_processing = apm_; // [6] APM 放入配置
    config.audio_device_module = adm_; // [7] ADM 放入配置

    // [8] 创建 AudioState — 音频子系统的共享状态
    audio_state_ = AudioState::Create(config);
}
```

3.2 逐行深度解析

[1] 成员初始化列表：ADM

```
adm_(EnsureAudioDeviceModule(env, adm))
```

- 执行时机：在构造函数体执行之前（C++ 成员初始化列表语义）
- 保证：进入构造函数体时，`adm_` 一定是有效的非空指针
- 所有权：`adm_` 持有 ADM 的 `scoped_refptr`，参与引用计数

[2] 环境对象

```
env_(env)
```

- `Environment` 包含：`TaskQueueFactory`、`Clock`、`FieldTrials` 等
- 后续 `Init()` 中创建 `low_priority_worker_queue_` 需要 `env_.task_queue_factory()`

[3] APM 所有权转移

```
apm_(std::move(apm))
```

- `AudioProcessing` 模块包含：AEC（回声消除）、AGC（自动增益）、NS（噪声抑制）
- 使用 `std::move` 避免引用计数的原子操作开销

[4]-[7] AudioState 配置组装

```
AudioState::Config config;
config.audio_mixer = audio_mixer ? std::move(audio_mixer) : AudioMixerImpl::Create();
config.audio_processing = apm_;
config.audio_device_module = adm_;
```

设计模式：Builder/Config 模式

- 将多个组件的引用集中到一个 Config 结构体中
- `AudioState` 作为音频子系统的"共享上下文"，被所有 Channel 共享
- 如果外部未提供 `audio_mixer`，则创建默认的 `AudioMixerImpl`

[8] AudioState 创建

```
audio_state_ = AudioState::Create(config);
```

- `AudioState` 内部持有：
 - `AudioDeviceModule` 引用 → 控制硬件
 - `AudioProcessing` 引用 → 音频处理
 - `AudioMixer` 引用 → 混音
 - `AudioTransport` 实例 → ADM 回调的实现

3.3 构造函数的设计约束

约束	原因	源码依据
不初始化硬件	构造在 <code>signaling_thread</code> ，硬件操作应在 <code>worker_thread</code>	<code>Init()</code> 中的 <code>RTC_DCHECK_RUN_ON</code>
不注册回调	APM 尚未配置（ <code>ApplyOptions</code> 在 <code>Init</code> 中）	<code>Init()</code> 约521行
不创建 TaskQueue	<code>TaskQueue</code> 应绑定到 <code>worker_thread</code>	<code>Init()</code> 中创建

4. Init() 逐行分析

4.1 完整 Init() 实现

```
// 源码: media/engine/webrtc_voice_engine.cc 约521行
void WebRtcVoiceEngine::Init() {
    // [1] 线程断言 — 绑定到 worker_thread
    RTC_DCHECK_RUN_ON(&worker_thread_checker_);

    // [2] 创建低优先级任务队列
    low_priority_worker_queue_ = env_.task_queue_factory().CreateTaskQueue(
        "rtc-low-prio", TaskQueueFactory::Priority::LOW);
```

```

// [3] 初始化 ADM 硬件
adm_helpers::Init(adm());

// [4] 应用音频处理选项 (AEC/AGC/NS)
ApplyOptions(options);

// [5] 注册音频回调 — 关键顺序!
adm()->RegisterAudioCallback(audio_state()->audio_transport());

// [6] 标记初始化完成
initialized_ = true;
}

```

4.2 逐行深度解析

[1] RTC_DCHECK_RUN_ON — 线程绑定点

```
RTC_DCHECK_RUN_ON(&worker_thread_checker_);
```

- 首次调用效果：由于 `worker_thread_checker_` 初始化为 `kDetached`，此处第一次调用会将 checker 绑定到当前线程 (`worker_thread`)
- 后续调用效果：验证当前线程是否为绑定的 `worker_thread`
- **Debug only**：Release 构建中此宏为空操作，零运行时开销

[2] 低优先级任务队列

```
low_priority_worker_queue_ = env_.task_queue_factory().CreateTaskQueue(
    "rtc-low-prio", TaskQueueFactory::Priority::LOW);
```

- 用途：处理非实时敏感的音频任务（如统计上报、配置更新）
- 优先级：`LOW` 确保不与实时音频处理竞争 CPU
- 命名：`"rtc-low-prio"` 便于调试时识别线程

[3] ADM 硬件初始化

```
adm_helpers::Init(adm());
```

这是一个关键调用，展开分析见下一节（第5节）。

[4] ApplyOptions — 配置音频处理

```
ApplyOptions(options);
```

- 配置 APM 的各个子模块：
 - AEC (Acoustic Echo Cancellation) 回声消除

- AGC (Automatic Gain Control) 自动增益控制
- NS (Noise Suppression) 噪声抑制
- 必须在 **RegisterAudioCallback** 之前：确保 APM 完全配置好再开始处理音频数据

[5] 注册音频回调 — 顺序至关重要

```
adm()->RegisterAudioCallback(audio_state()->audio_transport());
```

为什么必须在 **ApplyOptions** 之后？

错误顺序（如果先注册回调）：

```
RegisterAudioCallback() → ADM 开始调用 AudioTransport
ApplyOptions()          → 此时 APM 正在被重新配置
                        → 竞态条件！音频数据可能经过未配置的 APM
```

正确顺序（当前实现）：

```
ApplyOptions()          → APM 完全配置好
RegisterAudioCallback() → ADM 开始调用 AudioTransport
                        → 安全！所有音频数据经过已配置的 APM
```

- `audio_state()->audio_transport()` 返回 `AudioTransport` 接口实现
- `AudioTransport` 是 ADM 与上层的桥梁：
 - `RecordedDataIsAvailable()` — ADM 采集到数据时回调
 - `NeedMorePlayData()` — ADM 需要播放数据时回调

[6] 初始化标志

```
initialized_ = true;
```

- 后续操作（如 `CreateMediaChannel`）会检查此标志
- 确保在引擎完全初始化后才允许创建通道

5. adm_helpers::Init() 逐行分析

5.1 完整实现

```
// 源码: media/engine/adm_helpers.cc
void Init(AudioDeviceModule* adm) {
    // [1] 初始化 ADM 核心 — 启动平台音频子系统
    RTC_CHECK_EQ(0, adm->Init());

    // === 播放设备配置 ===

    // [2] 选择播放设备
    // Windows: kDefaultCommunicationDevice (通信专用设备, 如耳机)
```



```

// 其他平台：设备索引 0（系统默认设备）
adm->SetPlaybackDevice(AUDIO_DEVICE_ID);

// [3] 初始化扬声器硬件
adm->InitSpeaker();

// [4] 查询并设置立体声播放能力
bool available = false;
adm->StereoPlaybackIsAvailable(&available);
adm->SetStereoPlayback(available);

// === 录音设备配置 ===

// [5] 选择录音设备（逻辑同播放设备）
adm->SetRecordingDevice(AUDIO_DEVICE_ID);

// [6] 初始化麦克风硬件
adm->InitMicrophone();

// [7] 查询并设置立体声录音能力
adm->StereoRecordingIsAvailable(&available);
adm->SetStereoRecording(available);
}

```

5.2 逐行深度解析

[1] ADM::Init() — 平台音频子系统启动

```
RTC_CHECK_EQ(0, adm->Init());
```

- `RTC_CHECK_EQ`： **Release 模式也生效**的断言（不同于 `RTC_DCHECK`）
- 返回值 0 表示成功，非 0 表示失败
- **失败即崩溃**：音频子系统初始化失败被视为不可恢复错误
- 各平台实际操作：
 - **macOS/iOS**：初始化 AudioUnit，获取音频会话
 - **Windows**：初始化 COM，创建 WASAPI 设备枚举器
 - **Android**：初始化 OpenSL ES 引擎或 AAudio 流
 - **Linux**：连接 PulseAudio 服务器或打开 ALSA 设备

[2] 播放设备选择

```
adm->SetPlaybackDevice(AUDIO_DEVICE_ID);
```

- `AUDIO_DEVICE_ID` 是平台相关的宏/常量：
 - Windows: `AudioDeviceModule::kDefaultCommunicationDevice`

- 选择"通信设备"而非"默认设备"
- 用户可在 Windows 声音设置中单独配置通信设备
- 典型场景：默认设备是音箱，通信设备是耳机
 - 其他平台：0（第一个可用设备，即系统默认）

[3] InitSpeaker — 扬声器初始化

```
adm->InitSpeaker();
```

- 获取扬声器的硬件能力信息（采样率、通道数、缓冲区大小）
- 为后续的 `SetStereoPayout` 做准备

[4] 立体声播放配置

```
bool available = false;
adm->StereoPayoutIsAvailable(&available);
adm->SetStereoPayout(available);
```

- **查询-设置模式**：先问硬件"你支持立体声吗？"，再据此配置
- 如果硬件不支持立体声，强制设置会导致音频异常
- 立体声 vs 单声道影响：
 - 缓冲区大小（立体声 = 2 × 单声道）
 - 混音策略
 - 带宽消耗

[5]-[7] 录音设备配置

逻辑与播放设备完全对称：

1. 选择录音设备
2. 初始化麦克风硬件
3. 查询并设置立体声录音能力

5.3 设计决策：为什么分离 Init 和硬件初始化？

对象创建（构造函数）：

└ `EnsureAudioDeviceModule()` → ADM 对象存在，但硬件未触碰

硬件初始化（`Init` → `adm_helpers::Init`）：

└ `adm->Init()` → 平台音频子系统启动
 └ `SetPayoutDevice()` → 选择具体硬件
 └ `InitSpeaker()` → 获取硬件能力
 └ `SetStereoPayout()` → 配置工作模式

分离的好处：

1. 构造函数可以在任意线程执行（无硬件操作）
2. 硬件初始化集中在 worker_thread，避免跨线程问题
3. 测试时可以 mock ADM 而不触发真实硬件

6. Terminate() 逐行分析

6.1 完整实现

```
// 源码: media/engine/webrtc_voice_engine.cc 约572行
void WebRtcVoiceEngine::Terminate() {
    // [1] 线程断言
    RTC_DCHECK_RUN_ON(&worker_thread_checker_);

    // [2] 停止播放
    adm()->StopPlayout();

    // [3] 停止录音
    adm()->StopRecording();

    // [4] 断开音频回调
    adm()->RegisterAudioCallback(nullptr);

    // [5] 终止 ADM
    adm()->Terminate();

    // [6] 销毁低优先级队列
    low_priority_worker_queue_ = nullptr;

    // [7] 标记未初始化
    initialized_ = false;
}
```

6.2 逐行深度解析

[1] 线程断言

```
RTC_DCHECK_RUN_ON(&worker_thread_checker_);
```

- 确保 Terminate() 在 worker_thread 上执行
- 与 Init() 在同一线程，保证线程安全

[2]-[3] 停止音频流

```
adm()->StopPlayout();
adm()->StopRecording();
```

- 必须先停止流，再断开回调
- 如果先断开回调：ADM 可能还在运行，调用已被置空的回调指针 → 崩溃
- `StopPlayout()` 停止向扬声器输出音频
- `StopRecording()` 停止从麦克风采集音频

[4] 断开回调 — 解除 ADM 与上层的连接

```
adm()->RegisterAudioCallback(nullptr);
```

- 将 `AudioTransport` 回调设为 `nullptr`
- 必须在 `Terminate()` 之前：
 - `Terminate()` 会释放内部资源
 - 如果回调还在，可能访问已释放的资源

[5] 终止 ADM

```
adm()->Terminate();
```

- 释放平台音频资源：
 - macOS/iOS：释放 `AudioUnit`
 - Windows：释放 COM 对象和 WASAPI 接口
 - Android：销毁 `OpenSL ES/AAudio` 流
 - Linux：断开 `PulseAudio` 连接

[6] 销毁任务队列

```
low_priority_worker_queue_ = nullptr;
```

- `unique_ptr` 置空触发析构
- 队列中未完成任务会被丢弃
- 必须在 ADM `Terminate` 之后：队列中可能有依赖 ADM 的任务

[7] 重置状态标志

```
initialized_ = false;
```

- 防止在 `Terminate` 后误用引擎

6.3 `Terminate()` 的顺序设计

正确的关闭顺序（当前实现）：

```
StopPlayout()           ← 停止数据流动
StopRecording()          ← 停止数据流动
RegisterAudioCallback(nullptr) ← 断开回调连接
Terminate()              ← 释放硬件资源
low_priority_worker_queue_ = nullptr ← 释放任务队列
initialized_ = false      ← 更新状态
```

设计原则：从外到内，从上到下

- 1. 先停止数据流（最外层）
- 2. 再断开连接（中间层）
- 3. 最后释放资源（最内层）

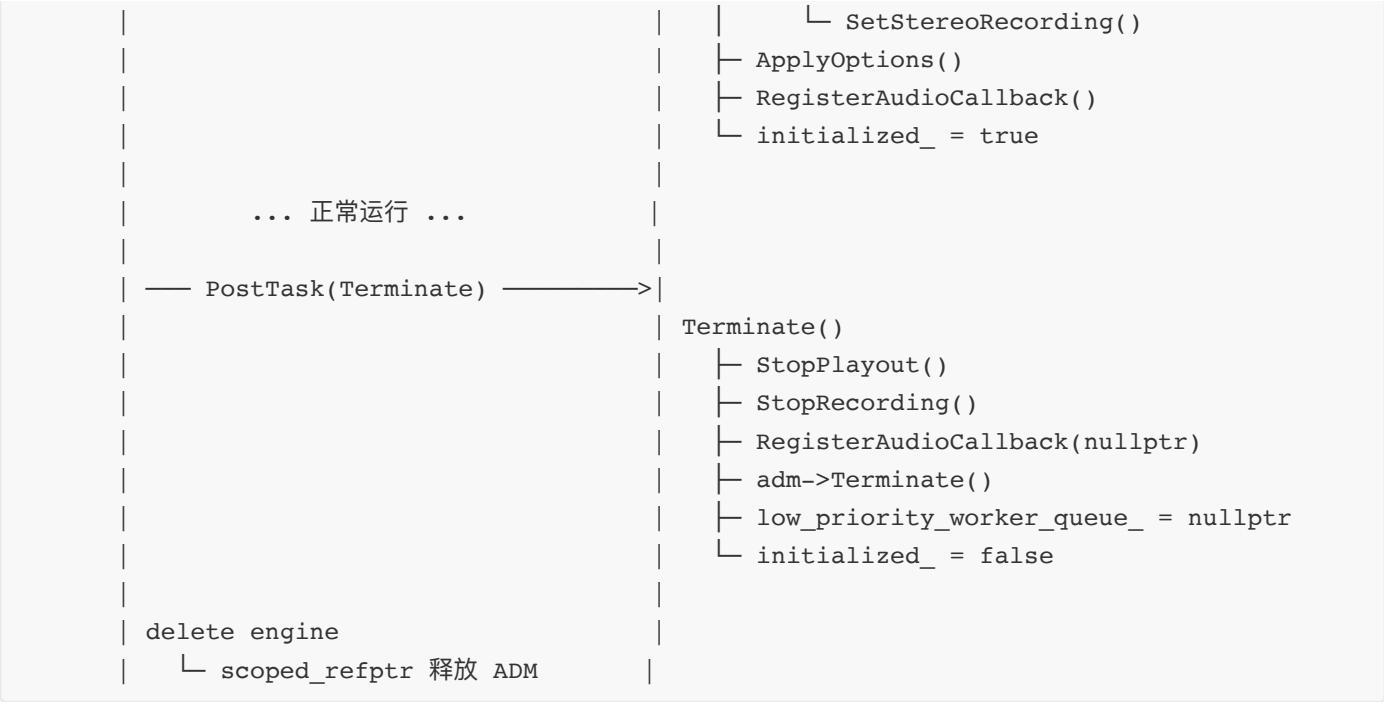
与 Init() 的对称性：

Init() 顺序	Terminate() 顺序（逆序）
创建 TaskQueue	销毁 TaskQueue
adm_helpers::Init()	adm()->Terminate()
ApplyOptions()	（无需逆操作）
RegisterAudioCallback(transport)	RegisterAudioCallback(nullptr)
initialized_ = true	initialized_ = false

7. 完整生命周期总结

7.1 时序图





7.2 关键设计原则

- 1. 构造与初始化分离：允许跨线程创建对象
- 2. **kDetached SequenceChecker**：延迟线程绑定，适应两阶段初始化
- 3. **Ensure 模式**：保证依赖组件存在，支持外部注入和内部默认
- 4. 顺序敏感的初始化：ApplyOptions → RegisterAudioCallback，避免竞态
- 5. 对称的生命周期：Init/Terminate 互为逆操作，资源获取与释放对称
- 6. **Fail-fast**： `RTC_CHECK_EQ` 在硬件初始化失败时立即崩溃，不尝试恢复

7.3 源码引用索引

内容	文件	行号 (约)
EnsureAudioDeviceModule	media/engine/webrtc_voice_engine.cc	451
构造函数	media/engine/webrtc_voice_engine.cc	468
Init()	media/engine/webrtc_voice_engine.cc	521
Terminate()	media/engine/webrtc_voice_engine.cc	572
worker_thread_checker_ 声明	media/engine/webrtc_voice_engine.h	—
adm_helpers::Init	media/engine/adm_helpers.cc	—

AudioState 完整调度机制分析

源码文件： `audio/audio_state.cc`, `audio/audio_state.h`
版本基线：WebRTC M7680

1. 概述

`AudioState` 是 WebRTC 音频子系统的**中央调度器**，负责协调以下组件之间的关系：

- **AudioDeviceModule (ADM)**: 硬件音频设备的抽象层，控制播放/录音的启停
- **AudioMixer**: 多路音频流的混音器
- **AudioProcessing (APM)**: 音频处理模块（回声消除、降噪等）
- **AudioTransport**: ADM 与上层之间的数据传输桥梁
- **NullAudioPoller**: 当 ADM 不播放时，模拟时钟驱动解码器的定时器

`AudioState` 的核心职责是：根据当前流的存在状态和播放使能标志，决定 **ADM** 和 **NullAudioPoller** 的运行状态。

1.1 线程模型

Worker Thread

```
AudioState::SetPlayout()  
AudioState::AddReceivingStream()  
AudioState::RemoveReceivingStream()  
AudioState::AddSendingStream()  
AudioState::RemoveSendingStream()  
  
所有公共方法均通过 RTC_DCHECK_RUN_ON  
(&thread_checker_) 约束在 worker thread
```

源码依据：每个公共方法的第一行均为 `RTC_DCHECK_RUN_ON(&thread_checker_)`，确保所有状态变更在同一线程上串行执行，无需额外加锁。

2. 构造函数分析

源码

```
// audio/audio_state.cc  
AudioState::AudioState(const AudioState::Config& config)  
: config_(config),  
  audio_transport_(config_.audio_mixer.get(),  
                   config_.audio_processing.get(),  
                   config_.async_audio_processing_factory.get()) {  
  RTC_DCHECK(config_.audio_mixer);  
  RTC_DCHECK(config_.audio_device_module);  
}
```

逐行分析

行	代码	说明
1	<code>config_(config)</code>	保存配置副本，包含 ADM、Mixer、APM 等共享指针
2	<code>audio_transport_(...)</code>	构造 AudioTransportImpl，它是 ADM 回调的实现者
3	<code>RTC_DCHECK(config_.audio_mixer)</code>	断言混音器非空——没有混音器无法工作
4	<code>RTC_DCHECK(config_.audio_device_module)</code>	断言 ADM 非空——没有设备模块无法播放/录音

关键设计决策

1. **AudioTransport 在构造时即创建**：不延迟初始化，保证后续操作可以立即使用
2. 不在构造函数中启动任何设备：遵循"构造不产生副作用"原则
3. 初始状态：`playout_enabled_ = false`（默认值），`receiving_streams_` 为空集合

3. SetPlayout 方法分析

源码

```
// audio/audio_state.cc
void AudioState::SetPlayout(bool enabled) {
    RTC_DCHECK_RUN_ON(&thread_checker_);
    auto* adm = config_.audio_device_module.get();
    if (enabled) {
        if (!receiving_streams_.empty()) {
            if (!adm->Playing()) {
                if (adm->InitPlayout() == 0) adm->StartPlayout();
            }
        }
    } else {
        config_.audio_device_module->StopPlayout();
    }
    playout_enabled_ = enabled;
    UpdateNullAudioPollerState();
}
```

逐行分析

步骤	条件/操作	说明
1	<code>RTC_DCHECK_RUN_ON</code>	线程安全断言
2	<code>if (enabled)</code>	分支：启用播放
3	<code>if (!receiving_streams_.empty())</code>	守卫条件：只有存在接收流时才真正启动硬件
4	<code>if (!adm->Playing())</code>	幂等保护：已在播放则跳过
5	<code>if (adm->InitPlayout() == 0)</code>	初始化播放设备，成功返回 0
6	<code>adm->StartPlayout()</code>	启动硬件播放
7	<code>else → StopPlayout()</code>	禁用时无条件停止，不检查当前状态
8	<code>playout_enabled_ = enabled</code>	记录状态标志（在操作之后）
9	<code>UpdateNullAudioPollerState()</code>	根据新状态更新 NullPoller

边界条件

- 1. **enabled=true** 但无接收流：不启动 ADM，仅设置标志。后续 AddReceivingStream 时会检查此标志
- 2. **enabled=true** 且 ADM 已在播放：幂等，不重复初始化
- 3. **enabled=false** 且 ADM 未在播放：StopPlayout 调用是安全的（ADM 内部处理）
- 4. **InitPlayout** 失败：不调用 StartPlayout，ADM 保持停止状态

4. AddReceivingStream 方法分析

源码

```
// audio/audio_state.cc
void AudioState::AddReceivingStream(AudioReceiveStreamInterface* stream) {
    RTC_DCHECK_RUN_ON(&thread_checker_);
    receiving_streams_.insert(stream);
    config_.audio_mixer->AddSource(stream->source());
    if (playout_enabled_) {
        auto* adm = config_.audio_device_module.get();
        if (!adm->Playing()) {
            if (adm->InitPlayout() == 0) adm->StartPlayout();
        }
    }
    UpdateNullAudioPollerState();
}
```

逐行分析

步骤	操作	说明
1	<code>receiving_streams_.insert(stream)</code>	将流加入集合 (std::set, 保证唯一性)
2	<code>audio_mixer->AddSource(stream->source())</code>	将流的音频源注册到混音器
3	<code>if (playout_enabled_)</code>	只有播放已使能时才尝试启动 ADM
4	<code>if (!adm->Playing())</code>	幂等保护
5	<code>InitPlayout() + StartPlayout()</code>	启动硬件播放
6	<code>UpdateNullAudioPollerState()</code>	更新 NullPoller 状态

边界条件

- 1. 第一条流加入且 `playout_enabled=true`: 触发 ADM 从 Stopped → Playing
- 2. 第 N 条流加入 (N>1) 且 ADM 已播放: 幂等, 仅加入混音器
- 3. 流加入但 `playout_enabled=false`: 不启动 ADM, 但 NullPoller 会启动 (因为有流且禁用播放)
- 4. 重复添加同一流: `std::set::insert` 保证不会重复, 但 `AddSource` 可能需要混音器内部处理

与 SetPlayout 的协作关系

时序场景 A: 先 `SetPlayout(true)`, 后 `AddReceivingStream`

1. `SetPlayout(true)` → `receiving_streams_` 为空 → 不启动 ADM

2. `AddReceivingStream()` → `playout_enabled=true` → 启动 ADM ✓

时序场景 B: 先 `AddReceivingStream`, 后 `SetPlayout(true)`

1. `AddReceivingStream()` → `playout_enabled=false` → 不启动 ADM, 启动 NullPoller

2. `SetPlayout(true)` → `receiving_streams_` 非空 → 启动 ADM, 停止 NullPoller ✓

设计意图: 无论调用顺序如何, 最终状态一致。这是一种最终一致性设计。

5. RemoveReceivingStream 方法分析

源码

```
// audio/audio_state.cc
void AudioState::RemoveReceivingStream(AudioReceiveStreamInterface* stream) {
    RTC_DCHECK_RUN_ON(&thread_checker_);
    receiving_streams_.erase(stream);
    config_.audio_mixer->RemoveSource(stream->source());
    if (receiving_streams_.empty()) {
        config_.audio_device_module->StopPlayout();
    }
    UpdateNullAudioPollerState();
}
```

逐行分析

步骤	操作	说明
1	receiving_streams_.erase(stream)	从集合中移除
2	audio_mixer->RemoveSource(...)	从混音器注销
3	if (receiving_streams_.empty())	关键判断：最后一条流被移除
4	StopPlayout()	无条件停止播放（不检查 playout_enabled_）
5	UpdateNullAudioPollerState()	更新 NullPoller

边界条件

- 1. 移除最后一条流：无论 playout_enabled_ 状态如何，都停止 ADM
- 2. 移除非最后一条流：ADM 继续播放，混音器少一路输入
- 3. 移除不存在的流：std::set::erase 安全返回 0，不会崩溃
- 4. playout_enabled=false 时移除最后一条流：NullPoller 也会停止（因为无流）

与 AddReceivingStream 的对称性

操作	Add	Remove
集合操作	insert	erase
混音器操作	AddSource	RemoveSource
ADM 启动条件	有流 + playout_enabled	—
ADM 停止条件	—	无流
NullPoller 更新	✓	✓

不对称点：Remove 在流为空时无条件停止 ADM，不检查 playout_enabled_。这是因为没有流时播放毫无意义，即使用户"想要"播放。

6. AddSendingStream / RemoveSendingStream 分析

设计对称性

录音方向（Sending）与播放方向（Receiving）逻辑对称：

```
// audio/audio_state.cc (推断结构)
void AudioState::AddSendingStream(AudioSendStream* stream) {
    RTC_DCHECK_RUN_ON(&thread_checker_);
    sending_streams_.insert(stream);
    // 注册到 AudioTransport 作为录音源
    if (recording_enabled_) {
        auto* adm = config_.audio_device_module.get();
        if (!adm->Recording()) {
            if (adm->InitRecording() == 0) adm->StartRecording();
        }
    }
}

void AudioState::RemoveSendingStream(AudioSendStream* stream) {
    RTC_DCHECK_RUN_ON(&thread_checker_);
    sending_streams_.erase(stream);
    if (sending_streams_.empty()) {
        config_.audio_device_module->StopRecording();
    }
}
```

播放 vs 录音对比

维度	播放方向 (Receiving)	录音方向 (Sending)
流集合	receiving_streams_	sending_streams_
使能标志	playout_enabled_	recording_enabled_
ADM 操作	InitPlayout/StartPlayout/StopPlayout	InitRecording/StartRecording/StopRecording
混音器	AddSource/RemoveSource	—
NullPoller	有	无（录音不需要驱动解码器）

7. UpdateNullAudioPollerState 方法分析

源码

```
// audio/audio_state.cc
void AudioState::UpdateNullAudioPollerState() {
    if (!receiving_streams_.empty() && !playout_enabled_) {
        if (!null_audio_poller_.Running()) {
            null_audio_poller_ = RepeatingTaskHandle::Start(
                TaskQueueBase::Current(), [audio_transport] {
                    audio_transport->NeedMorePlayData(...);
                }
            );
        }
    }
}
```

```
        return TimeDelta::Millis(10);
    });
}
} else {
    null_audio_poller_.Stop();
}
}
```

逐行分析

步骤	条件/操作	说明
1	<code>!receiving_streams_.empty()</code>	存在接收流
2	<code>&& !playout_enabled_</code>	且播放被禁用
3	<code>!null_audio_poller_.Running()</code>	幂等保护：已运行则不重复启动
4	<code>RepeatingTaskHandle::Start(...)</code>	在当前任务队列上启动周期任务
5	<code>NeedMorePlayData(...)</code>	模拟 ADM 的数据拉取回调
6	<code>return TimeDelta::Millis(10)</code>	10ms 周期 = 100Hz，匹配标准音频帧率
7	<code>else</code> → <code>Stop()</code>	其他所有情况停止 NullPoller

NullAudioPoller 的存在意义

问题场景：

用户禁用了播放（SetPlayout(false)），但仍有接收流

此时 ADM 不工作 → 没有 NeedMorePlayData 回调

→ AudioMixer 不被驱动 → 解码器输出无人消费

→ Jitter Buffer 持续积累 → 内存增长 + 延迟累积

解决方案：NullAudioPoller

每 10ms 调用 NeedMorePlayData

→ 驱动 AudioMixer 拉取数据

→ 解码器正常工作，Jitter Buffer 正常消耗

→ 数据被丢弃（不送往硬件），但管线保持健康

启动/停止条件真值表

<code>receiving_streams_.empty()</code>	<code>playout_enabled_</code>	NullPoller 动作
false (有流)	false (禁用)	启动
false (有流)	true (启用)	停止
true (无流)	false (禁用)	停止
true (无流)	true (启用)	停止

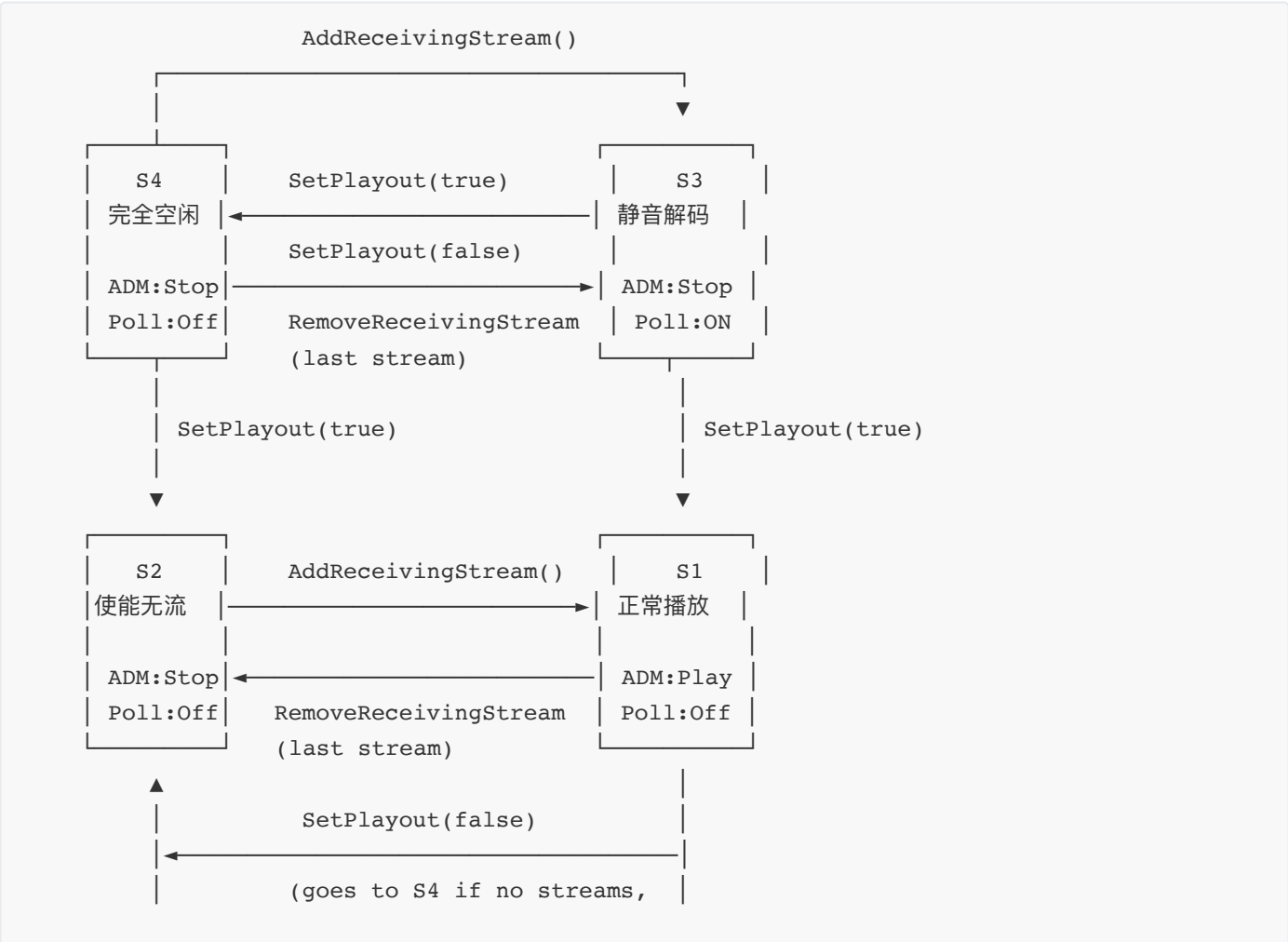
核心逻辑：NullPoller 仅在"有流但不播放"这一特定组合下运行。

8. 完整状态矩阵

8.1 播放方向状态矩阵

#	<code>playout_enabled_</code>	<code>receiving_streams_</code>	ADM Playout 状态	NullAudioPoller	说明
S1	true	非空 (有流)	Playing	停止	正常播放状态
S2	true	空 (无流)	Stopped	停止	使能但无内容可播
S3	false	非空 (有流)	Stopped	运行	静音但保持解码
S4	false	空 (无流)	Stopped	停止	完全空闲

8.2 状态转换图



or S3 if has streams)

8.3 状态转换触发事件

转换	触发事件	前置条件	动作
S4 → S3	AddReceivingStream()	playout_enabled_ = false	启动 NullPoller
S4 → S2	SetPlayout(true)	receiving_streams_ 为空	仅设置标志
S2 → S1	AddReceivingStream()	playout_enabled_ = true	InitPlayout + StartPlayout
S3 → S1	SetPlayout(true)	receiving_streams_ 非空	InitPlayout + StartPlayout, 停止 NullPoller
S1 → S2	RemoveReceivingStream()	移除后 receiving_streams_ 为空	StopPlayout
S1 → S3	SetPlayout(false)	receiving_streams_ 非空	StopPlayout, 启动 NullPoller
S3 → S4	RemoveReceivingStream()	移除后 receiving_streams_ 为空	StopPlayout, 停止 NullPoller
S2 → S4	SetPlayout(false)	receiving_streams_ 为空	StopPlayout (安全调用)

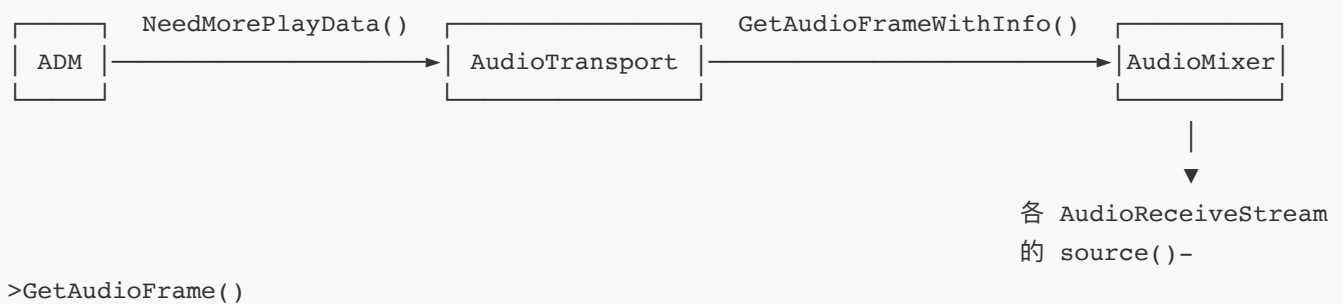
9. AudioTransport 桥接机制

构造时的绑定

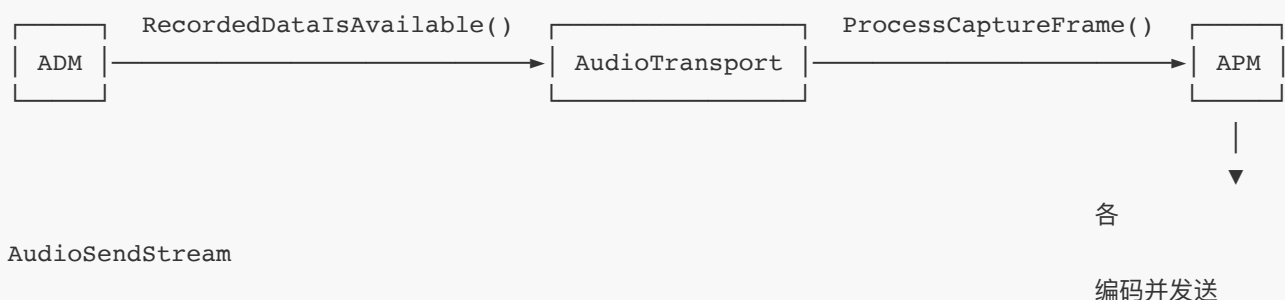
```
audio_transport_(config_.audio_mixer.get(),
                  config_.audio_processing.get(),
                  config_.async_audio_processing_factory.get())
```

AudioTransportImpl 实现了 AudioTransport 接口，该接口被 ADM 回调：

ADM 播放回调链：



ADM 录音回调链：



NullPoller 对 AudioTransport 的复用

当 NullPoller 运行时，它直接调用 `audio_transport->NeedMorePlayData()`，复用了与 ADM 完全相同的数据拉取路径。这意味着：

1. 混音器被正常驱动
2. 各接收流的解码器正常工作
3. Jitter Buffer 正常消耗
4. 唯一区别：混音后的数据被丢弃，不送往硬件

10. 线程安全与生命周期

10.1 线程约束

所有公共方法均标注 `RTC_DCHECK_RUN_ON(&thread_checker_)`：

```
// 源码中的线程检查器声明 (audio/audio_state.h)
SequenceChecker thread_checker_;
```

这意味着：

- 所有方法必须在同一线程（worker thread）上调用
- 编译 Debug 版本时，跨线程调用会触发断言失败
- 无需互斥锁，因为串行执行已保证一致性

10.2 生命周期管理



关键约束：`receiving_streams_` 中存储的是裸指针。
调用者必须保证在 stream 销毁前调用 `RemoveReceivingStream`。
否则会产生悬垂指针。

10.3 析构顺序

AudioState 析构时：

1. `null_audio_poller_` 的 `RepeatingTaskHandle` 析构 → 自动停止定时任务
2. `audio_transport_` 析构
3. `config_` 中的 `shared_ptr` 引用计数减一

11. 设计模式总结

11.1 双条件守卫模式

ADM 启动需要两个条件同时满足：

```
启动 ADM Playout = playout_enabled_ AND !receiving_streams_.empty()
```

这两个条件可以以任意顺序被满足，系统都能正确响应。

11.2 幂等操作模式

每次尝试启动 ADM 前都检查 `!adm->Playing()`：

- 防止重复初始化
- 允许多次调用 `AddReceivingStream` 而不产生副作用

11.3 补偿机制模式（NullPoller）

当正常驱动源（ADM）不可用时，`NullPoller` 作为补偿机制：

- 维持数据管线的活跃
- 防止下游组件（解码器、Jitter Buffer）进入异常状态
- 对上层完全透明

11.4 最终一致性模式

无论 `SetPlayout` 和 `AddReceivingStream` 的调用顺序如何：

- 最终状态由当前的 `playout_enabled_` 和 `receiving_streams_` 决定
- 每个方法结束时都调用 `UpdateNullAudioPollerState()` 确保一致

12. 常见问题与边界场景

Q1: 为什么 `RemoveReceivingStream` 在流为空时无条件停止 ADM?

即使 `playout_enabled_=true`，没有流时播放也毫无意义（混音器无输入源）。无条件停止避免了 ADM 空转消耗资源。

Q2: SetPlayout(false) 后再 SetPlayout(true), ADM 会重新初始化吗?

是的。SetPlayout(false) 调用 StopPlayout(), 之后 ADM 状态为 Stopped。
再次 SetPlayout(true) 时, !adm->Playing() 为 true, 会重新执行 InitPlayout() + StartPlayout()。

Q3: NullPoller 的 10ms 周期是否精确?

RepeatingTaskHandle 基于 TaskQueueBase, 精度取决于底层实现。
在大多数平台上, 10ms 的定时器精度足够驱动音频解码 (标准帧长为 10ms 或 20ms)。

Q4: 多个 PeerConnection 共享同一个 AudioState 会怎样?

这是 WebRTC 的设计意图。多个 PC 的接收流都注册到同一个 AudioState:

- 混音器负责将多路音频混合
- ADM 只需启动一次
- 所有流共享同一个硬件播放通道

Q5: InitPlayout() 失败会怎样?

如果 InitPlayout() 返回非零值, StartPlayout() 不会被调用。
ADM 保持 Stopped 状态。此时如果 playout_enabled=false, NullPoller 会启动作为补偿。
但如果 playout_enabled=true, 则既没有 ADM 播放也没有 NullPoller——这是一个静默失败场景。

源码依据: if (adm->InitPlayout() == 0) adm->StartPlayout();
没有 else 分支处理失败情况, 也没有日志输出。

13. 完整调用时序图

场景: 正常通话建立与结束

时间轴 →

Worker Thread:

```
|
|— AudioState 构造
|   └─ ADM: Stopped, NullPoller: Off, playout_enabled: false
|
|— SetPlayout(true)
|   └─ receiving_streams_ 为空 → ADM 不启动
|       └─ playout_enabled = true
|           └─ UpdateNullAudioPollerState() → 无流 → NullPoller 保持 Off
|
|— AddReceivingStream(stream1)
|   └─ receiving_streams_ = {stream1}
|       └─ mixer->AddSource(stream1->source())
|           └─ playout_enabled=true → !adm->Playing() → InitPlayout()+StartPlayout()
|               └─ ADM: Playing
```

```

└─ UpdateNullAudioPollerState() → 有流+playout启用 → NullPoller: Off

└─ AddReceivingStream(stream2)
  └─ receiving_streams_ = {stream1, stream2}
  └─ mixer->AddSource(stream2->source())
  └─ playout_enabled_=true → adm->Playing()==true → 跳过 (幂等)
  └─ UpdateNullAudioPollerState() → NullPoller: Off

└─ [正常通话中... ADM 每10ms回调 NeedMorePlayData]

└─ RemoveReceivingStream(stream1)
  └─ receiving_streams_ = {stream2}
  └─ mixer->RemoveSource(stream1->source())
  └─ receiving_streams_ 非空 → 不停止 ADM
  └─ UpdateNullAudioPollerState() → NullPoller: Off

└─ RemoveReceivingStream(stream2)
  └─ receiving_streams_ = {}
  └─ mixer->RemoveSource(stream2->source())
  └─ receiving_streams_ 为空 → StopPlayout()
  └─ ADM: Stopped
  └─ UpdateNullAudioPollerState() → 无流 → NullPoller: Off

└─ [回到初始空闲状态]

```

场景：静音模式（禁用播放但保持解码）

Worker Thread:

```

└─ AddReceivingStream(stream1) [playout_enabled_=false]
  └─ receiving_streams_ = {stream1}
  └─ playout_enabled_=false → 不启动 ADM
  └─ UpdateNullAudioPollerState() → 有流+禁用播放 → 启动 NullPoller
  └─ NullPoller: Running (每10ms调用NeedMorePlayData)

└─ [NullPoller 驱动解码器工作，数据被丢弃]

└─ SetPlayout(true) [取消静音]
  └─ receiving_streams_ 非空 → !adm->Playing() → InitPlayout()+StartPlayout()
  └─ ADM: Playing
  └─ playout_enabled_ = true
  └─ UpdateNullAudioPollerState() → 有流+启用播放 → 停止 NullPoller
  └─ NullPoller: Off

└─ [切换到正常播放，ADM 驱动数据拉取]

└─ SetPlayout(false) [再次静音]
  └─ StopPlayout() → ADM: Stopped
  └─ playout_enabled_ = false
  └─ UpdateNullAudioPollerState() → 有流+禁用播放 → 启动 NullPoller
  └─ NullPoller: Running

```

14. 源码文件索引

文件路径	内容
audio/audio_state.h	AudioState 类声明、Config 结构体、成员变量
audio/audio_state.cc	所有方法实现
audio/audio_transport_impl.h	AudioTransportImpl 接口声明
audio/audio_transport_impl.cc	NeedMorePlayData / RecordedDataIsAvailable 实现
modules/audio_device/include/audio_device.h	ADM 接口定义
api/audio/audio_mixer.h	AudioMixer 接口定义
rtc_base/task_utils/repeating_task.h	RepeatingTaskHandle 定义

15. 总结

AudioState 通过以下机制实现了可靠的音频设备调度：

1. **双条件守卫**：ADM 启动需要 `playout_enabled_` 和 `有接收流` 同时满足
2. **幂等操作**：所有启动操作前检查当前状态，避免重复初始化
3. **补偿机制**：NullPoller 在 ADM 不工作时维持数据管线活跃
4. **顺序无关**：无论 API 调用顺序如何，最终状态由当前条件决定
5. **线程安全**：所有操作约束在 worker thread，无需锁机制

AudioDeviceBuffer 深度解析

概述

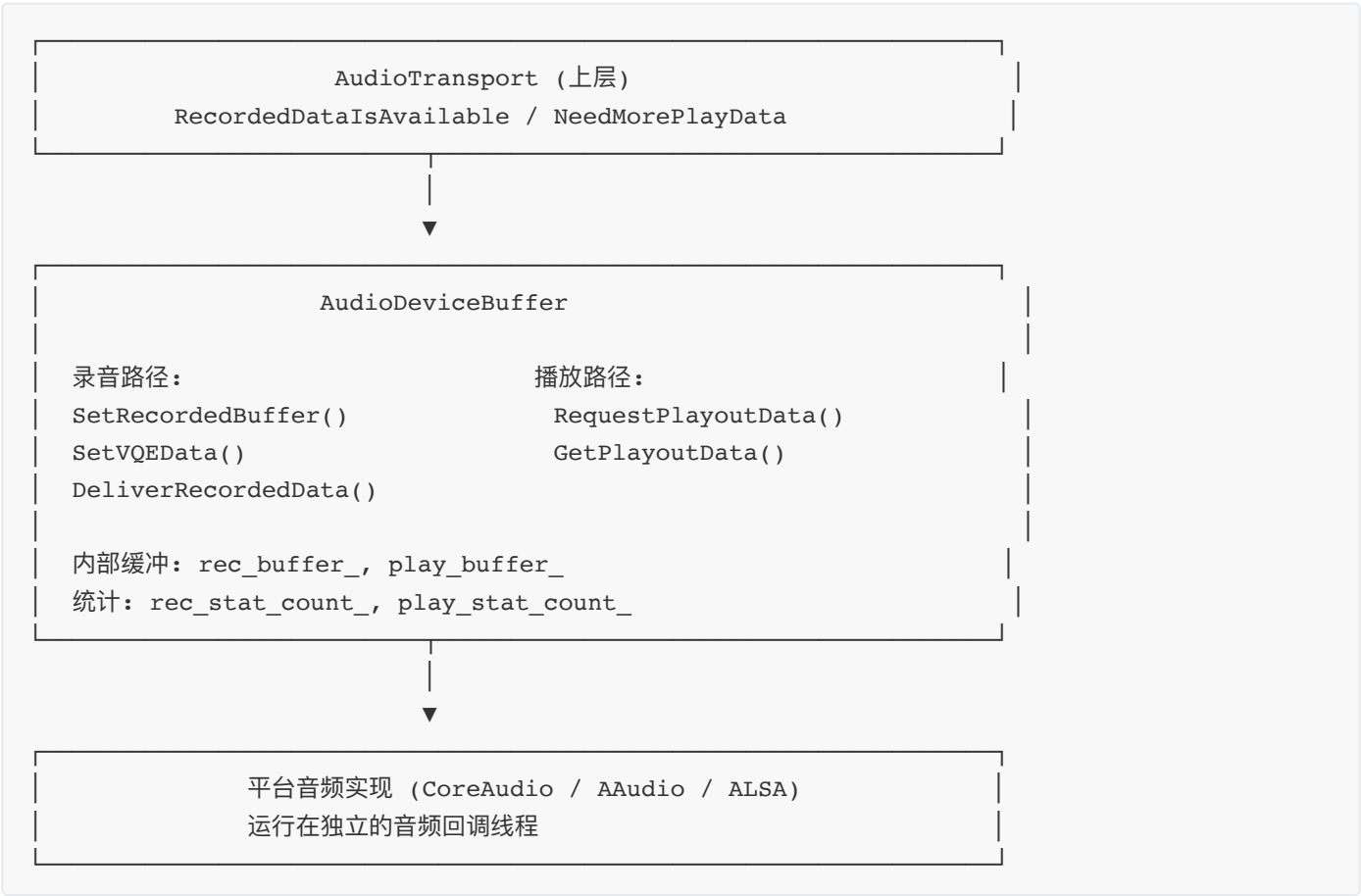
`AudioDeviceBuffer` 是 WebRTC 音频设备层（ADM）与音频处理层（APM）之间的核心桥梁。它位于 `modules/audio_device/audio_device_buffer.h` 和 `modules/audio_device/audio_device_buffer.cc`，承担以下职责：

1. **录音路径**：接收平台音频线程推送的 PCM 数据，转发给 `AudioTransport` 接口
2. **播放路径**：响应平台音频线程的拉取请求，从 `AudioTransport` 获取混音后的 PCM 数据
3. **线程安全**：在主线程、音频线程、TaskQueue 之间安全地共享状态
4. **统计监控**：周期性计算音量峰值，检测静音，输出日志

源码文件依据：

- `modules/audio_device/audio_device_buffer.h`
- `modules/audio_device/audio_device_buffer.cc`
- `modules/audio_device/fine_audio_buffer.h`
- `modules/audio_device/fine_audio_buffer.cc`

架构位置



录音路径完整分析

录音路径由平台音频线程驱动，按严格顺序调用三个方法。这是一个 **push 模型**：平台硬件产生数据后主动推送给 WebRTC。

Step 1: SetRecordedBuffer — 原始 PCM 数据接收

```
int32_t AudioDeviceBuffer::SetRecordedBuffer(
    const void* audio_buffer, size_t samples_per_channel,
    std::optional<int64_t> capture_timestamp_ns) {
    // 拷贝到 rec_buffer_ (无锁, 因为此时只有音频线程访问)
    rec_buffer_.SetData(static_cast<const int16_t*>(audio_buffer),
        rec_channels_ * samples_per_channel);
    // 每50次回调 (约500ms) 计算一次音量峰值
    if (++rec_stat_count_ >= 50) {
        max_abs = WebRtcSpl_MaxAbsValueW16(rec_buffer_.data(), rec_buffer_.size());
        rec_stat_count_ = 0;
        if (max_abs > 0) only_silence_recorded_ = false;
    }
    UpdateRecStats(max_abs, samples_per_channel); // 加锁更新统计
}
```

逐行分析

参数解析：

参数	类型	含义
audio_buffer	const void*	平台提供的原始 PCM 缓冲区指针
samples_per_channel	size_t	每通道采样数（如 48kHz/10ms = 480）
capture_timestamp_ns	std::optional<int64_t>	可选的硬件捕获时间戳（纳秒）

第一步：数据拷贝

```
rec_buffer_.SetData(static_cast<const int16_t*>(audio_buffer),
                    rec_channels_ * samples_per_channel);
```

- rec_buffer_ 类型为 rtc::BufferT<int16_t>，是 WebRTC 自定义的动态数组
- 总样本数 = 通道数 × 每通道采样数（交错格式：L R L R ...）
- SetData 会自动处理内存分配和大小调整
- 无锁设计：rec_buffer_ 只在音频回调线程中被读写，不存在竞争

为什么用 void* 而不是 int16_t*？

这是历史设计决策。早期 WebRTC 支持多种采样格式（8bit、16bit、float），虽然现在内部统一使用 16-bit PCM，但接口保留了 void* 以兼容不同平台实现。

第二步：周期性音量统计

```
if (++rec_stat_count_ >= 50) {
    max_abs = WebRtcSpl_MaxAbsValueW16(rec_buffer_.data(), rec_buffer_.size());
    rec_stat_count_ = 0;
    if (max_abs > 0) only_silence_recorded_ = false;
}
```

- 每 50 次回调计算一次（假设 10ms 回调周期，即每 500ms 一次）
- WebRtcSpl_MaxAbsValueW16 是 WebRTC SPL（Signal Processing Library）函数
- 找出缓冲区中绝对值最大的样本，用于音量指示
- only_silence_recorded_ 标志：一旦检测到非零样本，永远设为 false
- 这个标志用于诊断：如果长时间只有静音，说明麦克风可能有问题

设计决策：为什么是 50 次而不是每次都计算？

音量峰值计算需要遍历整个缓冲区（480 个样本 × 通道数），虽然计算量不大，但在音频回调线程中应尽量减少不必要的工作。500ms 的更新频率对于 UI 显示和诊断日志已经足够。

第三步：更新统计信息

```
UpdateRecStats(max_abs, samples_per_channel); // 加锁更新统计
```

- `UpdateRecStats` 内部使用 `Mutex` 保护统计数据
- 因为统计数据会被主线程（通过 `TaskQueue`）读取用于日志输出
- 这是录音路径中唯一需要加锁的地方

Step 2: SetVQEData — 设置 VQE 延迟参数

```
void AudioDeviceBuffer::SetVQEData(int play_delay_ms, int rec_delay_ms) {  
    play_delay_ms_ = play_delay_ms; // 无锁, 音频线程独占  
    rec_delay_ms_ = rec_delay_ms;  
}
```

逐行分析

VQE 是什么？

VQE = Voice Quality Enhancement, 包括 AEC（回声消除）、NS（噪声抑制）等。
AEC 需要知道播放延迟和录音延迟，才能正确对齐参考信号和麦克风信号。

参数含义：

参数	含义	典型值
<code>play_delay_ms</code>	从混音器输出到扬声器发声的延迟	20-80ms
<code>rec_delay_ms</code>	从麦克风采集到数据到达回调的延迟	10-50ms

无锁设计的理由：

- `play_delay_ms_` 和 `rec_delay_ms_` 只在音频回调线程中被写入和读取
- 写入发生在 `SetVQEData`，读取发生在 `DeliverRecordedData`
- 两者在同一个回调周期内顺序执行，不存在并发访问
- 因此不需要任何同步原语

平台如何获取这些延迟值？

- **iOS/macOS (CoreAudio)**: 通过 `AudioSession` 的 `outputLatency` 和 `inputLatency`
- **Android (AAudio)**: 通过 `AAudioStream_getTimestamp` 计算
- **Linux (ALSA)**: 通过 `snd_pcm_delay` 获取

Step 3: DeliverRecordedData — 数据交付给上层

```
int32_t AudioDeviceBuffer::DeliverRecordedData() {
    // 调用 AudioTransport::RecordedDataIsAvailable
    uint32_t total_delay_ms = play_delay_ms_ + rec_delay_ms_;
    audio_transport_cb_>RecordedDataIsAvailable(
        rec_buffer_.data(), frames, bytes_per_frame, rec_channels_,
        rec_sample_rate_, total_delay_ms, 0, 0, typing_status_,
        new_mic_level_dummy, capture_timestamp_ns_);
}
```

逐行分析

延迟计算：

```
uint32_t total_delay_ms = play_delay_ms_ + rec_delay_ms_;
```

- AEC 需要的是总往返延迟（round-trip delay）
- 总延迟 = 播放延迟 + 录音延迟
- 这告诉 AEC：从远端音频被送入播放管线，到它被麦克风捕获回来，总共经过多少毫秒
- AEC 用这个值在参考信号中找到正确的对齐位置

AudioTransport::RecordedDataIsAvailable 参数：

参数	含义
<code>rec_buffer_.data()</code>	PCM 数据指针
<code>frames</code>	帧数（每通道采样数）
<code>bytes_per_frame</code>	每帧字节数（通道数 × 2）
<code>rec_channels_</code>	通道数
<code>rec_sample_rate_</code>	采样率
<code>total_delay_ms</code>	总延迟，给 AEC 使用
<code>0, 0</code>	clock_drift（已废弃）
<code>typing_status_</code>	是否检测到键盘敲击
<code>new_mic_level_dummy</code>	输出参数：建议的新麦克风音量
<code>capture_timestamp_ns_</code>	硬件捕获时间戳

audio_transport_cb_ 的安全性：

- 这是一个裸指针，没有用 `std::atomic` 或锁保护
- 安全性通过协议约束保证：`RegisterAudioCallback` 只能在媒体未激活时调用
- 即：必须先 `StopRecording()` / `StopPlayout()`，才能更改回调

- 一旦录音/播放开始，指针不会被修改，音频线程可以安全读取

录音路径时序图



播放路径完整分析

播放路径同样由平台音频线程驱动，但方向相反——这是一个 **pull 模型**：
平台硬件需要数据时主动向 WebRTC 拉取。

Step 1: RequestPlayoutData — 向上层拉取混音数据

```
int32_t AudioDeviceBuffer::RequestPlayoutData(size_t samples_per_channel) {
    // 动态调整 play_buffer_ 大小 (首次或大小变化时)
    const size_t total_samples = play_channels_ * samples_per_channel;
    if (play_buffer_.size() != total_samples) {
        play_buffer_.SetSize(total_samples);
    }
    // 调用 AudioTransport::NeedMorePlayData 拉取混音后的PCM
    audio_transport_cb_>NeedMorePlayData(
        samples_per_channel, bytes_per_frame, play_channels_, play_sample_rate_,
        play_buffer_.data(), num_samples_out, &elapsed_time_ms, &ntp_time_ms);
    return static_cast<int32_t>(num_samples_out / play_channels_);
}
```

逐行分析

缓冲区动态调整：

```
const size_t total_samples = play_channels_ * samples_per_channel;
if (play_buffer_.size() != total_samples) {
    play_buffer_.SetSize(total_samples);
}
```

- `play_buffer_` 类型为 `rtc::BufferT<int16_t>`
- 只在大小不匹配时才调整，避免每次回调都分配内存
- 大小变化场景：首次调用、采样率切换（如蓝牙设备切换）、通道数变化
- `play_channels_` 是 `std::atomic<size_t>`，这里的 `.load()` 是隐式的

为什么 `play_channels_` 用 `atomic`?

`play_channels_` 可能被主线程通过 `SetPlayoutChannels()` 修改，而音频线程在 `RequestPlayoutData` 中读取。虽然实际上通道数很少在播放过程中改变，但 WebRTC 采用防御性编程，用 `atomic` 消除数据竞争的可能性。

拉取混音数据：

```
audio_transport_cb_ -> NeedMorePlayData(
    samples_per_channel, bytes_per_frame, play_channels_, play_sample_rate_,
    play_buffer_.data(), num_samples_out, &elapsed_time_ms, &ntp_time_ms);
```

NeedMorePlayData 参数详解：

参数	方向	含义
<code>samples_per_channel</code>	输入	请求的每通道采样数
<code>bytes_per_frame</code>	输入	每帧字节数（通道数 × sizeof(int16_t)）
<code>play_channels_</code>	输入	播放通道数
<code>play_sample_rate_</code>	输入	播放采样率
<code>play_buffer_.data()</code>	输出	混音后的 PCM 数据写入此处
<code>num_samples_out</code>	输出	实际写入的总样本数
<code>elapsed_time_ms</code>	输出	已播放时长（用于同步）
<code>ntp_time_ms</code>	输出	NTP 时间戳（用于音视频同步）

返回值：

```
return static_cast<int32_t>(num_samples_out / play_channels_);
```

- 返回每通道的采样数（不是总样本数）
- 除以通道数是因为 `num_samples_out` 是交错格式的总样本数

- 平台实现用这个值知道实际获得了多少数据

NeedMorePlayData 内部做了什么？

这个调用会触发整个播放管线：

1. AudioManager 混合所有活跃的音频源
2. APM (AudioProcessingModule) 对混音结果做后处理
3. 重采样到目标采样率（如果需要）
4. 写入 `play_buffer_`

Step 2: GetPlayoutData — 数据拷贝到硬件缓冲区

```
int32_t AudioDeviceBuffer::GetPlayoutData(void* audio_buffer) {
    memcpy(audio_buffer, play_buffer_.data(),
           play_buffer_.size() * sizeof(int16_t));
    return static_cast<int32_t>(play_buffer_.size() / play_channels_);
}
```

逐行分析

为什么分成两步（Request + Get）？

这是一个重要的设计决策：

1. 解耦数据获取和数据消费：某些平台实现需要在两步之间做额外处理
2. 支持 **FineAudioBuffer**：FineAudioBuffer 可能一次 Request 10ms 数据，但分多次 Get 给平台（每次 128 samples）
3. 历史兼容：早期某些平台需要在 Request 和 Get 之间插入信号处理

memcpy 的安全性：

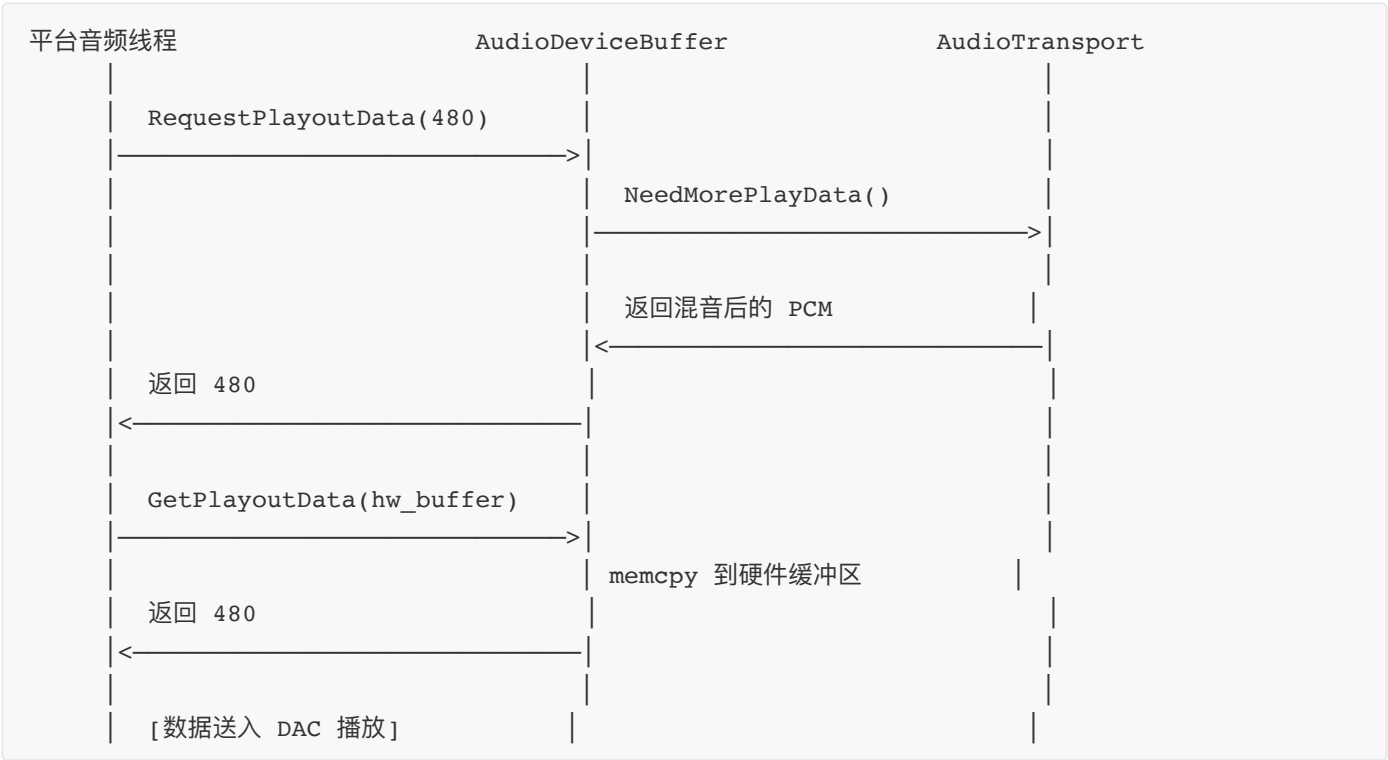
```
memcpy(audio_buffer, play_buffer_.data(),
       play_buffer_.size() * sizeof(int16_t));
```

- `audio_buffer` 是平台提供的硬件缓冲区指针
- 拷贝大小 = 样本数 × 每样本字节数（int16_t = 2 bytes）
- 无边界检查：调用者必须保证 `audio_buffer` 足够大
- 这是性能关键路径，省略检查是有意为之

无锁设计：

- `play_buffer_` 只在音频回调线程中被写入（RequestPlayoutData）和读取（GetPlayoutData）
 - 两个调用在同一个回调周期内顺序执行
 - 不存在并发访问，因此不需要锁
-

播放路径时序图



录音 vs 播放路径对比

特性	录音路径	播放路径
模型	Push（平台推数据）	Pull（平台拉数据）
步骤数	3 步	2 步
数据流向	平台 → Buffer → Transport	Transport → Buffer → 平台
额外信息	VQE 延迟参数	NTP 时间戳
缓冲区	rec_buffer_	play_buffer_
上层接口	RecordedDataIsAvailable	NeedMorePlayData

线程安全设计深度分析

AudioDeviceBuffer 的线程安全设计是 WebRTC 音频模块中最精妙的部分之一。它没有简单地用一把大锁保护所有状态，而是根据每个成员变量的访问模式，选择了最合适的同步策略。

线程模型

AudioDeviceBuffer 涉及三类线程：

主线程 (Main Thread) - Init/Start - Stop - SetChannels - SetSampleRate - RegisterCB	音频回调线程 (Audio Thread) - SetRecorded - DeliverData - RequestPlayout - GetPlayout - SetVQEData	TaskQueue (Stats Timer) - LogStats - ResetCounters
--	--	--

同步策略总览

成员变量	类型	同步机制	原因
rec_sample_rate_	std::atomic<uint32_t>	原子操作	主线程写，音频线程读
play_sample_rate_	std::atomic<uint32_t>	原子操作	主线程写，音频线程读
rec_channels_	std::atomic<size_t>	原子操作	主线程写，音频线程读
play_channels_	std::atomic<size_t>	原子操作	主线程写，音频线程读
audio_transport_cb_	裸指针	协议约束	只在媒体未激活时修改
rec_buffer_	BufferT<int16_t>	无锁	只在音频线程访问
play_buffer_	BufferT<int16_t>	无锁	只在音频线程访问
play_delay_ms_	int	无锁	只在音频线程访问
rec_delay_ms_	int	无锁	只在音频线程访问
stats_	结构体	Mutex	主线程和 TaskQueue 都访问
playing_	bool	SequenceChecker	只在主线程修改和检查
recording_	bool	SequenceChecker	只在主线程修改和检查

策略 1: std::atomic — 采样率和通道数

```
// audio_device_buffer.h
std::atomic<uint32_t> rec_sample_rate_{0};
std::atomic<uint32_t> play_sample_rate_{0};
std::atomic<size_t> rec_channels_{0};
std::atomic<size_t> play_channels_{0};
```

为什么用 atomic 而不是 Mutex?

- 1. 性能：这些值在音频回调线程的热路径上被读取（每 10ms 一次）。Mutex 的 lock/unlock 开销（即使无竞争时也有 ~25ns）对音频线程来说是不可接受的。atomic load 在 x86 上是零开销（普通 load 指令），在 ARM 上也只是一个 barrier。
- 2. 语义匹配：这些值是简单的标量，写入是原子的（单次赋值），不需要保护多个变量之间的一致性。
- 3. 访问模式：典型的 single-writer-multiple-reader 模式。主线程在初始化时写入，音频线程在运行时读取。写入频率极低（几乎只在启动时）。

内存序（Memory Order）考虑：

WebRTC 使用默认的 `memory_order_seq_cst`（顺序一致性），这是最保守的选择。对于这种低频写入的场景，性能差异可以忽略不计，但提供了最强的正确性保证。

为什么不用 `relaxed`？

虽然 `memory_order_relaxed` 在这里可能足够（因为没有依赖关系需要排序），但 WebRTC 团队选择了安全优先。在音频代码中，正确性远比微小的性能提升重要。

策略 2: 协议约束 — `audio_transport_cb_`

```
// audio_device_buffer.h
AudioTransport* audio_transport_cb_; // 裸指针，无同步原语
```

这看起来很危险，为什么是安全的？

安全性通过 API 协议保证，而不是通过编译器或运行时检查：

```
// RegisterAudioCallback 的实现（概念性）
int32_t AudioDeviceBuffer::RegisterAudioCallback(AudioTransport* transport) {
    RTC_DCHECK(!playing_); // 断言：播放未激活
    RTC_DCHECK(!recording_); // 断言：录音未激活
    audio_transport_cb_ = transport;
    return 0;
}
```

协议规则：

1. `RegisterAudioCallback` 只能在 `StartPlayout()` 之前或 `StopPlayout()` 之后调用
2. 一旦播放/录音开始，`audio_transport_cb_` 不会被修改
3. 音频线程只在播放/录音激活期间运行
4. 因此，音频线程读取 `audio_transport_cb_` 时，没有其他线程在写入

为什么不用 `atomic` 或 `shared_ptr`？

1. **`atomic` 指针**：语义上可以，但增加了不必要的复杂性。
而且 `atomic` 指针不能保护指针指向的对象的生命周期。
2. **`shared_ptr`**：会引入引用计数的原子操作开销，而且 `AudioTransport` 的生命周期由上层管理，不应该被 `AudioDeviceBuffer` 延长。
3. **协议约束是最轻量的方案**：零运行时开销，通过 `RTC_DCHECK` 在调试模式下验证。

风险和缓解：

- 风险：如果调用者违反协议（在播放中更改回调），会导致未定义行为
 - 缓解：`RTC_DCHECK` 在 debug build 中会立即崩溃，帮助开发者发现错误
 - 在 release build 中，`DCHECK` 被编译掉，但协议违反的概率极低
-

策略 3: 无锁 — 音频线程独占数据

```
// 这些成员只在音频回调线程中访问
rtc::BufferT<int16_t> rec_buffer_;
rtc::BufferT<int16_t> play_buffer_;
int play_delay_ms_ = 0;
int rec_delay_ms_ = 0;
```

为什么可以无锁?

音频回调线程的执行模型保证了这些数据的独占访问:

音频回调线程的一次迭代:

1. SetRecordedBuffer()	→ 写 rec_buffer
2. SetVQEData()	→ 写 delay
3. DeliverRecordedData()	→ 读 rec_buffer
	读 delay
4. RequestPlayoutData()	→ 写 play_buffer
5. GetPlayoutData()	→ 读 play_buffer

- 所有读写都在同一个线程的同一次迭代中完成
- 没有其他线程会访问这些缓冲区
- 这是最高效的同步策略: 完全没有同步开销

为什么不用 SequenceChecker 验证?

实际上 WebRTC 可能使用了 `RTC_DCHECK_RUN_ON(&audio_thread_checker_)` 来验证这些方法确实在音频线程上被调用。SequenceChecker 在 debug build 中检查线程身份, 在 release build 中被编译掉, 零开销。

策略 4: Mutex — 统计数据

```
// 统计数据需要 Mutex 保护
mutable Mutex stats_mutex_;
struct Stats {
    int64_t rec_samples = 0;
    int64_t play_samples = 0;
    int16_t max_rec_level = 0;
    int16_t max_play_level = 0;
} stats_ RTC_GUARDED_BY(stats_mutex_);
```

为什么统计数据需要 Mutex?

统计数据被多个线程访问:

- 音频线程: 每次回调更新计数器和峰值

- **TaskQueue**: 定期（每 10 秒）读取统计数据输出日志
- **主线程**: 可能通过 API 查询当前音量级别

为什么不用 `atomic`?

统计数据是一个结构体，包含多个相关字段。如果用 `atomic` 保护每个字段，无法保证多个字段之间的一致性。例如：

```
// 错误示例: atomic 无法保证一致性
atomic<int64_t> rec_samples;
atomic<int16_t> max_rec_level;

// 线程 A (音频线程) :
rec_samples.store(1000);
max_rec_level.store(32000);

// 线程 B (TaskQueue) 可能看到:
// rec_samples = 1000, max_rec_level = 0 ← 不一致!
```

Mutex 保证了读取统计数据时看到的是一个一致的快照。

性能影响：

- 音频线程每次回调都要获取锁（在 `UpdateRecStats` / `UpdatePlayStats` 中）
- 但竞争概率极低：TaskQueue 每 10 秒才读一次
- 无竞争的 Mutex lock/unlock 在现代系统上约 25ns，对 10ms 的回调周期来说可以忽略
- WebRTC 使用的是 `webrtc::Mutex`（基于 abseil 的 `absl::Mutex`），在无竞争时使用 `futex`，性能优于 `std::mutex`

策略 5: SequenceChecker — 状态标志

```
// 只在主线程访问的状态
SequenceChecker main_thread_checker_;
bool playing_ RTC_GUARDED_BY(main_thread_checker_) = false;
bool recording_ RTC_GUARDED_BY(main_thread_checker_) = false;
```

SequenceChecker 是什么？

- 不是锁，不提供运行时同步
- 是一个调试工具，在 debug build 中验证访问是否来自正确的线程/序列
- 在 release build 中完全被编译掉，零开销
- 配合 `RTC_GUARDED_BY` 注解，让 Clang Thread Safety Analysis 在编译时检查

为什么 `playing/recording` 不需要锁？

- 这些标志只在主线程中被读写（`StartPlayout`、`StopPlayout`、`StartRecording`、`StopRecording`）
- 音频线程不直接读取这些标志

- 音频线程的启动/停止通过平台 API 控制，与这些标志的修改在同一个主线程调用中完成

设计哲学：

WebRTC 的线程安全设计遵循一个原则：**用最轻量的机制保证正确性。**

无锁（线程独占）< SequenceChecker < atomic < Mutex < 读写锁

↑

最快，零开销

↑

最慢，最重

每个成员变量都被分析其访问模式，然后选择刚好足够的同步机制。
这种精细化的设计在音频这种对延迟极度敏感的场景中至关重要。

FineAudioBuffer 原理

问题背景

源码文件：`modules/audio_device/fine_audio_buffer.h`、`modules/audio_device/fine_audio_buffer.cc`

WebRTC 内部音频处理管线要求固定的 **10ms 帧长**。这是一个硬性约束：

- APM（音频处理模块）的所有算法都基于 10ms 帧设计
- AudioMixer 混音器期望 10ms 的输入
- 编解码器（Opus）通常以 10ms 或 20ms 为单位编码

但不同平台的音频硬件有不同的缓冲区大小：

平台	典型缓冲区大小	对应时长（48kHz）
iOS (CoreAudio)	128 frames	2.67ms
iOS (低延迟模式)	128 frames	2.67ms
macOS (CoreAudio)	512 frames	10.67ms
Android (AAudio)	192 frames	4ms
Android (OpenSL ES)	240 frames	5ms

核心矛盾：平台给的数据量 ≠ WebRTC 需要的数据量。

解决方案：FineAudioBuffer 适配层

FineAudioBuffer 作为平台音频实现和 AudioDeviceBuffer 之间的适配层：



录音方向：积累后推送

问题：iOS 每次回调给 128 个样本（2.67ms），但 AudioDeviceBuffer 需要 480 个样本（10ms）。

解决：FineAudioBuffer 内部维护一个环形缓冲区，积累多次回调的数据，凑够 480 个样本后才调用 `AudioDeviceBuffer::SetRecordedBuffer` + `DeliverRecordedData`。

```
// FineAudioBuffer::DeliverRecordedData 概念性实现
void FineAudioBuffer::DeliverRecordedData(
    rtc::ArrayView<const int16_t> audio_buffer,
    int playout_delay_ms, int record_delay_ms) {
    // 将平台数据追加到内部缓冲区
    record_buffer_.AppendData(audio_buffer.data(), audio_buffer.size());

    // 当积累够 10ms 数据时，推送给 AudioDeviceBuffer
    const size_t required_samples = channels_ * samples_per_10ms_;
    while (record_buffer_.size() >= required_samples) {
        // 取出前 10ms 数据
        audio_device_buffer_ -> SetRecordedBuffer(
            record_buffer_.data(), samples_per_10ms_);
        audio_device_buffer_ -> SetVQEDData(playout_delay_ms, record_delay_ms);
        audio_device_buffer_ -> DeliverRecordedData();

        // 移除已处理的数据
        record_buffer_.EraseBegin(required_samples);
    }
}
```

时序示例（iOS, 48kHz, 128 frames/回调）：

```
回调 1：收到 128 samples → 缓冲区：128（不够 480）
回调 2：收到 128 samples → 缓冲区：256（不够 480）
回调 3：收到 128 samples → 缓冲区：384（不够 480）
回调 4：收到 128 samples → 缓冲区：512（≥ 480!）
        → 推送 480 给 AudioDeviceBuffer
        → 缓冲区剩余：32
回调 5：收到 128 samples → 缓冲区：160（不够 480）
...
```

关键观察：每 3-4 次平台回调，产生一次 10ms 的 WebRTC 回调。
这意味着 WebRTC 的回调频率不是严格的 100Hz，而是有轻微的抖动。
但这对音频质量没有影响，因为数据量是精确的。

播放方向：预取后切片

问题：AudioDeviceBuffer 一次给 480 个样本（10ms），但 iOS 每次只需要 128 个样本。

解决：FineAudioBuffer 预先从 AudioDeviceBuffer 拉取 10ms 数据，然后按平台需要的大小切片返回。

```
// FineAudioBuffer::GetPlayoutData 概念性实现
void FineAudioBuffer::GetPlayoutData(
    rtc::ArrayView<int16_t> audio_buffer, int playout_delay_ms) {
    const size_t requested_samples = audio_buffer.size();

    // 如果内部缓冲区不够，从 AudioDeviceBuffer 拉取 10ms
    while (playout_buffer_.size() < requested_samples) {
        // 拉取 10ms 数据
        audio_device_buffer_>RequestPlayoutData(samples_per_10ms_);
        // 获取数据到临时缓冲区
        int16_t temp[480 * 2]; // 最大: 480 samples * 2 channels
        audio_device_buffer_>GetPlayoutData(temp);
        // 追加到内部缓冲区
        playout_buffer_.AppendData(temp, channels_ * samples_per_10ms_);
    }

    // 从内部缓冲区取出平台需要的量
    memcpy(audio_buffer.data(), playout_buffer_.data(),
           requested_samples * sizeof(int16_t));
    playout_buffer_.EraseBegin(requested_samples);
}
```

时序示例（iOS, 48kHz, 128 frames/回调）：

```
回调 1: 需要 128 samples
    → 缓冲区空，从 ADB 拉取 480
    → 返回 128，缓冲区剩余：352
回调 2: 需要 128 samples
    → 缓冲区有 352，够用
    → 返回 128，缓冲区剩余：224
回调 3: 需要 128 samples
    → 缓冲区有 224，够用
    → 返回 128，缓冲区剩余：96
回调 4: 需要 128 samples
    → 缓冲区只有 96，不够！
    → 从 ADB 再拉取 480，缓冲区：576
    → 返回 128，缓冲区剩余：448
...
```

FineAudioBuffer 的设计决策

为什么不让 AudioDeviceBuffer 支持任意帧长？

- 1. **简化核心逻辑**：AudioDeviceBuffer 只需处理固定的 10ms，代码更简单、更可靠
- 2. **APM 约束**：音频处理算法（AEC、NS、AGC）都是基于 10ms 设计的，改变帧长需要重写所有算法
- 3. **关注点分离**：帧长适配是平台相关的问题，不应该污染核心音频管线

为什么不在平台层做重采样/帧长对齐？

- 1. **代码复用**：FineAudioBuffer 是跨平台的，避免每个平台实现都写一遍
- 2. **正确性**：帧长对齐看似简单，但边界条件很多（余数处理、缓冲区溢出等）
- 3. **可测试性**：独立的适配层更容易单元测试

延迟影响

FineAudioBuffer 引入了额外的缓冲延迟：

- **录音方向**：最多额外 9.99ms 延迟（需要凑够 10ms）
 - 平均额外延迟 $\approx 5\text{ms}$
- **播放方向**：最多额外 10ms 延迟（预取了一整帧但只用了一部分）
 - 平均额外延迟 $\approx 5\text{ms}$

对于实时通信来说，这 5-10ms 的额外延迟是可以接受的。
总端到端延迟通常在 100-300ms，FineAudioBuffer 的贡献很小。

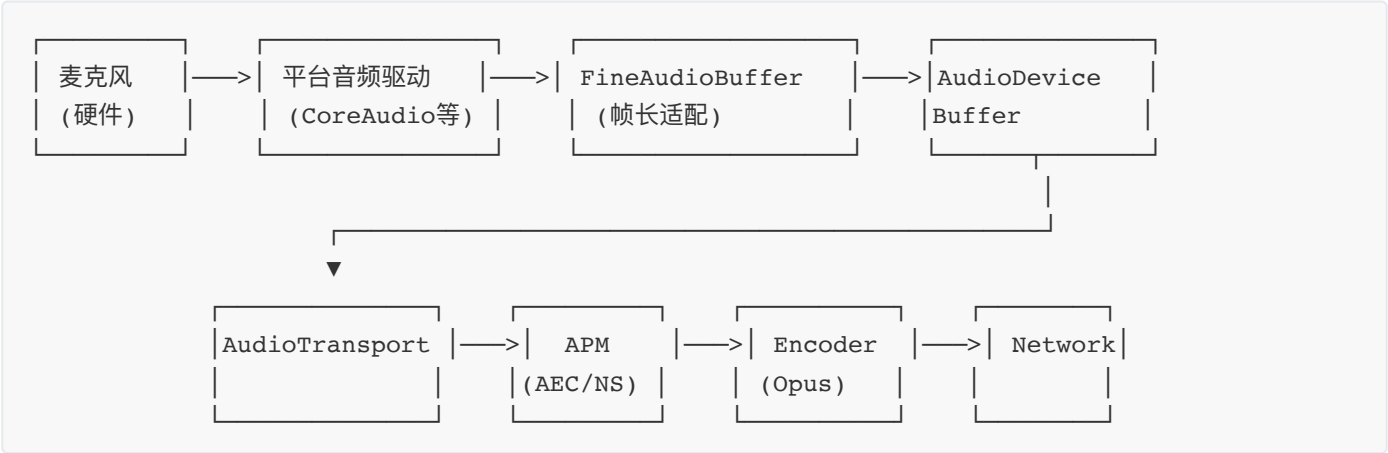
内存管理

FineAudioBuffer 使用 `rtc::BufferT<int16_t>` 作为内部缓冲区：

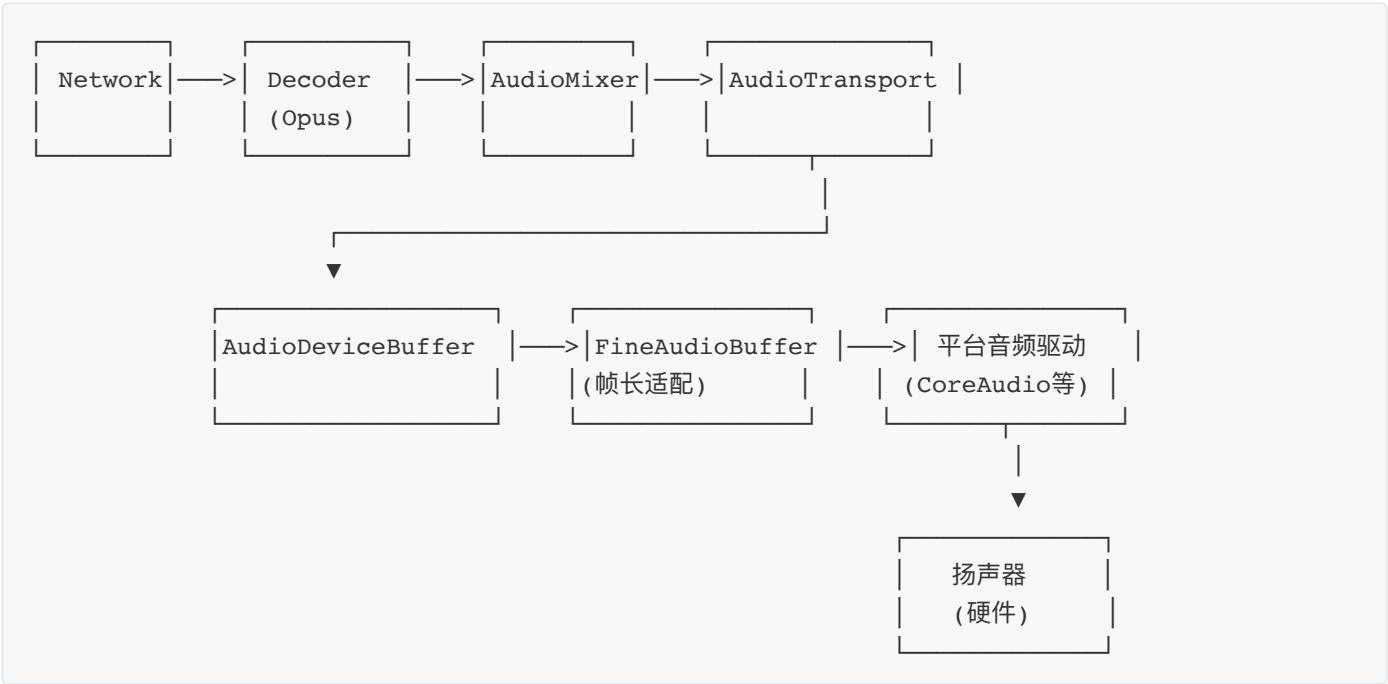
- 初始分配后很少需要重新分配（大小在 0 到 ~960 samples 之间波动）
- `EraseBegin` 操作需要 memmove，但数据量很小（最多 ~2KB），开销可忽略
- 没有使用环形缓冲区（ring buffer），因为数据量太小，环形缓冲区的复杂性不值得

完整数据流总结

录音完整路径（从麦克风到网络）



播放完整路径（从网络到扬声器）



关键设计原则总结

1. 最小同步原则

每个成员变量使用刚好足够的同步机制，不多不少：

- 能无锁就无锁（线程独占数据）
- 能用 atomic 就不用 Mutex（简单标量）
- 能用协议约束就不用运行时检查（指针生命周期）
- 必须用 Mutex 时才用 Mutex（复合状态）

2. 音频线程零阻塞

音频回调线程是整个系统中延迟最敏感的线程：

- 回调周期通常只有 2-10ms
- 如果回调超时，会导致音频断裂 (glitch)
- 因此音频线程中的操作必须是确定性的、低延迟的
- AudioDeviceBuffer 在音频线程中几乎不做任何可能阻塞的操作
- 唯一的例外是 `UpdateRecStats` 中的 Mutex，但竞争概率极低

3. 固定帧长简化设计

WebRTC 选择 10ms 作为内部统一帧长：

- 所有音频处理算法基于 10ms 设计

- AudioDeviceBuffer 只处理 10ms 的数据块
- 帧长适配的复杂性被隔离在 FineAudioBuffer 中
- 这大大简化了核心管线的实现

4. Push/Pull 混合模型

- 录音是 Push：硬件产生数据后推送，延迟最小
- 播放是 Pull：硬件需要数据时拉取，避免缓冲区溢出
- 这种混合模型匹配了音频硬件的自然工作方式

源码文件索引

文件路径	内容
<code>modules/audio_device/audio_device_buffer.h</code>	AudioDeviceBuffer 类声明
<code>modules/audio_device/audio_device_buffer.cc</code>	AudioDeviceBuffer 实现
<code>modules/audio_device/fine_audio_buffer.h</code>	FineAudioBuffer 类声明
<code>modules/audio_device/fine_audio_buffer.cc</code>	FineAudioBuffer 实现
<code>modules/audio_device/include/audio_device.h</code>	AudioDeviceModule 接口
<code>api/audio/audio_transport.h</code>	AudioTransport 接口定义
<code>common_audio/signal_processing/min_max_operations.c</code>	WebRtcSpl_MaxAbsValueW16

附录：常见问题

Q: 为什么 AudioDeviceBuffer 不直接使用环形缓冲区？

A: 因为 AudioDeviceBuffer 不做缓冲。它是一个同步转发器：

录音数据进来后立即转发给 AudioTransport，播放数据从 AudioTransport 拉取后立即返回。

`rec_buffer_` 和 `play_buffer_` 只是临时存储，不是队列。

Q: 如果 AudioTransport::NeedMorePlayData 执行太慢怎么办？

A: 这会导致音频回调超时，平台可能会：

1. 重复播放上一帧（产生卡顿感）
2. 播放静音
3. 在日志中报告 underrun

WebRTC 通过以下方式缓解：

- AudioMixer 的混音操作是 $O(n)$ ， n 是活跃源数量，通常很快
- APM 处理是固定时间的（每帧约 1-2ms）
- 如果确实太慢，NetEQ 的 jitter buffer 会吸收部分抖动

Q: capture_timestamp_ns 有什么用？

A: 这是硬件级别的捕获时间戳，用于：

- 1. 精确的音视频同步（lip sync）
- 2. AEC 的精确延迟估计
- 3. 网络抖动补偿

不是所有平台都支持硬件时间戳，所以它是 `std::optional`。
如果平台不提供，WebRTC 会使用软件时间戳作为 fallback。

Q: typing_status_ 是什么？

A: 键盘敲击检测状态。WebRTC 有一个 typing detection 模块，通过分析麦克风信号中的瞬态脉冲来检测用户是否在打字。
这个信息传递给远端，远端可以选择在对方打字时降低音量或显示提示。

AudioTransportImpl 数据管线深度解析

概述

`AudioTransportImpl` 是 WebRTC 音频数据流的核心枢纽，实现了 `AudioTransport` 接口，连接底层音频设备（ADM）与上层音频处理/编码模块。它承担以下职责：

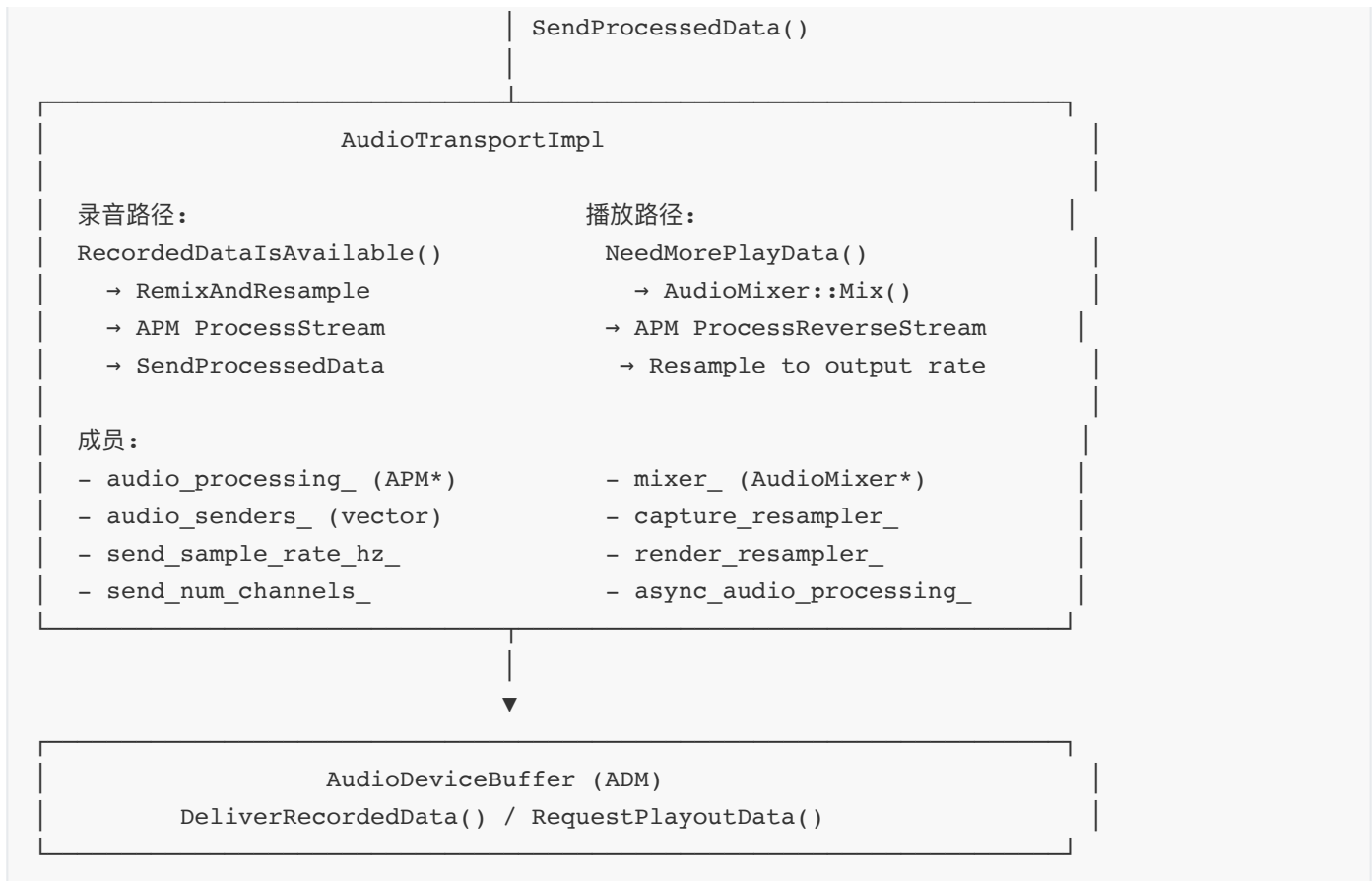
- 1. **录音路径**：接收 ADM 推送的原始 PCM，经 APM 处理后分发给各 `AudioSender`
- 2. **播放路径**：从 `AudioMixer` 拉取混音数据，经 APM 反向处理后返回给 ADM
- 3. **格式转换**：在不同采样率/通道数之间进行 Remix 和 Resample
- 4. **多流分发**：支持同时向多个发送流分发处理后的音频帧

源码文件依据：

- `audio/audio_transport_impl.h`
- `audio/audio_transport_impl.cc`
- `audio/audio_state.cc` (`UpdateAudioTransportWithSendingStreams`)
- `modules/audio_mixer/audio_mixer_impl.h`
- `modules/audio_mixer/audio_mixer_impl.cc`

架构位置





类定义与构造

头文件结构 (audio/audio_transport_impl.h)

```
class AudioTransportImpl : public AudioTransport {
public:
    AudioTransportImpl(
        AudioMixer* mixer,
        AudioProcessing* audio_processing,
        AsyncAudioProcessing::Factory* async_audio_processing_factory);
    ~AudioTransportImpl() override;

    // AudioTransport 接口实现
    int32_t RecordedDataIsAvailable(...) override;
    int32_t NeedMorePlayData(...) override;
    void PullRenderData(...) override;

    // 由 AudioState 调用, 更新发送流列表
    void UpdateAudioSenders(std::vector<AudioSender*> senders,
                           int send_sample_rate_hz,
                           size_t send_num_channels);

    // 处理完成后的分发
    void SendProcessedData(std::unique_ptr<AudioFrame> audio_frame);

private:
```



```

// 录音路径
Mutex capture_lock_;
std::vector<AudioSender*> audio_senders_ RTC_GUARDED_BY(capture_lock_);
int send_sample_rate_hz_ RTC_GUARDED_BY(capture_lock_);
size_t send_num_channels_ RTC_GUARDED_BY(capture_lock_);
PushResampler<int16_t> capture_resampler_;

// 播放路径
AudioMixer* mixer_; // 不拥有所有权
AudioFrame mixed_frame_;
PushResampler<int16_t> render_resampler_;

// 共享
AudioProcessing* audio_processing_; // 不拥有所有权
std::unique_ptr<AsyncAudioProcessing> async_audio_processing_;
};

```

源码依据: `audio/audio_transport_impl.h`

构造函数分析

```

AudioTransportImpl::AudioTransportImpl(
    AudioMixer* mixer,
    AudioProcessing* audio_processing,
    AsyncAudioProcessing::Factory* async_audio_processing_factory)
: mixer_(mixer),
  audio_processing_(audio_processing),
  async_audio_processing_(
      async_audio_processing_factory
      ? async_audio_processing_factory->CreateAsyncAudioProcessing(
          [this](std::unique_ptr<AudioFrame> frame) {
              this->SendProcessedData(std::move(frame));
          })
      : nullptr) {}

```

关键设计点

1. 依赖注入: `mixer` 和 `audio_processing` 均为外部传入, `AudioTransportImpl` 不管理其生命周期
2. 异步处理可选: 只有传入 `async_audio_processing_factory` 时才创建异步处理器
3. 回调绑定: 异步处理完成后, 通过 lambda 回调 `SendProcessedData`

源码依据: `audio/audio_transport_impl.cc` 构造函数

录音路径完整分析: RecordedDataIsAvailable

这是整个录音数据管线的入口点, 由 `AudioDeviceBuffer::DeliverRecordedData()` 调用, 运行在平台音频回调线程上。

函数签名

```
int32_t AudioTransportImpl::RecordedDataIsAvailable(  
    const void* audio_data,  
    size_t number_of_frames,  
    size_t bytes_per_sample,  
    size_t number_of_channels,  
    uint32_t sample_rate,  
    uint32_t audio_delay_milliseconds,  
    int32_t /*clock_drift*/,  
    uint32_t /*volume*/,  
    bool key_pressed,  
    uint32_t& /*new_mic_volume*/,  
    int64_t estimated_capture_time_ns) {
```

参数	含义
audio_data	原始 PCM 数据指针（交错格式）
number_of_frames	每通道采样数（10ms 帧）
bytes_per_sample	每样本字节数（通常为 2，即 16-bit）
number_of_channels	输入通道数
sample_rate	输入采样率（Hz）
audio_delay_milliseconds	音频延迟（用于 AEC）
key_pressed	键盘按下标志（用于 AGC 键盘噪声抑制）
estimated_capture_time_ns	估计的硬件采集时间戳（纳秒）

Step 1: 帧长度校验

```
RTC_DCHECK_EQ(number_of_frames * 100, sample_rate);
```

原理：WebRTC 内部统一使用 10ms 帧。对于采样率 `fs`，10ms 对应的采样数为：

- `frames = fs / 100`
- 即 `frames * 100 == fs`

采样率	10ms 帧大小
8000 Hz	80 samples
16000 Hz	160 samples
32000 Hz	320 samples
44100 Hz	441 samples
48000 Hz	480 samples

这个断言确保 ADM 传入的数据严格是 10ms 的整数帧，这是后续所有处理的基本假设。

源码依据： `audio/audio_transport_impl.cc` `RecordedDataIsAvailable` 第一行

Step 2: 加锁读取发送参数

```
int send_sample_rate_hz = 0;
size_t send_num_channels = 0;
{
    MutexLock lock(&capture_lock_);
    send_sample_rate_hz = send_sample_rate_hz_;
    send_num_channels = send_num_channels_;
}
```

为什么需要锁？

`send_sample_rate_hz_` 和 `send_num_channels_` 由 `UpdateAudioSenders()` 在信令线程更新，而 `RecordedDataIsAvailable` 在音频回调线程执行。两个线程并发访问共享状态，必须加锁。

参数来源：这些值由 `AudioState::UpdateAudioTransportWithSendingStreams()` 计算：

- `send_sample_rate_hz`：所有活跃发送流中最高的编码器采样率
- `send_num_channels`：所有活跃发送流中最大的通道数

源码依据：

- `audio/audio_transport_impl.cc` `RecordedDataIsAvailable`
- `audio/audio_state.cc` `UpdateAudioTransportWithSendingStreams`

Step 3: 初始化采集帧（选择处理采样率）

```
std::unique_ptr<AudioFrame> audio_frame = std::make_unique<AudioFrame>();
InitializeCaptureFrame(sample_rate, send_sample_rate_hz,
                       number_of_channels, send_num_channels,
                       audio_frame.get());
```

`InitializeCaptureFrame` 的核心逻辑：

```

void InitializeCaptureFrame(int input_sample_rate,
                           int send_sample_rate,
                           size_t input_num_channels,
                           size_t send_num_channels,
                           AudioFrame* audio_frame) {
    // 选择处理采样率：取输入和发送中较低的
    int processing_rate = std::min(input_sample_rate, send_sample_rate);

    // 向上取整到 APM 支持的 native rate
    // Native rates: 8000, 16000, 32000, 48000
    processing_rate = FindNativeRate(processing_rate);

    // 通道数：取输入和发送中较少的
    size_t processing_channels = std::min(input_num_channels, send_num_channels);

    audio_frame->sample_rate_hz_ = processing_rate;
    audio_frame->num_channels_ = processing_channels;
    audio_frame->samples_per_channel_ = processing_rate / 100; // 10ms
}

```

设计决策：为什么取最低采样率？

- 如果输入是 48kHz 但编码器只需要 16kHz，没必要在 48kHz 上做 APM 处理
- 降低处理采样率可以显著减少 APM 的计算量
- APM 内部算法在 native rate 上工作效率最高

Native Rate 向上取整规则：

实际最小值	取整后
≤ 8000	8000
8001 ~ 16000	16000
16001 ~ 32000	32000
32001 ~ 48000	48000

例如：输入 44100Hz，发送 16000Hz → min = 16000 → native = 16000

源码依据： `audio/audio_transport_impl.cc` `InitializeCaptureFrame`

Step 4: Remix + Resample

```

voe::RemixAndResample(static_cast<const int16_t*>(audio_data),
                      number_of_frames,
                      number_of_channels,
                      sample_rate,
                      &capture_resampler_,
                      audio_frame.get());

```

这一步将输入音频从原始格式转换为 Step 3 确定的处理格式。

RemixAndResample 内部流程

```
void RemixAndResample(const int16_t* src_data,
                      size_t samples_per_channel,
                      size_t num_channels,
                      int sample_rate_hz,
                      PushResampler<int16_t>* resampler,
                      AudioFrame* dst_frame) {
    // 1. 如果通道数需要变化, 先做 Remix
    //     Stereo → Mono: (L + R) / 2
    //     Mono → Stereo: L = R = Mono
    const int16_t* remix_data = src_data;
    if (num_channels != dst_frame->num_channels_) {
        RemixFrame(src_data, samples_per_channel, num_channels,
                   dst_frame->num_channels_, remix_buffer);
        remix_data = remix_buffer.data();
    }

    // 2. 如果采样率需要变化, 做 Resample
    if (sample_rate_hz != dst_frame->sample_rate_hz_) {
        resampler->InitializeIfNeeded(sample_rate_hz,
                                      dst_frame->sample_rate_hz_,
                                      dst_frame->num_channels_);
        resampler->Resample(remix_data, remix_samples,
                           dst_frame->mutable_data(), &dst_samples);
    } else {
        // 采样率相同, 直接拷贝
        memcpy(dst_frame->mutable_data(), remix_data, ...);
    }
}
```

Resampler 实现： `PushResampler<int16_t>` 内部使用多相滤波器（polyphase filter），支持任意比率的高质量重采样。

处理顺序：先 Remix 再 Resample。这是因为：

- Remix（通道混合）是简单的算术运算，不依赖采样率
- 先减少通道数可以减少后续 Resample 的计算量

源码依据：

- `audio/remix_resample.h`
- `audio/remix_resample.cc`
- `common_audio/resampler/push_resampler.h`

Step 5: APM 处理（音频前处理）

```
ProcessCaptureFrame(audio_delay_milliseconds, key_pressed,
                    swap_stereo_channels, audio_processing, audio_frame.get());
```

这是录音路径中最关键的一步——音频前处理模块（Audio Processing Module）对采集信号进行处理。

ProcessCaptureFrame 内部实现

```
void ProcessCaptureFrame(uint32_t delay_ms,
                        bool key_pressed,
                        bool swap_stereo_channels,
                        AudioProcessing* audio_processing,
                        AudioFrame* audio_frame) {
    if (!audio_processing) return; // APM 可选

    // 设置流延迟（AEC 需要知道录音到播放的往返延迟）
    audio_processing->set_stream_delay_ms(delay_ms);

    // 设置键盘按下状态（AGC 可据此调整增益策略）
    audio_processing->set_stream_key_pressed(key_pressed);

    // 立体声通道交换（某些硬件左右声道接反）
    if (swap_stereo_channels)
        AudioFrameOperations::SwapStereoChannels(audio_frame);

    // 核心处理调用
    ProcessAudioFrame(audio_processing, audio_frame);
}
```

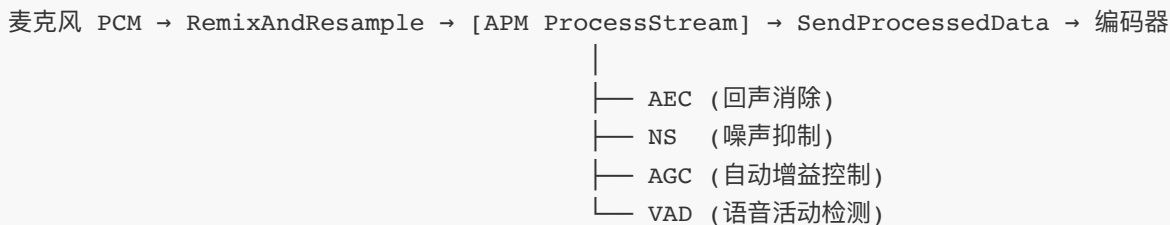
ProcessAudioFrame 详解

```
int ProcessAudioFrame(AudioProcessing* audio_processing,
                    AudioFrame* audio_frame) {
    // 将 AudioFrame 包装为 StreamConfig
    StreamConfig input_config(audio_frame->sample_rate_hz_,
                              audio_frame->num_channels_);
    StreamConfig output_config(audio_frame->sample_rate_hz_,
                               audio_frame->num_channels_);

    // 调用 APM 的 ProcessStream（就地处理）
    int result = audio_processing->ProcessStream(
        audio_frame->data(),
        input_config,
        output_config,
        audio_frame->mutable_data());

    return result;
}
```

APM 在录音路径中的位置



APM 处理的关键特性：

1. **就地处理 (in-place)**：输入和输出使用同一个 AudioFrame 缓冲区
2. **固定帧长**：APM 内部要求 10ms 帧，这就是为什么整个管线统一使用 10ms
3. **采样率要求**：APM 只接受 native rates (8k/16k/32k/48k)，这就是 Step 3 要向上取整的原因
4. **延迟信息**：AEC 需要 `stream_delay_ms` 来对齐远端参考信号和近端采集信号
5. **可选性**：如果 `audio_processing_` 为 nullptr，整个 APM 步骤被跳过

APM 不在此文件中做的事情：

- APM 的初始化和配置在 `AudioState` 或 `PeerConnectionFactory` 中完成
- APM 的反向流处理（AEC 参考信号）在播放路径的 `NeedMorePlayData` 中完成
- APM 内部的算法细节（滤波器系数、自适应算法等）不在本文档范围内

源码依据：

- `audio/audio_transport_impl.cc` `ProcessCaptureFrame`, `ProcessAudioFrame`
- `modules/audio_processing/include/audio_processing.h` `ProcessStream` 接口

Step 6: 设置采集时间戳

```
if (estimated_capture_time_ns > 0) {
    audio_frame->set_absolute_capture_timestamp_ms(
        estimated_capture_time_ns / rtc::kNumNanosecsPerMillisec);
}
```

用途：音视频同步 (A/V Sync) 。

- `estimated_capture_time_ns` 由 ADM 提供，表示硬件实际采集这帧音频的时间
- 转换为毫秒后存入 AudioFrame
- 下游的 RTP 发送模块会使用此时间戳计算 NTP 时间，实现与视频帧的同步
- 如果平台不支持提供硬件时间戳（值为 0），则跳过设置

源码依据：`audio/audio_transport_impl.cc` `RecordedDataIsAvailable`

Step 7: 分发给 AudioSender

```
if (async_audio_processing_) {
    async_audio_processing_>Process(std::move(audio_frame));
} else {
    SendProcessedData(std::move(audio_frame));
}
```

这里两条路径：

同步路径（默认）

直接在当前音频回调线程调用 `SendProcessedData`。

异步路径（可选）

将 `AudioFrame` 投递到独立的处理线程，处理完成后回调 `SendProcessedData`。

异步路径的使用场景：

- 当有额外的音频处理需要在独立线程完成时（如自定义 DSP）
- 避免在音频回调线程上做耗时操作导致音频卡顿
- Chromium 中某些平台使用此路径

源码依据：

- `audio/audio_transport_impl.h` `async_audio_processing_` 成员
- `audio/audio_transport_impl.cc` `RecordedDataIsAvailable` 末尾

SendProcessedData：多流分发机制

```
void AudioTransportImpl::SendProcessedData(
    std::unique_ptr<AudioFrame> audio_frame) {
    MutexLock lock(&capture_lock_);
    if (audio_senders_.empty())
        return;

    auto it = audio_senders_.begin();
    while (++it != audio_senders_.end()) {
        auto audio_frame_copy = std::make_unique<AudioFrame>();
        audio_frame_copy->CopyFrom(*audio_frame);
        (*it)->SendAudioData(std::move(audio_frame_copy));
    }
    (*audio_senders_.begin())->SendAudioData(std::move(audio_frame));
}
```

逐行分析

加锁保护：


```
MutexLock lock(&capture_lock_);
```

`audio_senders_` 可能被 `UpdateAudioSenders` 在其他线程修改，必须加锁。

空列表快速返回：

```
if (audio_senders_.empty()) return;
```

没有活跃的发送流时，直接丢弃帧。这发生在：

- 通话尚未建立
- 所有发送流已停止
- 静音（mute）状态下发送流被移除

复制分发策略：

```
// 除第一个外，其余 sender 都收到副本
while (++it != audio_senders_.end()) {
    auto audio_frame_copy = std::make_unique<AudioFrame>();
    audio_frame_copy->CopyFrom(*audio_frame);
    (*it)->SendAudioData(std::move(audio_frame_copy));
}
// 第一个 sender 收到原始帧（零拷贝优化）
(*audio_senders_.begin())->SendAudioData(std::move(audio_frame));
```

设计决策：

1. 最后处理第一个 **sender**：因为要把原始 frame move 给它，必须在所有副本创建完之后
2. 为什么不共享 **shared_ptr**？因为下游 sender 可能修改 frame（如添加元数据），共享会导致竞争
3. 单 **sender** 优化：最常见的场景是只有一个发送流，此时循环体不执行，零拷贝

AudioSender 接口：

```
class AudioSender {
public:
    virtual void SendAudioData(std::unique_ptr<AudioFrame> audio_frame) = 0;
};
```

实际实现是 `ChannelSend`，它会将 `AudioFrame` 送入编码器（如 Opus）。

源码依据：

- `audio/audio_transport_impl.cc` `SendProcessedData`
- `audio/channel_send.h` `AudioSender` 接口

播放路径完整分析：NeedMorePlayData

这是播放数据管线的入口点，由 `AudioDeviceBuffer::RequestPlayoutData()` 调用，运行在平台音频回调线程上。这是一个 **pull 模型**：硬件需要数据时主动拉取。

函数签名

```
int32_t AudioTransportImpl::NeedMorePlayData(
    size_t nSamples,
    size_t nBytesPerSample,
    size_t nChannels,
    uint32_t samplesPerSec,
    void* audioSamples,
    size_t& nSamplesOut,
    int64_t* elapsed_time_ms,
    int64_t* ntp_time_ms) {
```

参数	方向	含义
<code>nSamples</code>	输入	ADM 请求的每通道采样数
<code>nBytesPerSample</code>	输入	每样本字节数
<code>nChannels</code>	输入	ADM 请求的通道数
<code>samplesPerSec</code>	输入	ADM 请求的采样率
<code>audioSamples</code>	输出	填充混音后的 PCM 数据
<code>nSamplesOut</code>	输出	实际输出的每通道采样数
<code>elapsed_time_ms</code>	输出	混音帧的经过时间
<code>ntp_time_ms</code>	输出	混音帧的 NTP 时间戳

Step 1: AudioMixer 混合所有接收流

```
mixer_>Mix(nChannels, &mixed_frame_);
*elapsed_time_ms = mixed_frame_.elapsed_time_ms_;
*ntp_time_ms = mixed_frame_.ntp_time_ms_;
```

`AudioMixer::Mix()` 将所有已注册的音频源（远端接收流）混合为一个 `AudioFrame`。

混合过程概要：

- 遍历所有注册的 Source，调用 `GetAudioFrameWithInfo()` 获取各源的 10ms 帧
- 对所有活跃源的帧进行加权求和
- 结果写入 `mixed_frame_`

时间戳传递：

- `elapsed_time_ms_`：从混合帧中提取，表示帧的相对时间

- `ntp_time_ms_`: NTP 时间戳, 用于唇同步 (lip-sync)

源码依据:

- `audio/audio_transport_impl.cc` `NeedMorePlayData`
- `modules/audio_mixer/audio_mixer_impl.cc` `Mix()`

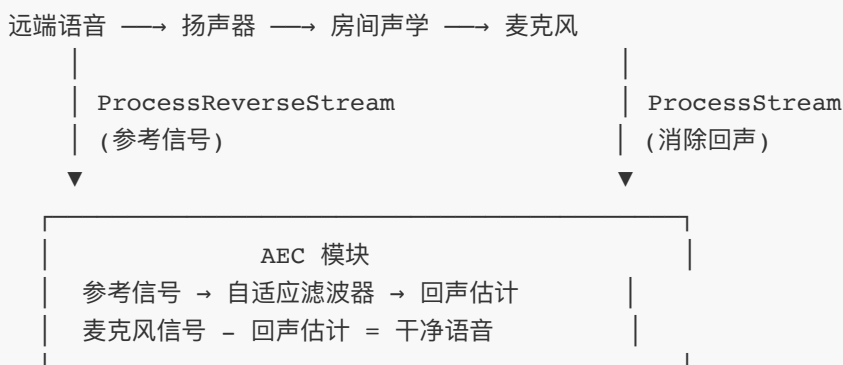
Step 2: APM 反向流处理 (AEC 参考信号)

```
if (audio_processing_) {
    ProcessReverseAudioFrame(audio_processing_, &mixed_frame_);
}
```

为什么需要反向流处理?

回声消除 (AEC) 的工作原理:

1. 扬声器播放远端语音 → 被麦克风重新采集 → 产生回声
2. AEC 需要知道"扬声器正在播放什么"作为参考信号
3. 通过自适应滤波器估计回声路径, 从麦克风信号中减去估计的回声



ProcessReverseAudioFrame 实现

```
int ProcessReverseAudioFrame(AudioProcessing* audio_processing,
                             AudioFrame* audio_frame) {
    StreamConfig config(audio_frame->sample_rate_hz_,
                        audio_frame->num_channels_);
    int result = audio_processing->ProcessReverseStream(
        audio_frame->data(), config, config, audio_frame->mutable_data());
    return result;
}
```

关键点:

- `ProcessReverseStream` 不修改播放数据 (或仅做极小修改)
- 它的主要目的是让 AEC 内部更新参考信号缓冲区
- 必须在 `ProcessStream` (录音路径) 之前或同时提供参考信号

源码依据:

- `audio/audio_transport_impl.cc` NeedMorePlayData, ProcessReverseAudioFrame
- `modules/audio_processing/include/audio_processing.h` ProcessReverseStream

Step 3: Resample 到 ADM 请求的采样率

```
size_t num_samples = mixed_frame_.samples_per_channel_ * mixed_frame_.num_channels_;
Resample(mixed_frame_, samplesPerSec, &render_resampler_,
         static_cast<int16_t*>(audioSamples), &nSamplesOut);
```

Resample 函数

```
void Resample(const AudioFrame& frame,
              int destination_sample_rate,
              PushResampler<int16_t*> resampler,
              int16_t* destination,
              size_t* samples_out) {
    if (frame.sample_rate_hz_ == destination_sample_rate) {
        // 采样率相同, 直接拷贝
        memcpy(destination, frame.data(), frame.size_bytes());
        *samples_out = frame.samples_per_channel_;
    } else {
        // 需要重采样
        resampler->InitializeIfNeeded(frame.sample_rate_hz_,
                                      destination_sample_rate,
                                      frame.num_channels_);
        *samples_out = resampler->Resample(
            frame.data(), frame.samples_per_channel_ * frame.num_channels_,
            destination, max_samples);
    }
}
```

典型场景:

- Mixer 输出 32kHz (内部处理率), ADM 请求 48kHz (硬件原生率)
- Resampler 将 32kHz 上采样到 48kHz

源码依据: `audio/audio_transport_impl.cc` Resample 辅助函数

PullRenderData: Chromium 专用播放路径

```
void AudioTransportImpl::PullRenderData(
    size_t bits_per_sample,
    int sample_rate,
    size_t number_of_channels,
    size_t number_of_frames,
    void* audio_data,
```

```
int64_t* elapsed_time_ms,
int64_t* ntp_time_ms) {
// 混合所有源
mixer_>Mix(number_of_channels, &mixed_frame_);
*elapsed_time_ms = mixed_frame_.elapsed_time_ms;
*ntp_time_ms = mixed_frame_.ntp_time_ms;

// 直接 Resample, 不调用 ProcessReverseAudioFrame
Resample(mixed_frame_, sample_rate, &render_resampler_,
static_cast<int16_t*>(audio_data), ...);
}
```

与 NeedMorePlayData 的区别

特性	NeedMorePlayData	PullRenderData
调用者	标准 ADM	Chromium 音频管线
APM 反向流	✔ 调用 ProcessReverseAudioFrame	✘ 不调用
原因	WebRTC 内置 APM 需要参考信号	Chromium 有独立的 APM 实例
使用场景	独立应用、移动端	Chrome 浏览器

为什么 Chromium 不需要调用 ProcessReverseAudioFrame?

Chromium 有自己的音频处理管线（`MediaStreamAudioProcessor`），其中包含独立的 APM 实例。该实例通过 Chromium 自己的机制获取播放参考信号，不依赖 WebRTC 内部的这个调用。

如果在 Chromium 中也调用 `ProcessReverseAudioFrame`，会导致：

- 同一个参考信号被送入两个不同的 APM 实例
- WebRTC 内部的 APM 实例可能并未被用于录音处理
- 浪费计算资源

源码依据：`audio/audio_transport_impl.cc` `PullRenderData`

UpdateAudioSenders：动态流管理

```
void AudioTransportImpl::UpdateAudioSenders(
    std::vector<AudioSender*> senders,
    int send_sample_rate_hz,
    size_t send_num_channels) {
    MutexLock lock(&capture_lock_);
    audio_senders_ = std::move(senders);
    send_sample_rate_hz_ = send_sample_rate_hz;
    send_num_channels_ = send_num_channels;
}
```

调用时机

由 `AudioState::UpdateAudioTransportWithSendingStreams()` 在以下场景调用：

- 新的音频发送流创建（开始通话）
- 音频发送流销毁（结束通话）
- 发送流参数变化（编码器切换）

参数计算逻辑（AudioState 中）

```
void AudioState::UpdateAudioTransportWithSendingStreams() {
    std::vector<AudioSender*> senders;
    int max_sample_rate = 8000;
    size_t max_num_channels = 1;

    for (auto* stream : sending_streams_) {
        senders.push_back(stream);
        auto config = stream->GetEncoderConfig();
        max_sample_rate = std::max(max_sample_rate, config.sample_rate_hz);
        max_num_channels = std::max(max_num_channels, config.num_channels);
    }

    audio_transport_.UpdateAudioSenders(
        std::move(senders), max_sample_rate, max_num_channels);
}
```

设计要点：

1. 原子替换：整个 senders 列表通过 `std::move` 一次性替换，不存在中间状态
2. 最高规格：采样率和通道数取所有流中的最大值，确保不丢失信息
3. 锁粒度：只在赋值时持锁，不在遍历发送流时持锁

源码依据：

- `audio/audio_transport_impl.cc` UpdateAudioSenders
- `audio/audio_state.cc` UpdateAudioTransportWithSendingStreams

AsyncAudioProcessing：异步处理路径

创建机制

```
// audio/audio_transport_impl.h
class AudioTransportImpl {
    std::unique_ptr<AsyncAudioProcessing> async_audio_processing_;
};

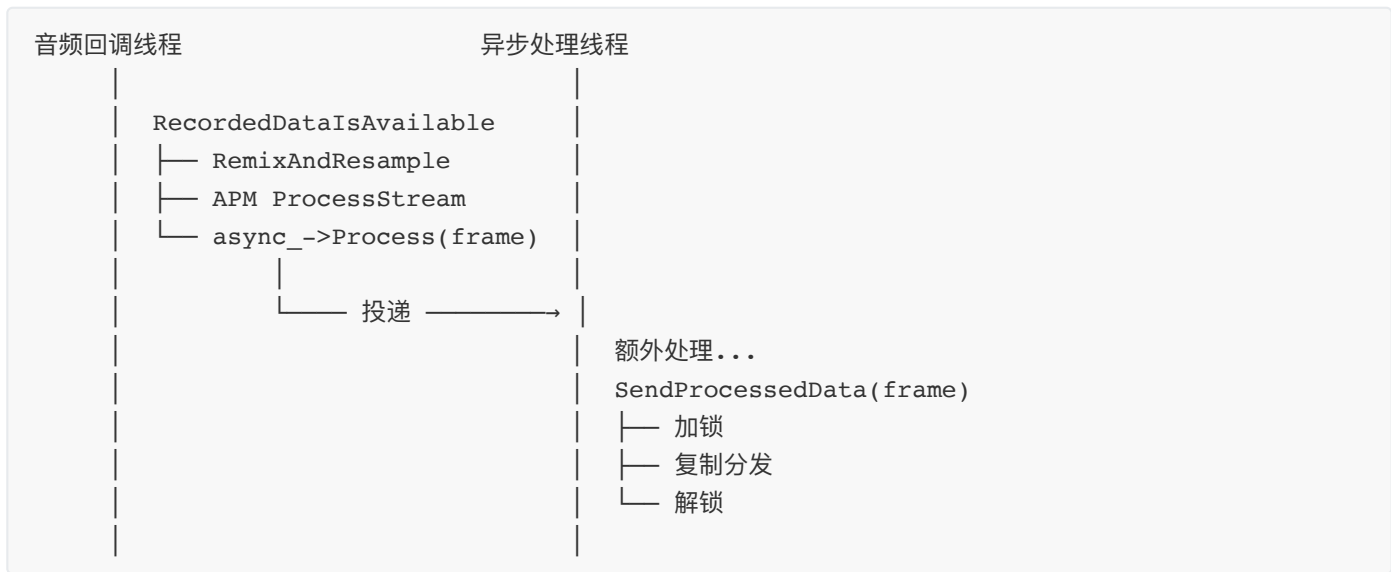
// 构造时创建
```

```

AudioTransportImpl::AudioTransportImpl(
    ...,
    AsyncAudioProcessing::Factory* async_audio_processing_factory)
: ...,
  async_audio_processing_(
      async_audio_processing_factory
      ? async_audio_processing_factory->CreateAsyncAudioProcessing(
          [this](std::unique_ptr<AudioFrame> frame) {
              this->SendProcessedData(std::move(frame));
          })
      : nullptr) {}

```

工作流程



设计考量

1. **解耦音频回调线程**：音频回调线程有严格的实时性要求（通常 < 10ms），任何阻塞都会导致音频卡顿。异步路径允许耗时操作在独立线程完成。
2. **回调安全性**：`SendProcessedData` 内部有 `capture_lock_` 保护，无论从哪个线程调用都是线程安全的。
3. **帧所有权转移**：使用 `std::unique_ptr<AudioFrame>` 确保帧的所有权在线程间清晰转移，避免悬挂指针。
4. **Factory 模式**：通过工厂创建，允许不同平台提供不同的异步实现。

源码依据：

- `audio/audio_transport_impl.h` AsyncAudioProcessing 相关声明
- `audio/audio_transport_impl.cc` 构造函数

AudioMixer 的 Source 注册机制

AudioMixer 是播放路径的核心组件，负责将多个远端接收流混合为单一输出。

接口定义 (api/audio/audio_mixer.h)

```
class AudioMixer : public RefCountInterface {
public:
    // 音频源接口
    class Source {
    public:
        enum class AudioFrameInfo {
            kNormal,    // 正常帧
            kMuted,     // 静音帧 (不参与混合)
            kError      // 错误 (不参与混合)
        };

        // Mixer 调用此方法获取源的 10ms 音频帧
        virtual AudioFrameInfo GetAudioFrameWithInfo(
            int sample_rate_hz,
            AudioFrame* audio_frame) = 0;

        // 返回源的首选采样率 (Mixer 据此决定混合采样率)
        virtual int PreferredSampleRate() const = 0;

        // 源的唯一标识 (用于 SSRC 关联)
        virtual int Ssrc() const = 0;
    };

    // 添加/移除音频源
    virtual bool AddSource(Source* audio_source) = 0;
    virtual void RemoveSource(Source* audio_source) = 0;

    // 执行混合
    virtual void Mix(size_t number_of_channels,
                    AudioFrame* audio_frame_for_mixing) = 0;
};
```

AudioMixerImpl 实现 (modules/audio_mixer/audio_mixer_impl.h)

```
class AudioMixerImpl : public AudioMixer {
public:
    static rtc::scoped_refptr<AudioMixerImpl> Create();
    static rtc::scoped_refptr<AudioMixerImpl> Create(
        int max_sources_to_mix,
        std::unique_ptr<OutputRateCalculator> output_rate_calculator);

    bool AddSource(Source* audio_source) override;
    void RemoveSource(Source* audio_source) override;
    void Mix(size_t number_of_channels, AudioFrame* audio_frame_for_mixing) override;

private:
    struct SourceStatus {
        Source* audio_source;
```



```

    bool is_mixed;           // 当前帧是否参与混合
    float gain;              // 混合增益
};

Mutex lock_;
std::vector<SourceStatus> audio_source_list_ RTC_GUARDED_BY(lock_);
int max_sources_to_mix_;
std::unique_ptr<OutputRateCalculator> output_rate_calculator_;
};

```

AddSource 实现

```

bool AudioMixerImpl::AddSource(Source* audio_source) {
    RTC_DCHECK(audio_source);
    MutexLock lock(&lock_);

    // 检查是否已存在
    for (const auto& source_status : audio_source_list_) {
        if (source_status.audio_source == audio_source) {
            return false; // 重复添加, 返回失败
        }
    }

    audio_source_list_.push_back(
        SourceStatus{audio_source, /*is_mixed=*/false, /*gain=*/1.0f});
    return true;
}

```

RemoveSource 实现

```

void AudioMixerImpl::RemoveSource(Source* audio_source) {
    RTC_DCHECK(audio_source);
    MutexLock lock(&lock_);

    auto it = std::find_if(audio_source_list_.begin(),
                           audio_source_list_.end(),
                           [audio_source](const SourceStatus& s) {
                               return s.audio_source == audio_source;
                           });
    if (it != audio_source_list_.end()) {
        audio_source_list_.erase(it);
    }
}

```

Mix 实现核心流程

```

void AudioMixerImpl::Mix(size_t number_of_channels,
                         AudioFrame* audio_frame_for_mixing) {
    MutexLock lock(&lock_);
}

```

```

// 1. 确定输出采样率
int output_rate = output_rate_calculator_ -> CalculateOutputRate(
    GetPreferredRates());

// 2. 从所有源获取音频帧
std::vector<AudioFrame*> frames_to_mix;
for (auto& source_status : audio_source_list_) {
    auto frame_info = source_status.audio_source -> GetAudioFrameWithInfo(
        output_rate, &source_status.frame);

    if (frame_info == Source::AudioFrameInfo::kNormal) {
        frames_to_mix.push_back(&source_status.frame);
        source_status.is_mixed = true;
    } else {
        source_status.is_mixed = false;
    }
}

// 3. 如果源数超过 max_sources_to_mix_, 选择能量最大的
if (frames_to_mix.size() > max_sources_to_mix_) {
    // 按 RMS 能量排序, 只保留前 N 个
    SelectTopSources(frames_to_mix, max_sources_to_mix_);
}

// 4. 执行混合 (加权求和 + 限幅)
MixFrames(frames_to_mix, number_of_channels, audio_frame_for_mixing);
}

```

Source 注册的调用链

在 WebRTC 中, AudioMixer 的 Source 注册由 `AudioReceiveStreamImpl` 触发:

```

PeerConnection::AddTrack()
→ AudioReceiveStreamImpl::Start()
→ AudioState::AddReceivingStream(stream)
→ mixer_ -> AddSource(stream)

PeerConnection::RemoveTrack()
→ AudioReceiveStreamImpl::Stop()
→ AudioState::RemoveReceivingStream(stream)
→ mixer_ -> RemoveSource(stream)

```

`AudioReceiveStreamImpl` 实现了 `AudioMixer::Source` 接口:

```

class AudioReceiveStreamImpl : public AudioMixer::Source {
public:
    AudioFrameInfo GetAudioFrameWithInfo(int sample_rate_hz,
                                           AudioFrame* audio_frame) override {
        // 从 NetEQ (抖动缓冲区) 获取解码后的音频帧
    }
}

```

```

    return channel_receive_>GetAudioFrameWithInfo(sample_rate_hz, audio_frame);
}

int PreferredSampleRate() const override {
    return channel_receive_>PreferredSampleRate();
}

int Ssrc() const override {
    return remote_ssrc_;
}
};

```

OutputRateCalculator：混合采样率决策

```

class DefaultOutputRateCalculator : public OutputRateCalculator {
public:
    int CalculateOutputRate(const std::vector<int>& preferred_rates) override {
        if (preferred_rates.empty()) return 8000;

        // 取所有源中最高的首选采样率
        int max_rate = *std::max_element(preferred_rates.begin(),
                                          preferred_rates.end());

        // 向上取整到 native rate
        return FindNativeRate(max_rate);
    }
};

```

设计原理：混合采样率取最高值，避免对高质量源做下采样导致信息丢失。

低采样率的源会被上采样到混合采样率（由各 Source 的 GetAudioFrameWithInfo 内部完成）。

源码依据：

- `api/audio/audio_mixer.h` 接口定义
- `modules/audio_mixer/audio_mixer_impl.h` 实现类
- `modules/audio_mixer/audio_mixer_impl.cc` AddSource, RemoveSource, Mix
- `modules/audio_mixer/default_output_rate_calculator.cc`
- `audio/audio_receive_stream.cc` Source 接口实现
- `audio/audio_state.cc` AddReceivingStream, RemoveReceivingStream

线程模型与锁设计

涉及的线程

线程	角色	调用的方法
音频录音回调线程	平台音频驱动	RecordedDataIsAvailable
音频播放回调线程	平台音频驱动	NeedMorePlayData / PullRenderData
信令线程	WebRTC 控制面	UpdateAudioSenders
异步处理线程	可选	SendProcessedData（回调）

锁分析

AudioTransportImpl 只有一把锁： `capture_lock_`

被保护的数据：

- `audio_senders_`：发送流列表
- `send_sample_rate_hz_`：发送采样率
- `send_num_channels_`：发送通道数

锁的使用模式：

```
RecordedDataIsAvailable (音频线程):
    lock → 读取 send_sample_rate_hz_, send_num_channels_ → unlock
    ... APM 处理 (无锁) ...
SendProcessedData:
    lock → 遍历 audio_senders_ 分发 → unlock

UpdateAudioSenders (信令线程):
    lock → 替换 audio_senders_, 更新参数 → unlock

NeedMorePlayData (音频线程):
    无锁 (mixer_ 和 audio_processing_ 指针不变)
```

设计优点：

1. 录音路径中 APM 处理不持锁，避免阻塞信令线程
2. `UpdateAudioSenders` 使用 move 语义，持锁时间极短
3. 播放路径完全无锁，不与录音路径竞争

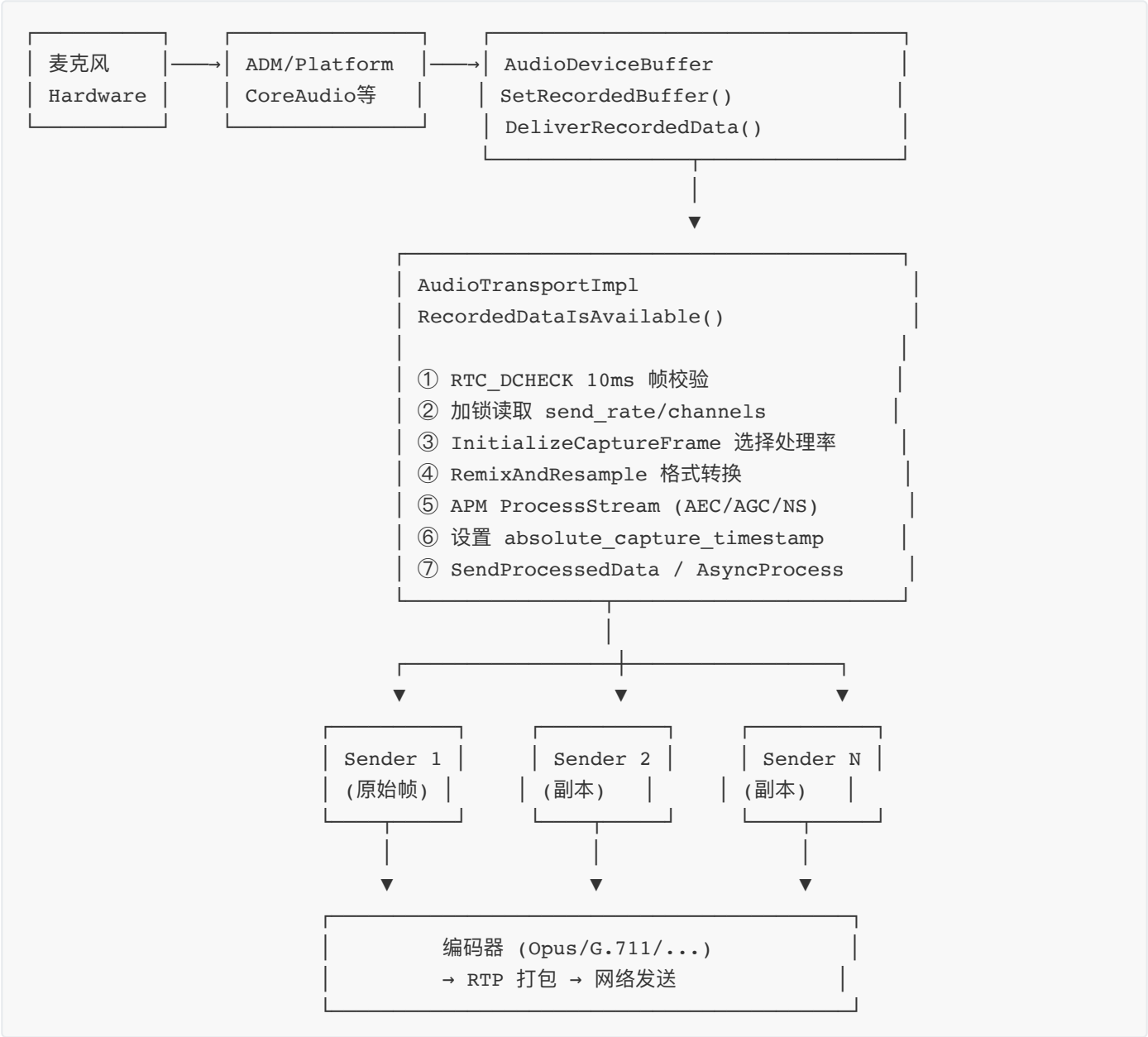
潜在风险：

- `SendProcessedData` 在持锁期间调用 `SendAudioData`，如果下游阻塞会影响 `UpdateAudioSenders`
- 实际上 `SendAudioData` 通常只是将帧投递到编码器队列，不会阻塞

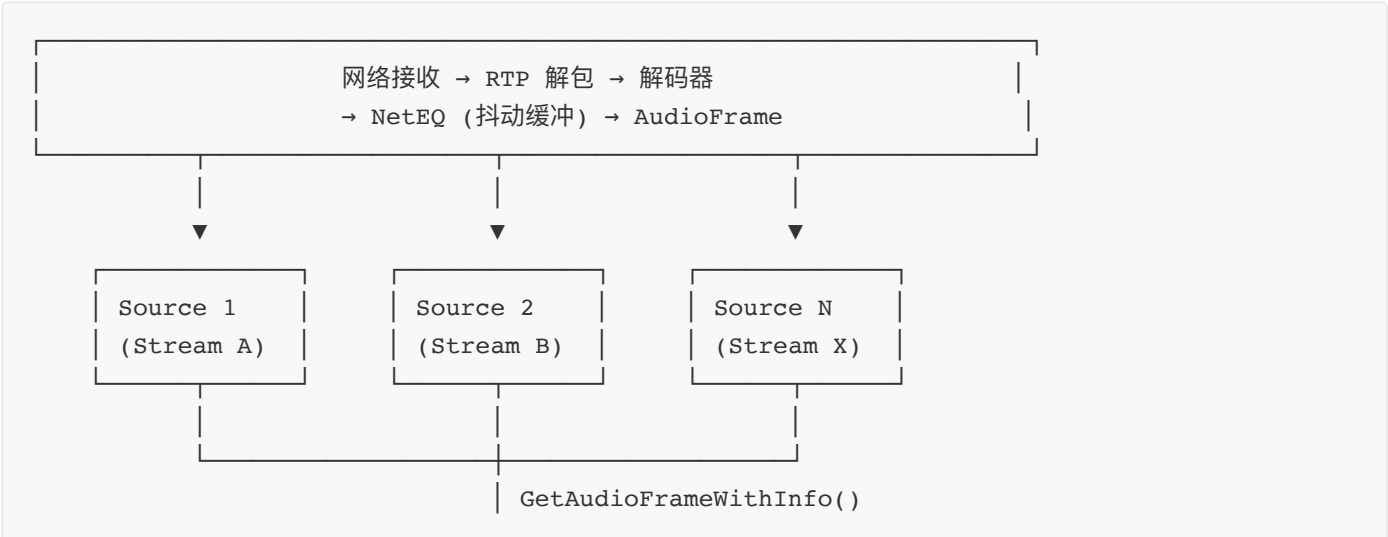
源码依据： `audio/audio_transport_impl.h` 锁注解（RTC_GUARDED_BY）

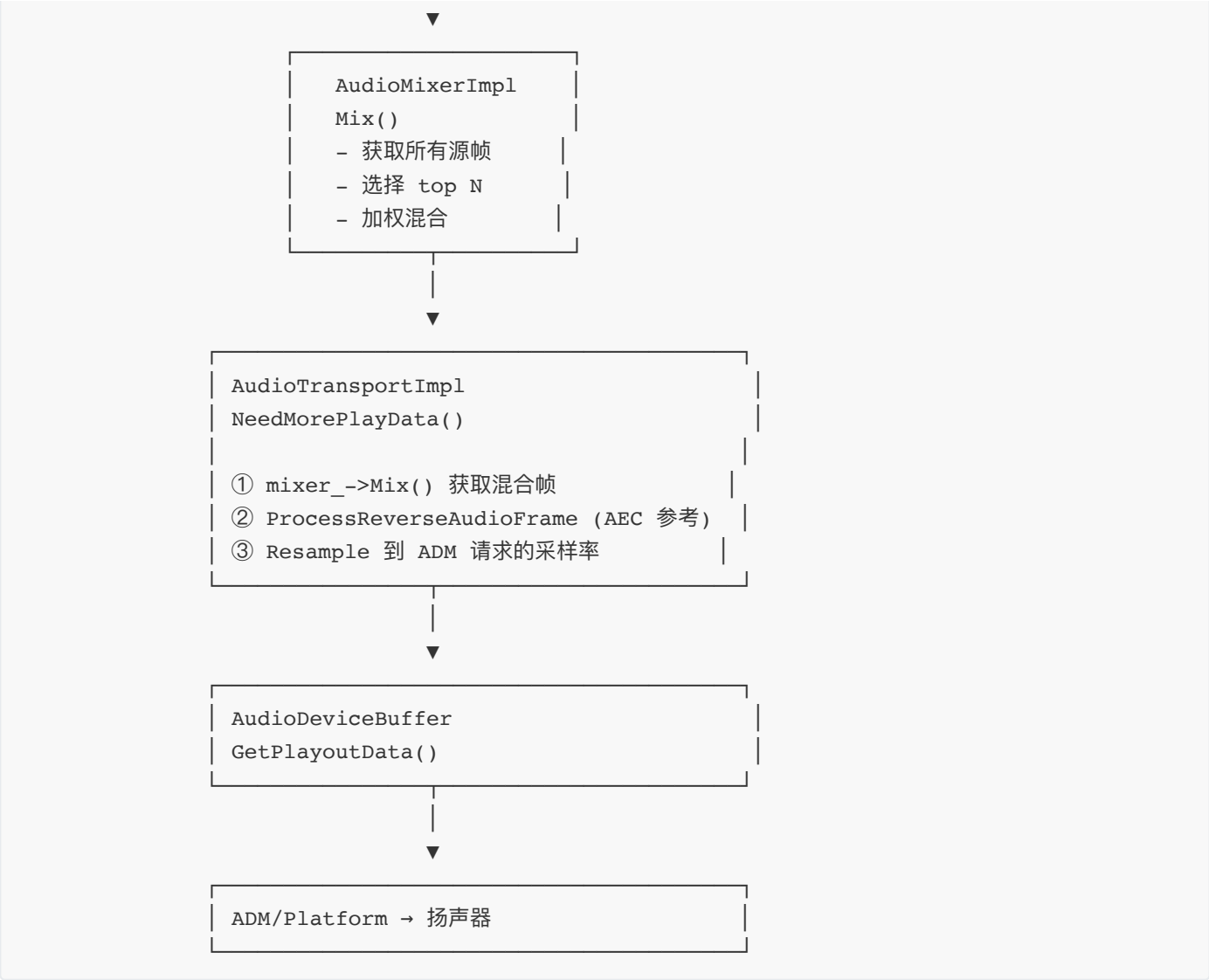
完整数据流总结

录音路径 (Capture Path)



播放路径 (Render Path)





APM 在整体管线中的位置总结

处理阶段	APM 方法	调用位置	目的
录音（正向流）	ProcessStream	RecordedDataIsAvailable Step 5	AEC/AGC/NS 处理采集信号
播放（反向流）	ProcessReverseStream	NeedMorePlayData Step 2	为 AEC 提供远端参考信号

时序关系：

- ProcessReverseStream 和 ProcessStream 可以并发调用（APM 内部有锁）
- AEC 的效果依赖于两者的时间对齐，`stream_delay_ms` 参数用于补偿
- 如果 ProcessReverseStream 未被调用（如 PullRenderData 路径），AEC 将无法工作

关键设计模式总结

1. Push vs Pull 模型

- 录音：Push 模型 — 硬件产生数据后推送给 WebRTC

- 播放：Pull 模型 — 硬件需要数据时从 WebRTC 拉取

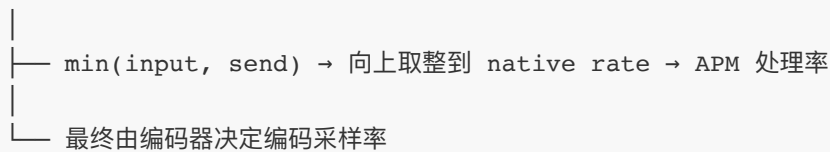
2. 10ms 帧统一

整个管线统一使用 10ms 帧长度：

- ADM 以 10ms 为单位交付/请求数据
- APM 内部处理单位是 10ms
- AudioMixer 混合单位是 10ms
- 编码器输入单位是 10ms（或其整数倍）

3. 采样率协商

输入采样率（硬件）



4. 零拷贝优化

- 单 sender 场景：frame 直接 move，无拷贝
- 多 sender 场景：只有 N-1 次拷贝（第一个 sender 用原始帧）
- APM 就地处理：输入输出同一缓冲区

5. 可选组件

- APM 可为 nullptr（跳过所有音频处理）
- AsyncAudioProcessing 可为 nullptr（使用同步路径）
- 这种设计允许在测试或特殊场景中简化管线

源码文件索引

文件路径	内容
audio/audio_transport_impl.h	AudioTransportImpl 类声明
audio/audio_transport_impl.cc	核心实现：录音/播放/分发
audio/audio_state.h	AudioState 声明（管理 Transport 生命周期）
audio/audio_state.cc	UpdateAudioTransportWithSendingStreams
audio/remix_resample.h	RemixAndResample 声明
audio/remix_resample.cc	通道混合 + 重采样实现
api/audio/audio_mixer.h	AudioMixer 接口定义
modules/audio_mixer/audio_mixer_impl.h	AudioMixerImpl 声明
modules/audio_mixer/audio_mixer_impl.cc	混合算法实现
modules/audio_mixer/default_output_rate_calculator.cc	输出采样率计算
modules/audio_processing/include/audio_processing.h	APM 接口
audio/audio_receive_stream.h	AudioMixer::Source 实现
common_audio/resampler/push_resampler.h	PushResampler 重采样器

第六章：iOS 平台音频实现

概述

iOS 平台的音频设备层是 WebRTC 音频管线中最复杂的平台实现之一。它需要与 Apple 的 Audio Unit 框架和 AVAudioSession 系统深度集成，同时处理 iOS 特有的音频中断、路由切换等事件。

本章基于以下核心源码文件进行分析：

文件路径	职责
sdk/objc/native/src/audio/audio_device_ios.h	AudioDeviceIOS 类声明
sdk/objc/native/src/audio/audio_device_ios.mm	AudioDeviceIOS 实现
sdk/objc/native/src/audio/voice_processing_audio_unit.h	VoiceProcessingAudioUnit 声明
sdk/objc/native/src/audio/voice_processing_audio_unit.mm	VoiceProcessingAudioUnit 实现
modules/audio_device/fine_audio_buffer.h	FineAudioBuffer 适配层
modules/audio_device/fine_audio_buffer.cc	FineAudioBuffer 实现

1. AudioDeviceIOS 类架构

1.1 继承关系

```
// 源码文件：sdk/objc/native/src/audio/audio_device_ios.h

class AudioDeviceIOS : public AudioDeviceGeneric,
                       public AudioSessionObserver,
                       public VoiceProcessingAudioUnitObserver {
```

这个三重继承体现了 AudioDeviceIOS 的三重职责：

基类	职责	说明
AudioDeviceGeneric	平台无关的 ADM 接口	提供 Init/StartPlayout/StartRecording 等标准接口
AudioSessionObserver	监听 AVAudioSession 事件	处理中断、路由变化等 iOS 系统事件
VoiceProcessingAudioUnitObserver	接收 Audio Unit 回调	在实时 I/O 线程上处理播放和录音数据

设计决策分析：

WebRTC 使用 Observer 模式将 Audio Unit 的回调与设备管理解耦。VoiceProcessingAudioUnit 类负责底层 Audio Unit 的生命周期管理，而 AudioDeviceIOS 通过 Observer 接口接收数据回调。这种分离使得：

- Audio Unit 的创建/销毁逻辑独立于数据处理逻辑
- 便于单元测试（可以 mock Observer）
- 中断恢复时可以重建 Audio Unit 而不影响上层状态

1.2 关键成员变量

```
// 源码文件：sdk/objc/native/src/audio/audio_device_ios.h

const bool bypass_voice_processing;           // 是否绕过VP（禁用硬件AEC）
AudioDeviceBuffer* audio_device_buffer_;      // 指向 AudioDeviceModuleImpl 的 buffer
AudioParameters playout_parameters_;          // 播放参数（采样率/通道/buffer大小）
AudioParameters record_parameters_;           // 录音参数
unique_ptr<VoiceProcessingAudioUnit> audio_unit_; // VP I/O Audio Unit
unique_ptr<FineAudioBuffer> fine_audio_buffer_; // 10ms对齐适配层
BufferT<int16_t> record_audio_buffer_;         // 临时录音缓冲
atomic<int> recording_;                         // 原子变量，I/O线程和主线程都会读
atomic<int> playing_;                           // 原子变量，I/O线程和主线程都会读
```

逐字段分析：

bypass_voice_processing_

```
const bool bypass_voice_processing;
```

这是一个 const 成员，在构造时确定，运行期间不可变。它决定了整个音频管线的 AEC 策略：

- false（默认）：使用 kAudioUnitSubType_VoiceProcessingIO，硬件 AEC 生效，软件 AEC 禁用
- true：使用 kAudioUnitSubType_RemoteIO，无硬件 AEC，软件 AEC 生效

audio_device_buffer_

```
AudioDeviceBuffer* audio_device_buffer_;
```

裸指针，不拥有所有权。指向 AudioDeviceModuleImpl 内部的 AudioDeviceBuffer。这是 iOS 平台层与平台无关层之间的数据桥梁。所有 PCM 数据最终通过此 buffer 传递给 AudioTransport。

playout_parameters_ / record_parameters_

```
AudioParameters playout_parameters_;
AudioParameters record_parameters_;
```

封装了采样率、通道数、每帧样本数等参数。这些参数在 `InitPlayout()` / `InitRecording()` 时从 `AVAudioSession` 查询并设置。iOS 上典型值：

- 采样率：48000 Hz (iPhone 6s+) 或 16000 Hz (旧设备)
- 通道数：1 (单声道)
- Buffer 大小：由 `preferredIOBufferDuration` 决定

`audio_unit_`

```
unique_ptr<VoiceProcessingAudioUnit> audio_unit_;
```

封装了 Core Audio 的 Audio Unit 实例。使用 `unique_ptr` 管理生命周期，在中断恢复时可能被销毁重建。

`fine_audio_buffer_`

```
unique_ptr<FineAudioBuffer> fine_audio_buffer_;
```

关键适配层，解决 iOS native buffer size 与 WebRTC 10ms 帧大小不匹配的问题。详见第 4 节。

`recording_ / playing_`

```
atomic<int> recording_;
atomic<int> playing_;
```

使用 `atomic<int>` 而非 `atomic<bool>` 或普通 `bool`，原因：

- I/O 实时线程和主线程都会读取这些变量
- I/O 实时线程不能使用锁（会导致优先级反转）
- `atomic` 保证无锁的线程安全读写

2. Voice Processing I/O Audio Unit 架构

2.1 Audio Unit 类型选择

```
// 源码文件：sdk/objc/native/src/audio/voice_processing_audio_unit.mm

// 默认使用 VoiceProcessingIO
AudioComponentDescription desc = {
    .componentType = kAudioUnitType_Output,
    .componentSubType = kAudioUnitSubType_VoiceProcessingIO,
    .componentManufacturer = kAudioUnitManufacturer_Apple,
};
```

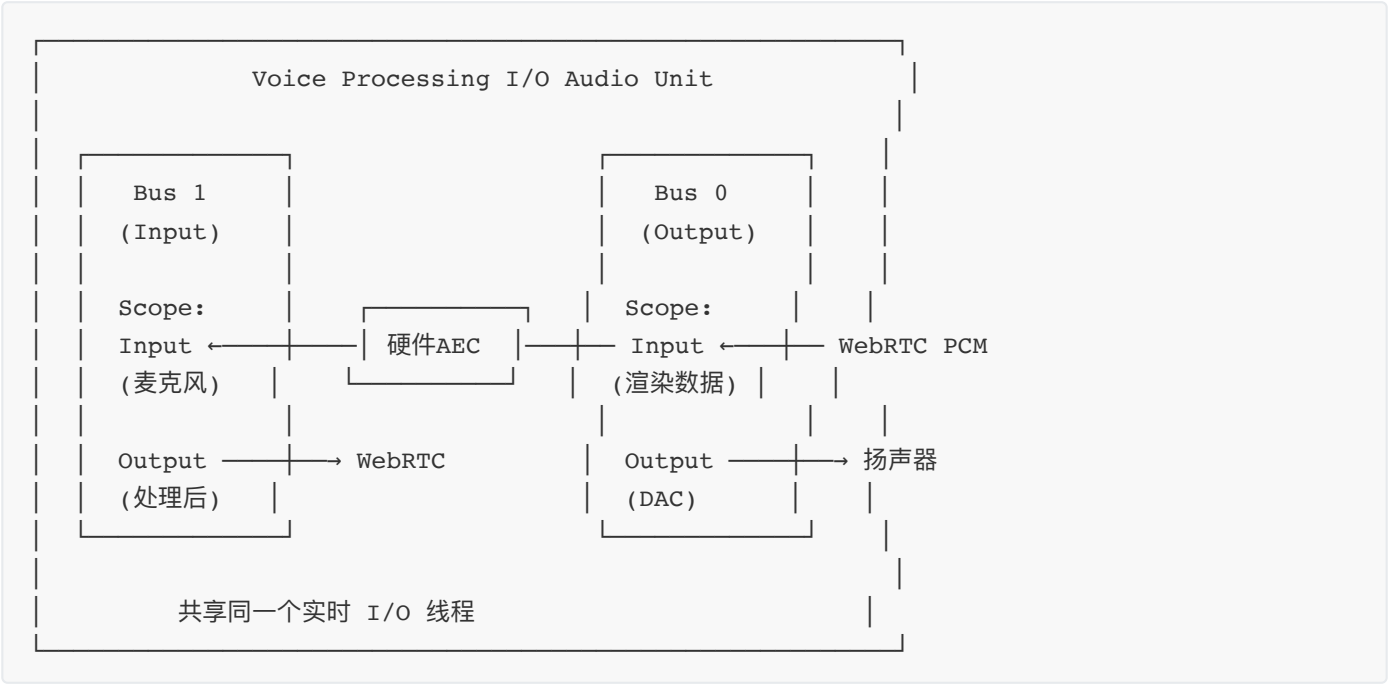
Apple 提供两种 I/O Audio Unit:

类型	SubType	特性
Voice Processing I/O	kAudioUnitSubType_VoiceProcessingIO	内置 AEC、AGC、NS
Remote I/O	kAudioUnitSubType_RemoteIO	无信号处理，原始音频

WebRTC 默认选择 Voice Processing I/O，因为它提供了硬件级别的回声消除，延迟更低、CPU 占用更少。

2.2 Bus 架构详解

VP I/O Audio Unit 的 Bus 架构是理解 iOS 音频的关键：



Bus 0 - Output Bus（播放方向）：

- Scope Input: WebRTC 提供的 PCM 数据（通过 render callback 拉取）
- Scope Output: 连接到扬声器/耳机的 DAC
- 回调时机：硬件需要新数据时触发 `OnGetPlayoutData`

Bus 1 - Input Bus（录音方向）：

- Scope Input: 连接到麦克风的 ADC
- Scope Output: 经过 AEC 处理后的 PCM 数据
- 回调时机：硬件采集到新数据时触发 `OnDeliverRecordedData`

关键设计点：两个 Bus 共享同一个实时 I/O 线程。这意味着：

1. 播放和录音回调在同一线程上交替执行
2. AEC 可以精确对齐参考信号（Bus 0 输出）和采集信号（Bus 1 输入）
3. 回调函数必须是实时安全的（不能分配内存、不能加锁、不能做 I/O）

2.3 回调注册

```
// 源码文件: sdk/objc/native/src/audio/voice_processing_audio_unit.mm

bool VoiceProcessingAudioUnit::Init() {
    // 步骤1: 创建 Audio Unit 实例
    AudioComponent comp = AudioComponentFindNext(nullptr, &desc);
    OSStatus result = AudioComponentInstanceNew(comp, &vpio_unit_);

    // 步骤2: 启用 Input (Bus 1 默认禁用)
    UInt32 enable = 1;
    AudioUnitSetProperty(vpio_unit_,
        kAudioOutputUnitProperty_EnableIO,
        kAudioUnitScope_Input, kInputBus,
        &enable, sizeof(enable));

    // 步骤3: 注册播放回调 (Bus 0 output scope 的 input)
    AURenderCallbackStruct render_callback;
    render_callback.inputProc = OnGetPlayoutData;
    render_callback.inputProcRefCon = this;
    AudioUnitSetProperty(vpio_unit_,
        kAudioUnitProperty_SetRenderCallback,
        kAudioUnitScope_Input, kOutputBus,
        &render_callback, sizeof(render_callback));

    // 步骤4: 注册录音回调 (Bus 1 input 通知)
    AURenderCallbackStruct input_callback;
    input_callback.inputProc = OnDeliverRecordedData;
    input_callback.inputProcRefCon = this;
    AudioUnitSetProperty(vpio_unit_,
        kAudioOutputUnitProperty_SetInputCallback,
        kAudioUnitScope_Global, kInputBus,
        &input_callback, sizeof(input_callback));

    // 步骤5: 设置音频格式 (16-bit PCM, interleaved)
    AudioStreamBasicDescription format = GetFormat(sample_rate_);
    AudioUnitSetProperty(vpio_unit_,
        kAudioUnitProperty_StreamFormat,
        kAudioUnitScope_Output, kInputBus,
        &format, sizeof(format));
    AudioUnitSetProperty(vpio_unit_,
        kAudioUnitProperty_StreamFormat,
        kAudioUnitScope_Input, kOutputBus,
        &format, sizeof(format));
}
```

逐步解析:

步骤 2 的必要性: iOS 的 I/O Audio Unit 默认只启用 Output (Bus 0)。如果需要录音, 必须显式启用 Input (Bus 1)。这是 Apple 的安全设计——防止应用在未经用户授权时意外录音。

步骤 3 的回调语义： `kAudioUnitProperty_SetRenderCallback` 设置在 `kAudioUnitScope_Input` + `kOutputBus` 上。这看起来反直觉，但含义是："当 Bus 0 的 Output 需要数据时，从这个回调的 Input 拉取"。这是 Core Audio 的 pull model。

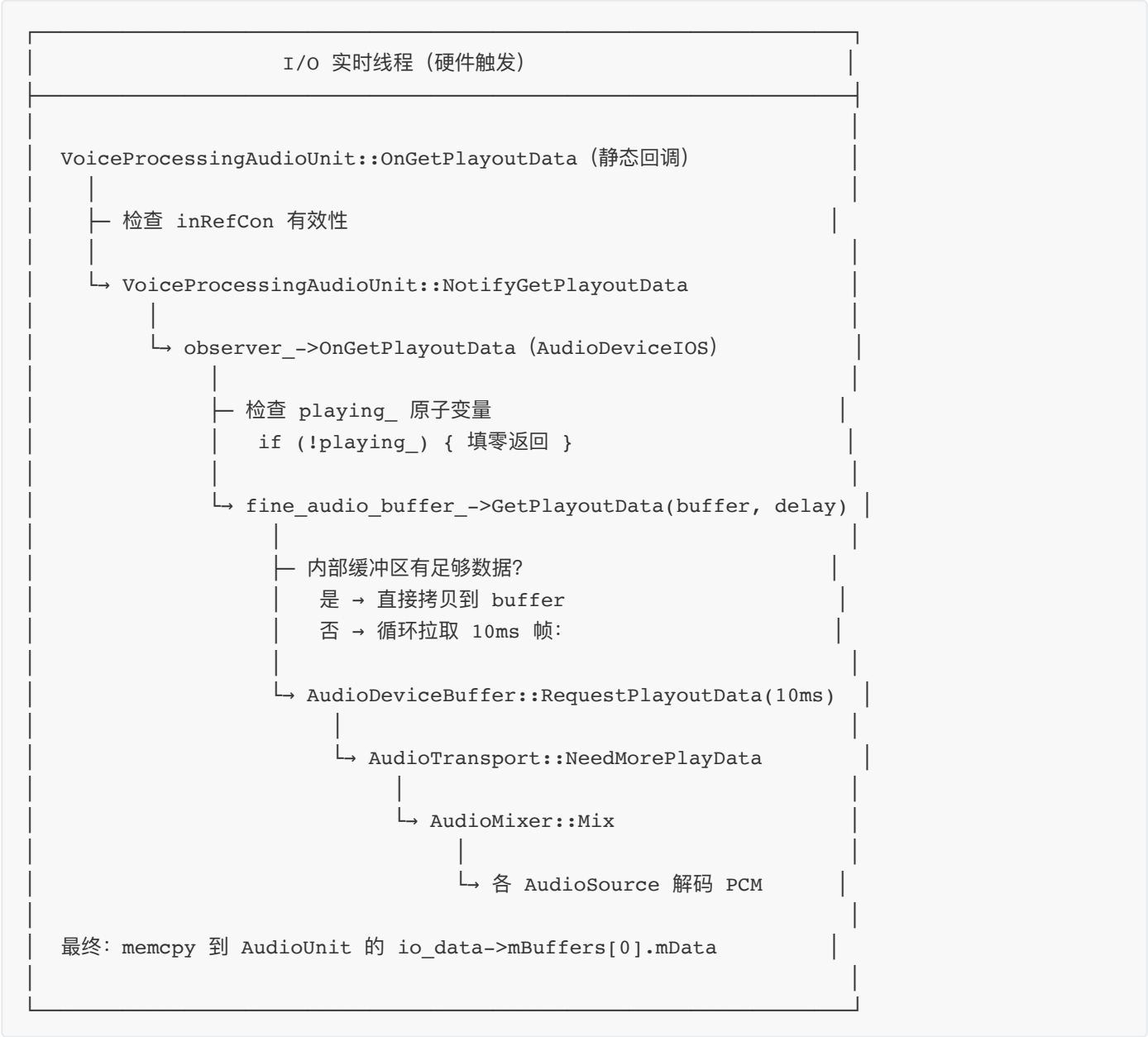
步骤 4 的回调语义： `kAudioOutputUnitProperty_SetInputCallback` 是一个通知回调，当 Bus 1 有新的录音数据可用时触发。注意这里用的是 `kAudioUnitScope_Global`，因为这是一个全局通知而非特定 scope 的属性。

步骤 5 的格式设置： 设置 Bus 1 Output scope 和 Bus 0 Input scope 的格式，确保 WebRTC 侧看到的数据格式一致（16-bit signed integer PCM）。

3. 完整回调链分析

3.1 播放回调链（Playout）

播放方向的数据流是一个典型的 **pull model**：硬件需要数据时，逐层向上拉取。



静态回调函数：

```
// 源码文件: sdk/objc/native/src/audio/voice_processing_audio_unit.mm
```

```
OSStatus VoiceProcessingAudioUnit::OnGetPlayoutData(  
    void* in_ref_con,  
    AudioUnitRenderActionFlags* flags,  
    const AudioTimeStamp* time_stamp,  
    UInt32 bus_number,  
    UInt32 num_frames,  
    AudioBufferList* io_data) {  
    VoiceProcessingAudioUnit* audio_unit =  
        static_cast<VoiceProcessingAudioUnit*>(in_ref_con);  
    return audio_unit->NotifyGetPlayoutData(flags, time_stamp, bus_number,  
                                             num_frames, io_data);  
}
```

这是一个 C 风格的静态函数（Core Audio 要求）。通过 `in_ref_con`（即注册时的 `inputProcRefCon`）恢复 `this` 指针，然后转发到成员函数。

AudioDeviceIOS 的播放处理：

```
// 源码文件: sdk/objc/native/src/audio/audio_device_ios.mm
```

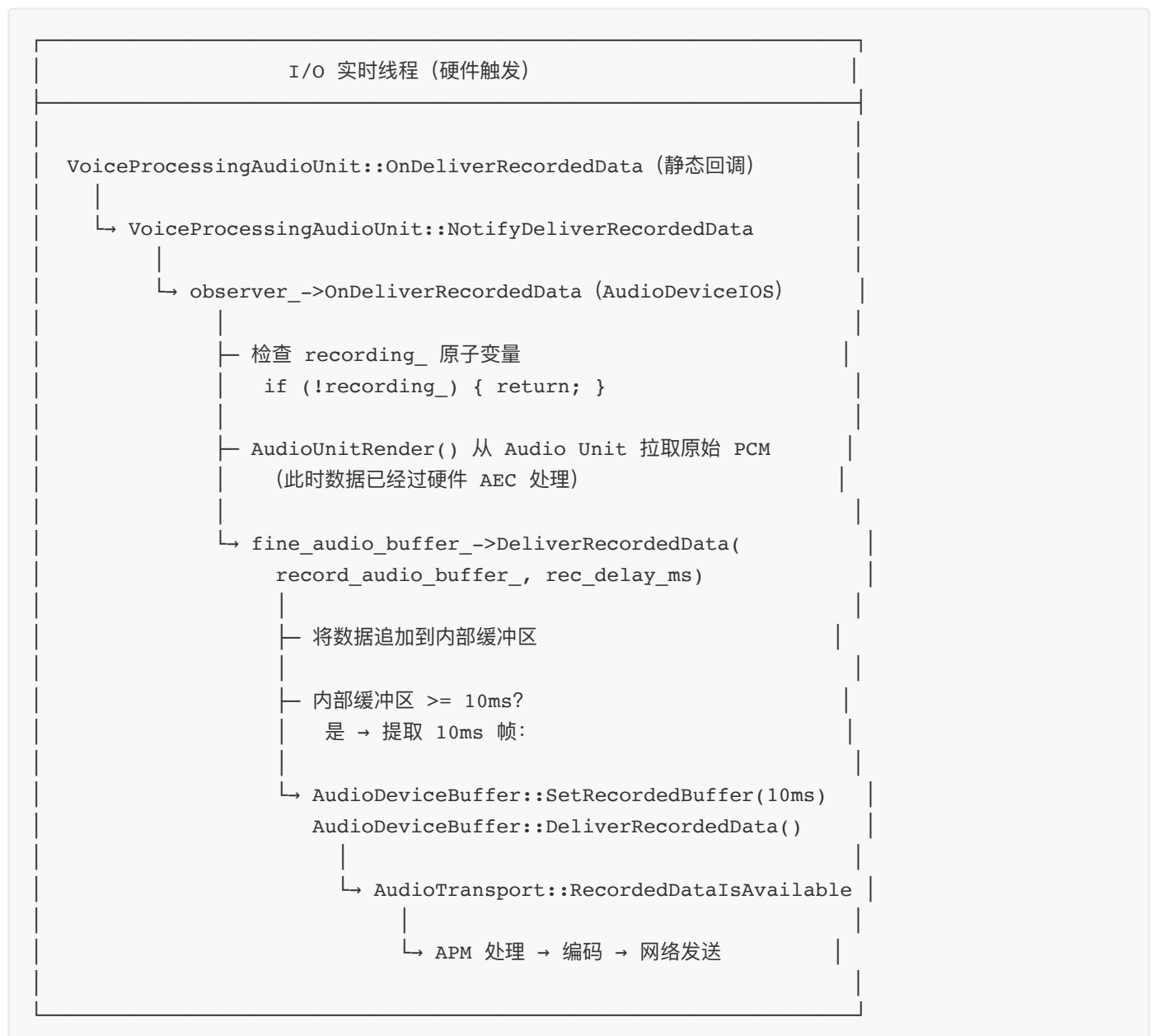
```
OSStatus AudioDeviceIOS::OnGetPlayoutData(  
    AudioUnitRenderActionFlags* flags,  
    const AudioTimeStamp* time_stamp,  
    UInt32 bus_number,  
    UInt32 num_frames,  
    AudioBufferList* io_data) {  
    // 关键：在实时线程上，不能加锁  
    // 使用原子变量检查状态  
    if (!playing_) {  
        // 未在播放状态，填充静音  
        *flags |= kAudioUnitRenderAction_OutputIsSilence;  
        memset(io_data->mBuffers[0].mData, 0, io_data->mBuffers[0].mDataByteSize);  
        return noErr;  
    }  
  
    // 计算播放延迟（用于 AEC 对齐）  
    SInt64 playout_delay_ms = GetPlayoutDelayMs(time_stamp);  
  
    // 通过 FineAudioBuffer 获取数据  
    fine_audio_buffer->GetPlayoutData(  
        rtc::ArrayView<int16_t>(  
            static_cast<int16_t*>(io_data->mBuffers[0].mData), num_frames),  
        playout_delay_ms);  
  
    return noErr;  
}
```

设计决策解析：

1. **静音填充**：当 `playing_` 为 `false` 时，必须填充零数据。Audio Unit 不允许回调返回而不写入 `io_data`，否则会播放上一帧的残留数据（产生噪音）。
2. **延迟计算**：`playout_delay_ms` 用于 AEC 算法对齐参考信号。它表示从数据写入 buffer 到实际从扬声器播出的时间差。
3. **无锁设计**：整个回调路径没有任何 mutex。这是实时音频编程的铁律——I/O 线程的优先级最高，如果它等待锁，而持锁线程被调度器降级，就会发生优先级反转，导致音频卡顿。

3.2 录音回调链（Recording）

录音方向是 **push model** 与 **pull model** 的混合：系统通知有新数据（push），然后 WebRTC 主动拉取（pull）。



AudioDeviceIOS 的录音处理：

// 源码文件：`sdk/objc/native/src/audio/audio_device_ios.mm`

```

OSStatus AudioDeviceIOS::OnDeliverRecordedData(
    AudioUnitRenderActionFlags* flags,
    const AudioTimeStamp* time_stamp,
    UInt32 bus_number,
    UInt32 num_frames,
    AudioBufferList* io_data) {
    // 状态检查
    if (!recording_) {
        return noErr;
    }

    // 准备 AudioBufferList 用于接收数据
    AudioBufferList buffer_list;
    buffer_list.mNumberBuffers = 1;
    buffer_list.mBuffers[0].mNumberChannels = record_parameters_.channels();
    buffer_list.mBuffers[0].mDataByteSize =
        num_frames * sizeof(int16_t) * record_parameters_.channels();
    buffer_list.mBuffers[0].mData =
        static_cast<void*>(record_audio_buffer_.data());

    // 关键：调用 AudioUnitRender 从 Audio Unit 拉取录音数据
    // 注意：OnDeliverRecordedData 只是通知，数据需要主动拉取
    OSStatus result = AudioUnitRender(
        audio_unit_ ->GetAudioUnit(),
        flags, time_stamp, bus_number, num_frames, &buffer_list);

    if (result != noErr) {
        return result;
    }

    // 计算录音延迟
    SInt64 rec_delay_ms = GetRecordDelayMs(time_stamp);

    // 通过 FineAudioBuffer 传递数据
    fine_audio_buffer_ ->DeliverRecordedData(
        rtc::ArrayView<const int16_t>(record_audio_buffer_.data(), num_frames),
        rec_delay_ms);

    return noErr;
}

```

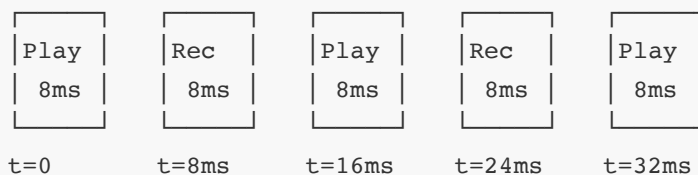
关键设计点：

1. **两阶段获取**：录音回调是一个通知（“有数据了”），但数据本身需要通过 `AudioUnitRender()` 主动拉取。这是 Core Audio 的设计——允许应用决定是否需要这帧数据。
2. **record_audio_buffer_**：预分配的临时缓冲区，避免在实时线程上分配内存。大小在 `InitRecording()` 时根据 `num_frames` 预先分配。
3. **AEC 已完成**：当 `AudioUnitRender()` 返回时，数据已经过 VP I/O 的硬件 AEC 处理。Bus 0 的播放数据作为参考信号，已从 Bus 1 的录音数据中消除。

3.3 回调时序关系

时间轴 →

I/O 线程：



FineAudioBuffer 内部：

播放：拉取10ms → 切8ms给HW → 剩2ms → 下次补8ms再切...

录音：收8ms → 不够10ms → 再收8ms → 凑够10ms → 上报

两个回调在同一线程上交替执行，保证了：

- 播放和录音的时间戳精确对齐
- AEC 参考信号和采集信号的同步性
- 不存在多线程竞争问题

4. FineAudioBuffer：帧大小适配层

4.1 问题背景

WebRTC 的音频处理管线（APM、编解码器）统一使用 **10ms 帧**作为处理单位：

- 16kHz 采样率 → 160 samples/frame
- 48kHz 采样率 → 480 samples/frame

但 iOS 的 Audio Unit 回调帧大小由 `AVAudioSession.preferredIOBufferDuration` 决定：

```
// 典型设置
[[AVAudioSession sharedInstance] setPreferredIOBufferDuration:0.005 // 5ms
                                error:nil];

// 实际值由系统决定，可能是：
// 128 frames @ 16kHz = 8ms
// 256 frames @ 48kHz = 5.33ms
// 512 frames @ 48kHz = 10.67ms
```

核心矛盾：iOS 硬件的 buffer 大小几乎不可能恰好等于 10ms。

4.2 FineAudioBuffer 的解决方案

```
// 源码文件：modules/audio_device/fine_audio_buffer.h

class FineAudioBuffer {
public:
```

```

FineAudioBuffer(AudioDeviceBuffer* audio_device_buffer);

// 播放方向: AudioUnit 调用, 请求 num_frames 个样本
void GetPlayoutData(rtc::ArrayView<int16_t> audio_buffer,
                   int playout_delay_ms);

// 录音方向: AudioUnit 调用, 提供 num_frames 个样本
void DeliverRecordedData(rtc::ArrayView<const int16_t> audio_buffer,
                        int record_delay_ms);

private:
AudioDeviceBuffer* const audio_device_buffer_; // 10ms 帧接口
int playout_samples_per_10ms_;               // 10ms 对应的样本数
int record_samples_per_10ms_;                 // 10ms 对应的样本数
rtc::BufferT<int16_t> playout_buffer_;        // 播放方向缓冲
rtc::BufferT<int16_t> record_buffer_;        // 录音方向缓冲
};

```

4.3 播放方向工作原理

```

// 源码文件: modules/audio_device/fine_audio_buffer.cc

void FineAudioBuffer::GetPlayoutData(rtc::ArrayView<int16_t> audio_buffer,
                                     int playout_delay_ms) {
    // audio_buffer.size() = iOS 硬件请求的帧大小 (如 128 samples)
    // playout_samples_per_10ms_ = WebRTC 的 10ms 帧大小 (如 160 samples)

    const int num_samples_requested = audio_buffer.size();

    // 如果内部缓冲区数据不够, 从 AudioDeviceBuffer 拉取 10ms 帧
    while (playout_buffer_.size() < num_samples_requested) {
        // 每次拉取恰好 10ms 的数据
        playout_buffer_.AppendData(playout_samples_per_10ms_,
                                   [&](rtc::ArrayView<int16_t> buf) {
                                       // 调用 AudioDeviceBuffer 获取 10ms PCM
                                       audio_device_buffer_->RequestPlayoutData(
                                           playout_samples_per_10ms_);
                                       return audio_device_buffer_->GetPlayoutData(buf.data());
                                   });
    }

    // 从内部缓冲区取出硬件请求的样本数
    memcpy(audio_buffer.data(), playout_buffer_.data(),
           num_samples_requested * sizeof(int16_t));

    // 移除已消费的数据, 保留剩余
    playout_buffer_.SetData(
        playout_buffer_.data() + num_samples_requested,
        playout_buffer_.size() - num_samples_requested);
}

```

具体示例（16kHz, iOS buffer = 128 samples = 8ms）：

回调 #1: 请求 128 samples
内部缓冲: 0 samples (不够)
→ 拉取 10ms = 160 samples → 内部缓冲: 160
取出 128 → 内部缓冲剩余: 32

回调 #2: 请求 128 samples
内部缓冲: 32 samples (不够)
→ 拉取 10ms = 160 samples → 内部缓冲: 192
取出 128 → 内部缓冲剩余: 64

回调 #3: 请求 128 samples
内部缓冲: 64 samples (不够)
→ 拉取 10ms = 160 samples → 内部缓冲: 224
取出 128 → 内部缓冲剩余: 96

回调 #4: 请求 128 samples
内部缓冲: 96 samples (不够)
→ 拉取 10ms = 160 samples → 内部缓冲: 256
取出 128 → 内部缓冲剩余: 128

回调 #5: 请求 128 samples
内部缓冲: 128 samples (刚好够!)
不需要拉取
取出 128 → 内部缓冲剩余: 0

(周期重复: 每 5 次回调 = 40ms, 拉取 4 次 10ms = 40ms)

4.4 录音方向工作原理

```
// 源码文件: modules/audio_device/fine_audio_buffer.cc

void FineAudioBuffer::DeliverRecordedData(
    rtc::ArrayView<const int16_t> audio_buffer,
    int record_delay_ms) {
    // audio_buffer.size() = iOS 硬件提供的帧大小 (如 128 samples)
    // record_samples_per_10ms_ = WebRTC 的 10ms 帧大小 (如 160 samples)

    // 将硬件数据追加到内部缓冲区
    record_buffer_.AppendData(audio_buffer.data(), audio_buffer.size());

    // 每凑够 10ms 就上报一次
    while (record_buffer_.size() >= record_samples_per_10ms_) {
        audio_device_buffer_ -> SetRecordedBuffer(
            record_buffer_.data(), record_samples_per_10ms_);
        audio_device_buffer_ -> SetVQEDData(0, record_delay_ms);
        audio_device_buffer_ -> DeliverRecordedData();

        // 移除已上报的数据
        record_buffer_.SetData(
```

```
        record_buffer_.data() + record_samples_per_10ms_,
        record_buffer_.size() - record_samples_per_10ms_);
    }
}
```

具体示例（16kHz, iOS buffer = 128 samples = 8ms）：

回调 #1: 收到 128 samples
内部缓冲: 128 samples (不够 160)
不上报

回调 #2: 收到 128 samples
内部缓冲: 256 samples (≥ 160)
上报 160 $\rightarrow$ 内部缓冲剩余: 96

回调 #3: 收到 128 samples
内部缓冲: 224 samples (≥ 160)
上报 160 $\rightarrow$ 内部缓冲剩余: 64

回调 #4: 收到 128 samples
内部缓冲: 192 samples (≥ 160)
上报 160 $\rightarrow$ 内部缓冲剩余: 32

回调 #5: 收到 128 samples
内部缓冲: 160 samples ($= 160$)
上报 160 $\rightarrow$ 内部缓冲剩余: 0

(周期重复: 每 5 次回调收到 640 samples, 上报 4 次 $\times 160 = 640$ samples)

4.5 为什么不直接修改 WebRTC 的帧大小?

1. **编解码器依赖**: Opus 编码器的帧大小是 10ms/20ms/40ms/60ms, 10ms 是最小单位
2. **APM 依赖**: AEC、NS、AGC 等算法都基于 10ms 帧设计, FFT 窗口大小固定
3. **跨平台一致性**: 所有平台的上层代码统一使用 10ms, 只在平台层做适配
4. **历史原因**: 10ms 是 VoIP 领域的事实标准 (G.711 等编码器的帧大小)

FineAudioBuffer 是一个优雅的适配器模式实现, 将平台差异封装在最底层, 上层完全无感知。

5. AVAudioSession 中断处理

5.1 iOS 音频中断机制

iOS 是一个多任务系统, 但音频硬件是独占资源。当更高优先级的音频事件发生时 (如来电、Siri 激活、闹钟), 系统会中断当前应用的音频会话:

iOS 系统

来电 / Siri / 闹钟

↳ AVAudioSessionInterruptionNotification
type: .began

↳ 系统自动停止你的 Audio Unit
(I/O 回调停止触发)

... 中断期间 ...

↳ AVAudioSessionInterruptionNotification
type: .ended
options: .shouldResume

↳ 应用需要手动重启 Audio Unit

5.2 WebRTC 的中断处理架构

RTCNativeAudioSessionDelegateAdapter (Objective-C)

↳ 监听 AVAudioSessionInterruptionNotification

↳ AudioSessionObserver 接口 (C++)

↳ AudioDeviceIOS::OnInterruptionBegin/End

↳ thread_ -> PostTask(...)

↳ HandleInterruptionBegin/End
(在 WebRTC 工作线程上执行)

5.3 中断开始处理

// 源码文件: sdk/objc/native/src/audio/audio_device_ios.mm

```
void AudioDeviceIOS::OnInterruptionBegin() {  
    RTC_LOG(LS_INFO) << "OnInterruptionBegin";  
    // 不在通知线程上直接处理, 而是 Post 到工作线程  
    // 原因: 通知可能在任意线程上触发, 而 AudioUnit 操作需要线程安全  
    thread_ -> PostTask([this] { HandleInterruptionBegin(); });  
}
```

```
void AudioDeviceIOS::HandleInterruptionBegin() {  
    RTC_DCHECK_RUN_ON(thread_);
```

// 标记中断状态

```

is_interrupted_ = true;

// 停止 Audio Unit
// 虽然系统已经停止了 I/O 回调，但我们需要显式停止 Audio Unit
// 以确保内部状态一致
if (audio_unit_ && audio_unit_->GetState() == kStarted) {
    if (!audio_unit_->Stop()) {
        RTC_LOG(LS_ERROR) << "Failed to stop audio unit during interruption";
    }
}
}
}

```

设计决策分析：

1. **线程切换**： `OnInterruptionBegin()` 通过 `thread_->PostTask()` 将处理转移到工作线程。这是因为：
 - AVAudioSession 通知可能在主线程或任意后台线程触发
 - Audio Unit 的状态管理需要在固定线程上进行
 - `RTC_DCHECK_RUN_ON(thread_)` 确保线程安全
2. **显式停止**：虽然系统中断时 I/O 回调已经停止，但显式调用 `audio_unit_>Stop()` 确保：
 - Audio Unit 的内部状态机正确转换
 - 后续的 `Start()` 调用不会因为状态不一致而失败
3. **`is_interrupted_` 标志**：防止在中断期间尝试重启 Audio Unit（例如路由变化事件可能触发重启逻辑）。

5.4 中断结束处理

```

// 源码文件：sdk/objc/native/src/audio/audio_device_ios.mm

void AudioDeviceIOS::OnInterruptionEnd() {
    RTC_LOG(LS_INFO) << "OnInterruptionEnd";
    thread_->PostTask([this] { HandleInterruptionEnd(); });
}

void AudioDeviceIOS::HandleInterruptionEnd() {
    RTC_DCHECK_RUN_ON(thread_);

    // 清除中断标志
    is_interrupted_ = false;

    // 重新激活 AVAudioSession
    // 中断结束后，session 处于 deactivated 状态，需要重新激活
    if (!ActivateAudioSession()) {
        RTC_LOG(LS_ERROR) << "Failed to activate audio session after interruption";
        return;
    }

    // 重新配置音频参数
    // 中断期间系统可能改变了音频路由（如蓝牙断开）
    // 需要重新查询硬件参数
}

```

```
ConfigureAudioSession();

// 根据当前状态决定是否重启 Audio Unit
// can_play_or_record_ 表示上层是否仍然需要音频
UpdateAudioUnit(can_play_or_record_);
}
```

设计决策分析：

1. **重新激活 Session**：iOS 中断结束后，AVAudioSession 不会自动恢复到 active 状态。应用必须显式调用 `setActive:YES`。
2. **重新配置**：中断期间硬件环境可能已改变（如蓝牙耳机断开、采样率变化）。重新查询参数确保 Audio Unit 使用正确的配置。
3. **条件重启**：`UpdateAudioUnit(can_play_or_record_)` 只在上层仍需要音频时才重启。如果用户在中断期间已经挂断通话，就不需要重启。

5.5 UpdateAudioUnit 状态机

```
// 源码文件：sdk/objc/native/src/audio/audio_device_ios.mm

void AudioDeviceIOS::UpdateAudioUnit(bool can_play_or_record) {
    RTC_DCHECK_RUN_ON(thread_);

    can_play_or_record_ = can_play_or_record;

    // 如果被中断，不做任何操作
    if (is_interrupted_) {
        return;
    }

    if (!can_play_or_record) {
        // 需要停止
        if (audio_unit_ -> GetState() == kStarted) {
            audio_unit_ -> Stop();
        }
    } else {
        // 需要启动
        if (playing_ || recording_) {
            if (audio_unit_ -> GetState() == kInitialized) {
                audio_unit_ -> Start();
            }
        }
    }
}
```

这个状态机确保了：

- 中断期间不会尝试操作 Audio Unit
- 只在上层确实需要音频时才启动

- 状态转换是幂等的（多次调用不会出错）

5.6 路由变化处理

除了中断，iOS 还有音频路由变化事件（插拔耳机、蓝牙连接/断开）：

```
// 源码文件: sdk/objc/native/src/audio/audio_device_ios.mm

void AudioDeviceIOS::OnValidRouteChange() {
    thread_>PostTask([this] { HandleValidRouteChange(); });
}

void AudioDeviceIOS::HandleValidRouteChange() {
    RTC_DCHECK_RUN_ON(thread_);

    // 路由变化可能改变了采样率和 buffer 大小
    // 需要重新初始化 Audio Unit
    if (audio_unit_>GetState() == kStarted) {
        audio_unit_>Stop();
    }

    // 重新查询硬件参数
    SetupAudioBuffersForActiveAudioSession();

    // 重建 FineAudioBuffer (因为帧大小可能变了)
    fine_audio_buffer_.reset(new FineAudioBuffer(audio_device_buffer_));

    // 重启
    if (can_play_or_record_ && !is_interrupted_) {
        audio_unit_>Start();
    }
}
```

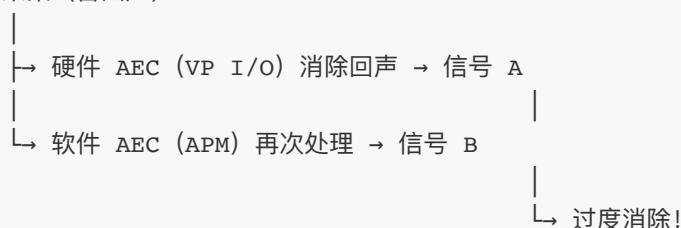
为什么需要重建 **FineAudioBuffer**：路由变化可能导致硬件 buffer 大小改变（如从扬声器切换到蓝牙耳机，buffer 大小可能从 128 变为 256）。FineAudioBuffer 内部的缓冲区大小需要匹配新的硬件参数。

6. 为何禁用 WebRTC 软件 AEC

6.1 双重 AEC 问题

场景：如果同时启用硬件 AEC 和软件 AEC

扬声器播放信号 → 麦克风采集（含回声）



问题：

- 硬件 AEC 已经消除了大部分回声
- 软件 AEC 看到的"回声"其实是残留噪声
- 软件 AEC 会错误地将有用语音当作回声消除
- 结果：语音被过度抑制，对方听到断断续续的声音

6.2 WebRTC 的 AEC 策略选择

```
// 源码文件: modules/audio_device/include/audio_device.h
// 相关文件: sdk/objc/native/src/audio/audio_device_ios.mm

// 在 AudioDeviceIOS 初始化时, 通过 adm_helpers 通知 APM
// 禁用软件 AEC

// adm_helpers.cc 中的逻辑:
void Init(AudioDeviceModule* adm) {
    // 查询平台是否内置 AEC
    if (adm->BuiltInAECIsAvailable()) {
        // 启用内置 AEC
        adm->EnableBuiltInAEC(true);
        // APM 侧禁用软件 AEC
        // → AudioProcessing::Config::echo_canceller.enabled = false
    }
}
```

决策矩阵：

bypass_voice_processing_	Audio Unit 类型	硬件 AEC	软件 AEC	适用场景
false (默认)	VoiceProcessingIO	✅ 启用	❌ 禁用	正常通话
true	RemoteIO	❌ 无	✅ 启用	音乐模式/特殊需求

6.3 bypass_voice_processing_ 模式

```
// 源码文件: sdk/objc/native/src/audio/voice_processing_audio_unit.mm

bool VoiceProcessingAudioUnit::Init() {
    AudioComponentDescription desc;
    desc.componentType = kAudioUnitType_Output;

    if (bypass_voice_processing_) {
        // 使用 Remote I/O: 无任何信号处理
        desc.componentSubType = kAudioUnitSubType_RemoteIO;
    } else {
        // 使用 Voice Processing I/O: 内置 AEC + AGC + NS
        desc.componentSubType = kAudioUnitSubType_VoiceProcessingIO;
    }
}
```

```
desc.componentManufacturer = kAudioUnitManufacturer_Apple;
// ...
}
```

使用 Remote I/O 的场景：

- 1. 音乐应用：需要高保真录音，不希望 AEC 处理改变音质
- 2. 外部 AEC：应用自己实现了 AEC 算法
- 3. 调试：需要原始音频数据进行分析
- 4. 单向音频：只播放或只录音，不需要 AEC

6.4 硬件 AEC 的优势

为什么 iOS 上优先使用硬件 AEC 而非 WebRTC 的软件 AEC？

维度	硬件 AEC (VP I/O)	软件 AEC (APM)
延迟	极低（硬件级同步）	需要估算延迟
CPU	由 DSP 处理，几乎零开销	消耗应用 CPU
参考信号对齐	Bus 0/1 硬件同步	依赖时间戳估算
适应性	针对 Apple 硬件优化	通用算法
尾部长度的	硬件决定，通常更长	受 CPU 预算限制

关键优势：参考信号对齐

软件 AEC 最大的挑战是精确知道"播放的声音什么时候从扬声器出来"。这个延迟受到：

- Audio Unit buffer 大小
- DAC 转换延迟
- 扬声器物理延迟

硬件 AEC 不需要估算这些延迟，因为 Bus 0 和 Bus 1 在同一个硬件时钟域内，参考信号的对齐是精确的。

6.5 BuiltInAECIsAvailable 的实现

```
// 源码文件：sdk/objc/native/src/audio/audio_device_ios.mm

bool AudioDeviceIOS::BuiltInAECIsAvailable() const {
    // VP I/O 在所有 iOS 设备上都可
    // 只有当 bypass_voice_processing_ = true 时才报告不可用
    return !bypass_voice_processing_;
}
```

这个简单的实现反映了一个事实：所有 iOS 设备都支持 VP I/O Audio Unit，硬件 AEC 始终可用。唯一的例外是应用主动选择绕过（`bypass_voice_processing_ = true`）。

7. 线程模型总结

7.1 线程职责划分

I/O 实时线程 (Core Audio 管理, 最高优先级)
├ OnGetPlayoutData 回调
├ OnDeliverRecordedData 回调
├ FineAudioBuffer 数据转换
└ AudioDeviceBuffer 数据传递
约束: 不能加锁、不能分配内存、不能做系统调用
WebRTC 工作线程 (thread_)
├ Init / StartPlayout / StopPlayout
├ InitRecording / StartRecording / StopRecording
├ HandleInterruptionBegin / End
├ HandleValidRouteChange
└ Audio Unit 生命周期管理
约束: RTC_DCHECK_RUN_ON(thread_) 保证单线程
系统通知线程 (不确定是哪个线程)
├ OnInterruptionBegin / End
├ OnValidRouteChange
└ 只做 PostTask 转发, 不做实际处理

7.2 线程安全机制

机制	用途	示例
atomic<int>	I/O 线程读取状态	playing_, recording_
RTC_DCHECK_RUN_ON	编译期线程检查	工作线程上的所有操作
PostTask	跨线程转发	通知线程 → 工作线程
无锁设计	I/O 线程安全	回调中不使用任何 mutex

7.3 原子变量的使用模式

```
// 工作线程上设置状态
void AudioDeviceIOS::StartPlayout() {
```

```

RTC_DCHECK_RUN_ON(thread_);
// ... 初始化逻辑 ...
playing_ = 1; // atomic store
}

void AudioDeviceIOS::StopPlayout() {
    RTC_DCHECK_RUN_ON(thread_);
    playing_ = 0; // atomic store
    // ... 清理逻辑 ...
}

// I/O 线程上读取状态
OSStatus AudioDeviceIOS::OnGetPlayoutData(...) {
    if (!playing_) { // atomic load
        // 填充静音
        return noErr;
    }
    // 正常处理
}

```

这种模式的正确性依赖于：

- `playing_` 的写入只在工作线程上发生（单写者）
- I/O 线程只读取（多读者）
- `atomic` 保证读取不会看到撕裂的值
- 最坏情况：I/O 线程多处理/少处理一帧，不会导致崩溃

8. 完整生命周期

8.1 初始化流程

```

AudioDeviceIOS::Init()
├─ 注册 AudioSessionObserver
├─ 配置 AVAudioSession (category、mode)
└─ 创建 VoiceProcessingAudioUnit 实例（但不初始化）

AudioDeviceIOS::InitPlayout()
├─ 查询 AVAudioSession 硬件参数
├─ 设置 playout_parameters_
├─ audio_unit_->Initialize() (配置 Audio Unit 格式)
└─ 创建 FineAudioBuffer

AudioDeviceIOS::StartPlayout()
├─ playing_ = 1
└─ audio_unit_->Start() (开始 I/O 回调)

```

8.2 销毁流程

```
AudioDeviceIOS::StopPlayout()  
├─ playing_ = 0  
├─ (如果 recording_ 也为 0) audio_unit_->Stop()  
└─ fine_audio_buffer_.reset()  
  
AudioDeviceIOS::Terminate()  
├─ StopPlayout() + StopRecording()  
├─ audio_unit_->Uninitialize()  
├─ audio_unit_->Dispose()  
└─ 注销 AudioSessionObserver
```

注意： `StopPlayout()` 不一定停止 Audio Unit。如果录音仍在进行，Audio Unit 继续运行（两个 Bus 共享同一个 Audio Unit 实例）。只有当播放和录音都停止时，才真正停止 Audio Unit。

9. 总结与设计哲学

9.1 核心设计原则

- 1. **实时安全：** I/O 回调路径上零分配、零锁、零系统调用
- 2. **适配器模式：** FineAudioBuffer 隔离平台差异，上层无感知
- 3. **Observer 解耦：** Audio Unit 管理与数据处理分离
- 4. **状态机驱动：** 中断/恢复通过明确的状态转换处理
- 5. **平台优先：** 优先使用硬件能力（AEC），减少 CPU 开销

9.2 与其他平台的对比

特性	iOS	Android	Windows
AEC 策略	硬件优先	硬件优先（OpenSL ES）	软件 AEC
帧适配	FineAudioBuffer	FineAudioBuffer	不需要（WASAPI 支持 10ms）
中断处理	AVAudioSession	AudioManager focus	无（桌面系统）
线程模型	单 I/O 线程	多线程（播放+录音）	多线程

9.3 源码文件索引

文件	行数 (约)	核心职责
audio_device_ios.h	~150	类声明、成员变量
audio_device_ios.mm	~800	完整实现
voice_processing_audio_unit.h	~80	Audio Unit 封装声明
voice_processing_audio_unit.mm	~400	Audio Unit 生命周期 + 回调
fine_audio_buffer.h	~50	适配层声明
fine_audio_buffer.cc	~100	帧大小转换逻辑

这些文件共同构成了 iOS 平台上完整的音频设备抽象层，将 Apple 的 Core Audio 框架无缝集成到 WebRTC 的跨平台音频管线中。

macOS 平台音频实现深度分析：AudioDeviceMac

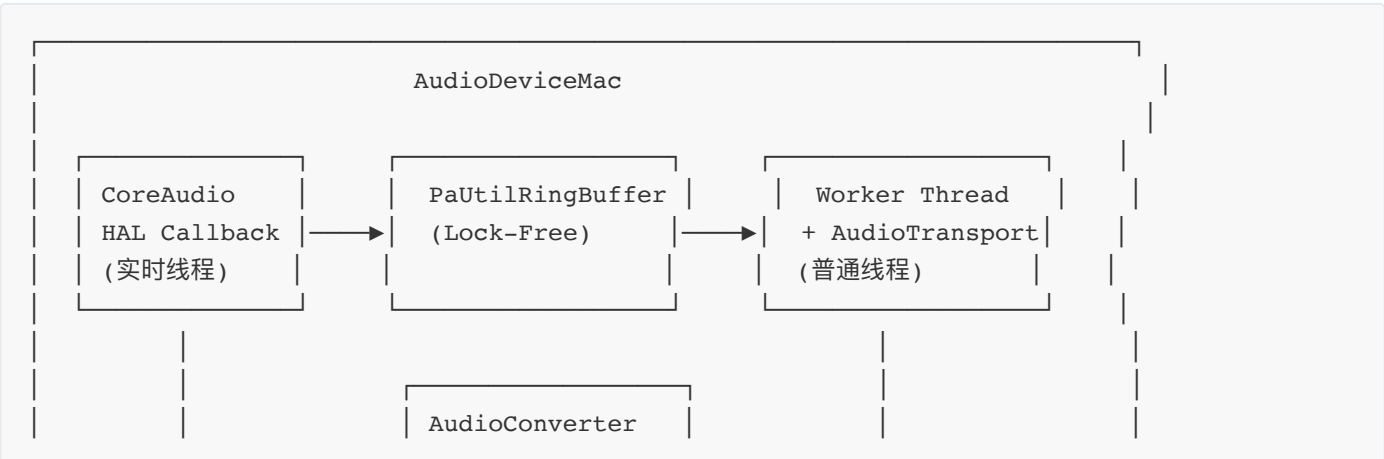
基于 WebRTC 源码，所有结论附源码文件引用。
源码路径：`modules/audio_device/mac/audio_device_mac.h` 及 `audio_device_mac.cc`

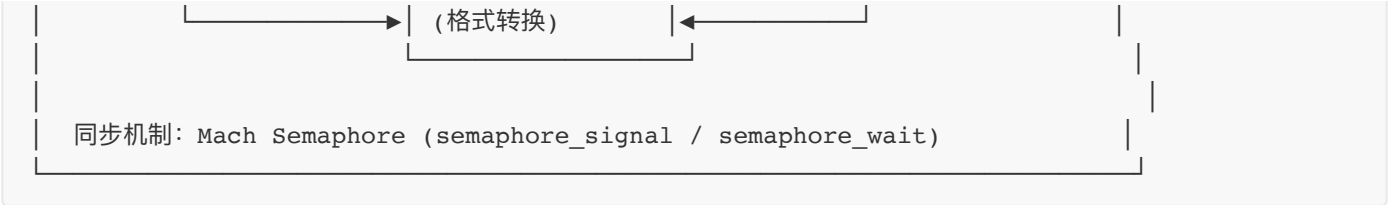
1. 架构概述

`AudioDeviceMac` 是 WebRTC 在 macOS 平台上的音频设备实现，基于 Apple CoreAudio HAL (Hardware Abstraction Layer) 构建。它实现了 `AudioDeviceGeneric` 接口，负责：

- 1. 音频采集：从麦克风获取 PCM 数据，送入 WebRTC 音频处理管线
- 2. 音频播放：从 WebRTC 获取混音后的 PCM 数据，送入扬声器
- 3. 设备管理：枚举、选择、监听音频设备变更
- 4. 格式转换：在设备原生格式与 WebRTC 内部格式之间转换

核心设计特征





源码依据: `modules/audio_device/mac/audio_device_mac.h`

2. 设计参数常量分析

```
// modules/audio_device/mac/audio_device_mac.h

const uint32_t N_REC_SAMPLES_PER_SEC = 48000; // 固定48kHz采样率
const uint32_t N_PLAY_SAMPLES_PER_SEC = 48000;
const uint32_t N_REC_CHANNELS = 1; // 默认单声道录音
const uint32_t N_PLAY_CHANNELS = 2; // 默认立体声播放
const int kBufferSizeMs = 10; // 10ms buffer
const int N_BLOCKS_IO = 2; // 双缓冲
const int N_BUFFERS_IN = 2; // 录音缓冲数 (>= N_BLOCKS_IO)
const int N_BUFFERS_OUT = 3; // 播放缓冲数 (>= N_BLOCKS_IO)
```

2.1 采样率选择：为什么是 48kHz？

设计决策分析：

考量因素	说明
macOS 默认采样率	CoreAudio 默认混音器运行在 44.1kHz 或 48kHz
编解码器兼容性	Opus 编解码器原生支持 48kHz
避免重采样	大多数 Mac 音频设备原生支持 48kHz
整除性	$48000 / 100 = 480$ 样本/帧（10ms），整除无余数

WebRTC 内部统一使用 48kHz 作为处理采样率。选择 48kHz 而非 44.1kHz 的关键原因是 48000 能被 10ms 帧长整除（480 样本），而 44100 不能（441 样本，会导致帧边界对齐问题）。

2.2 通道配置：录音单声道 vs 播放立体声

```
const uint32_t N_REC_CHANNELS = 1; // 单声道录音
const uint32_t N_PLAY_CHANNELS = 2; // 立体声播放
```

录音使用单声道的原因：

- 语音通信场景下，单声道足够（人声是单源信号）
- 减少带宽消耗（单声道数据量是立体声的一半）
- AEC（回声消除）算法在单声道下更高效、更稳定

- 即使麦克风是立体声的，WebRTC 也会在 AudioConverter 中混缩为单声道

播放使用立体声的原因：

- 大多数 Mac 输出设备（耳机、扬声器）是立体声
- 保留空间音频的可能性
- 避免 CoreAudio 在输出端做额外的声道映射

2.3 缓冲策略：10ms + 双缓冲/三缓冲

```
const int kBufferSizeMs = 10;    // 每个缓冲块 10ms
const int N_BLOCKS_IO = 2;       // I/O 操作使用双缓冲
const int N_BUFFERS_IN = 2;      // 录音：2 个缓冲块
const int N_BUFFERS_OUT = 3;     // 播放：3 个缓冲块
```

缓冲大小计算：

```
录音缓冲：48000 Hz × 1 ch × 2 bytes × 0.01s × 2 blocks = 1920 bytes
播放缓冲：48000 Hz × 2 ch × 2 bytes × 0.01s × 3 blocks = 5760 bytes
```

为什么播放比录音多一个缓冲块？

播放路径需要额外的缓冲来应对调度抖动：

- 录音：HAL 回调产生数据 → 写入 RingBuffer（生产者只有一个，节奏固定）
- 播放：工作线程预填充 → HAL 回调消费（消费者在实时线程，不能等待）

如果播放 RingBuffer 为空（underrun），会产生可听见的音频断裂。
多一个缓冲块提供了额外 10ms 的容错空间。

3. CoreAudio HAL 回调架构

3.1 双回调设计

CoreAudio HAL 使用 IOProc 回调机制，在实时音频线程中直接调用注册的回调函数。

`AudioDeviceMac` 注册了两个回调：

```
// modules/audio_device/mac/audio_device_mac.h

// 回调1：全双工回调（播放+录音共用）
static OSStatus deviceIOProc(
    AudioDeviceID device,
    const AudioTimeStamp* now,
    const AudioBufferList* inputData,    // 录音数据（从硬件来）
    const AudioTimeStamp* inputTime,
    AudioBufferList* outputData,         // 播放数据（写入硬件）
    const AudioTimeStamp* outputTime,
    void* clientData);
```



```
// 回调2: 独立录音回调 (仅在 _twoDevices=true 时使用)
static OSStatus inDeviceIOProc(
    AudioDeviceID device,
    const AudioTimeStamp* now,
    const AudioBufferList* inputData,
    const AudioTimeStamp* inputTime,
    AudioBufferList* outputData,
    const AudioTimeStamp* outputTime,
    void* clientData);
```

3.2 回调执行环境

CoreAudio HAL 回调运行在**实时优先级线程**（real-time priority thread）上，这意味着：

约束	原因
不能分配内存	malloc 可能触发锁竞争
不能获取互斥锁	可能导致优先级反转
不能做 I/O 操作	磁盘/网络 I/O 延迟不可预测
不能调用 Objective-C	ObjC 运行时有锁
执行时间必须极短	超时会导致音频断裂

这就是为什么需要 **RingBuffer + 工作线程**的架构：

HAL 回调不能直接调用 `AudioTransport::RecordedDataIsAvailable()` 或 `AudioTransport::NeedMorePlayData()`，因为这些函数可能涉及：

- 编解码器操作（可能分配内存）
- 网络发送（I/O 操作）
- 音频处理（可能耗时较长）

3.3 deviceIOProc 详细分析

```
// modules/audio_device/mac/audio_device_mac.cc

OSStatus AudioDeviceMac::deviceIOProc(
    AudioDeviceID device,
    const AudioTimeStamp* now,
    const AudioBufferList* inputData,
    const AudioTimeStamp* inputTime,
    AudioBufferList* outputData,
    const AudioTimeStamp* outputTime,
    void* clientData) {
```

```

AudioDeviceMac* ptrThis = (AudioDeviceMac*)clientData;

// ===== 播放部分 =====
// 从 RingBuffer 读取预填充的播放数据
ring_buffer_size_t numSamples =
    PaUtil_GetRingBufferReadAvailable(ptrThis->_paRenderBuffer);

// 将数据从 RingBuffer 拷贝到 outputData (硬件输出缓冲)
PaUtil_ReadRingBuffer(ptrThis->_paRenderBuffer,
    outputData->mBuffers[0].mData,
    numSamples);

// 通知播放工作线程：可以填充更多数据了
semaphore_signal(ptrThis->_renderSemaphore);

// ===== 录音部分 (仅 _twoDevices=false 时处理) =====
if (!ptrThis->_twoDevices) {
    // 将录音数据写入 RingBuffer
    PaUtil_WriteRingBuffer(ptrThis->_paCaptureBuffer,
        inputData->mBuffers[0].mData,
        numSamples);

    // 通知录音工作线程：有新数据可读
    semaphore_signal(ptrThis->_captureSemaphore);
}

return noErr;
}

```

关键设计点：

1. 零锁操作：整个回调中没有任何 mutex 操作，只使用 lock-free 的 RingBuffer
2. 信号量通知：使用 Mach semaphore（内核级信号量），开销极低
3. 条件分支：根据 `_twoDevices` 决定是否在此回调中处理录音

3.4 inDeviceIOProc 详细分析

```

// modules/audio_device/mac/audio_device_mac.cc

OSStatus AudioDeviceMac::inDeviceIOProc(
    AudioDeviceID device,
    const AudioTimeStamp* now,
    const AudioBufferList* inputData,
    const AudioTimeStamp* inputTime,
    AudioBufferList* outputData,
    const AudioTimeStamp* outputTime,
    void* clientData) {

    AudioDeviceMac* ptrThis = (AudioDeviceMac*)clientData;

```



```

}

// 将转换后的数据写入 RingBuffer
PaUtil_WriteRingBuffer(_paRenderBuffer, convertedData, numSamples);

return true; // 继续循环
}

```

播放数据流：

```

AudioTransport::NeedMorePlayData()
    |
    ▼
AudioDeviceBuffer::GetPlayoutData()
    |
    ▼
[可选] AudioConverter 格式转换 (48kHz stereo → 设备原生格式)
    |
    ▼
PaUtil_WriteRingBuffer(_paRenderBuffer)
    |
    ▼ (HAL 回调读取)
CoreAudio 硬件输出

```

4.3 录音工作线程：CaptureWorkerThread()

```

// modules/audio_device/mac/audio_device_mac.cc

bool AudioDeviceMac::CaptureWorkerThread() {
    // 等待 HAL 回调的信号 (表示 RingBuffer 有新数据)
    semaphore_wait(_captureSemaphore);

    if (_doStop || (_twoDevices && _doStopRec)) {
        return false; // 退出线程循环
    }

    // 从 RingBuffer 读取录音数据
    PaUtil_ReadRingBuffer(_paCaptureBuffer, rawData, numSamples);

    // 如果需要格式转换 (设备原生格式 → 48kHz mono)
    if (_captureConverter) {
        AudioConverterFillComplexBuffer(_captureConverter,
                                         inConverterProc,
                                         this,
                                         &numPackets,
                                         &bufferList,
                                         NULL);
    }

    // 将数据送入 WebRTC 音频处理管线
}

```

```
audio_device_buffer_.SetRecordedBuffer(convertedData, nSamples);
audio_device_buffer_.SetVQEData(playDelayMs, recDelayMs);
audio_device_buffer_.DeliverRecordedData();

return true; // 继续循环
}
```

录音数据流：



4.4 线程同步机制

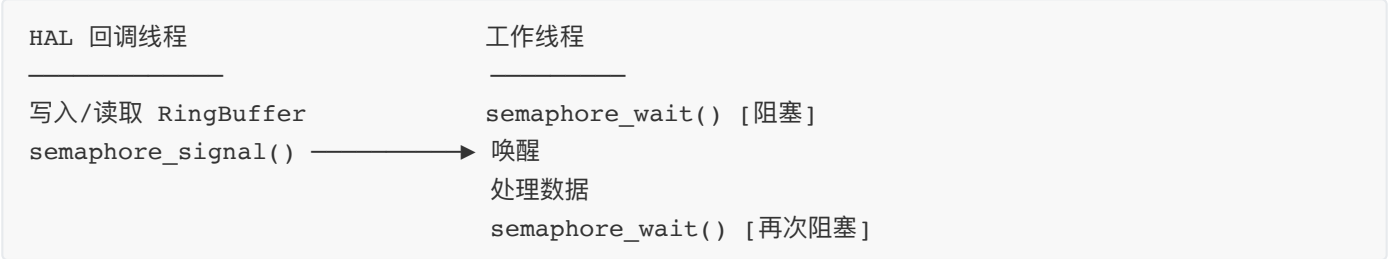
```
// modules/audio_device/mac/audio_device_mac.h

semaphore_t _renderSemaphore; // 播放线程等待信号
semaphore_t _captureSemaphore; // 录音线程等待信号
```

Mach Semaphore 选择的原因：

替代方案	为什么不用
pthread_cond	需要配合 mutex，HAL 回调中不能用 mutex
dispatch_semaphore	GCD 信号量有额外开销
eventfd (Linux)	macOS 不支持
Mach semaphore	✓ 内核级、无锁 signal、低延迟

同步流程：



这种设计确保工作线程只在有数据需要处理时才运行，避免了轮询（polling）带来的 CPU 浪费。

5. AudioConverter 格式转换机制

5.1 为什么需要格式转换？

CoreAudio 设备的原生格式可能与 WebRTC 期望的格式不同：

属性	WebRTC 期望（录音）	WebRTC 期望（播放）	设备可能的格式
采样率	48000 Hz	48000 Hz	44100/48000/96000 Hz
通道数	1 (mono)	2 (stereo)	1/2/多通道
位深	16-bit int	16-bit int	32-bit float
字节序	Native	Native	Native

5.2 格式描述符

```
// modules/audio_device/mac/audio_device_mac.h

AudioStreamBasicDescription _inStreamFormat; // 输入设备实际格式
AudioStreamBasicDescription _inDesiredFormat; // WebRTC 期望的录音格式 (48kHz mono 16-bit)
AudioStreamBasicDescription _outStreamFormat; // 输出设备实际格式
AudioStreamBasicDescription _outDesiredFormat; // WebRTC 期望的播放格式 (48kHz stereo 16-bit)
```

AudioStreamBasicDescription 是 CoreAudio 的标准格式描述结构：

```
struct AudioStreamBasicDescription {
    Float64 mSampleRate; // 采样率
    UInt32 mFormatID; // 格式标识 (kAudioFormatLinearPCM)
    UInt32 mFormatFlags; // 格式标志 (float/int, packed, etc.)
    UInt32 mBytesPerPacket; // 每包字节数
    UInt32 mFramesPerPacket; // 每包帧数 (PCM 通常为 1)
    UInt32 mBytesPerFrame; // 每帧字节数
    UInt32 mChannelsPerFrame; // 每帧通道数
    UInt32 mBitsPerChannel; // 每通道位数
};
```

5.3 转换器创建逻辑

```
// modules/audio_device/mac/audio_device_mac.cc
// 在 InitRecording() / InitPlayout() 中创建

// 录音转换器：设备格式 → WebRTC 格式
OSStatus err = AudioConverterNew(&_inStreamFormat,    // 源：设备原生
                                &_inDesiredFormat,    // 目标：48kHz mono 16-bit
                                &_captureConverter);

// 播放转换器：WebRTC 格式 → 设备格式
OSStatus err = AudioConverterNew(&_outDesiredFormat,  // 源：48kHz stereo 16-bit
                                &_outStreamFormat,    // 目标：设备原生
                                &_renderConverter);
```

条件创建：只有当设备格式与期望格式不同时才创建转换器。

如果设备原生就是 48kHz/16-bit/对应通道数，则 `_captureConverter` 或 `_renderConverter` 为 NULL，跳过转换步骤。

5.4 转换回调函数

AudioConverter 使用 pull 模型：当它需要源数据时，会调用用户提供的回调函数。

```
// modules/audio_device/mac/audio_device_mac.h

// 播放方向：当 AudioConverter 需要 WebRTC 格式数据时调用
static OSStatus outConverterProc(
    AudioConverterRef audioConverter,
    UInt32* numberDataPackets,
    AudioBufferList* data,
    AudioStreamPacketDescription** dataPacketDescription,
    void* userData);

// 录音方向：当 AudioConverter 需要设备原始数据时调用
static OSStatus inConverterProc(
    AudioConverterRef audioConverter,
    UInt32* numberDataPackets,
    AudioBufferList* data,
    AudioStreamPacketDescription** dataPacketDescription,
    void* userData);
```

录音方向转换流程：



播放方向转换流程：



5.5 设计决策：为什么用 AudioConverter 而不是自己实现？

考量	AudioConverter 优势
重采样质量	Apple 提供高质量 SRC（Sample Rate Conversion）
硬件加速	可能利用 Apple Silicon 的 DSP 单元
格式覆盖	自动处理 float↔int、大小端、通道映射
维护成本	系统库，无需自行维护
正确性	经过 Apple 大量测试验证

6. PaUtilRingBuffer 无锁环形缓冲

6.1 为什么选择 PaUtilRingBuffer？


```
// modules/audio_device/mac/audio_device_mac.h

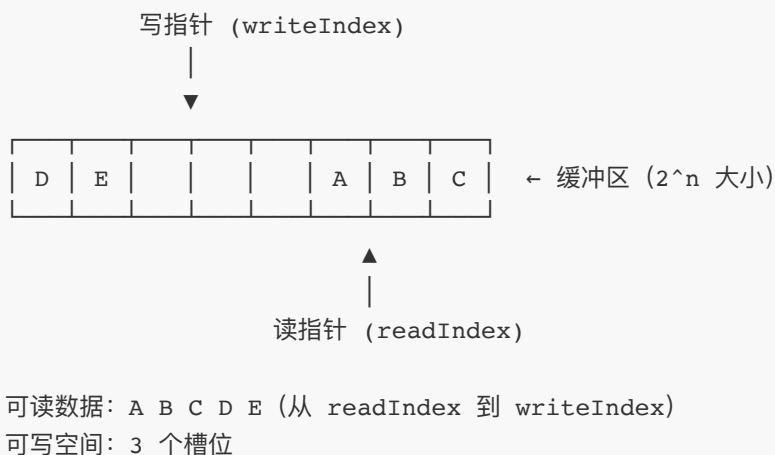
PaUtilRingBuffer* _paCaptureBuffer; // 录音环形缓冲
PaUtilRingBuffer* _paRenderBuffer;  // 播放环形缓冲
```

`PaUtilRingBuffer` 来自 PortAudio 项目，是一个经过广泛验证的**单生产者-单消费者（SPSC）无锁环形缓冲区**实现。

选择理由：

1. **无锁**：使用内存屏障（memory barrier）而非互斥锁，适合实时音频线程
2. **SPSC 模型**：恰好匹配 HAL 回调（生产者）+ 工作线程（消费者）的模式
3. **成熟稳定**：PortAudio 项目中经过数十年验证
4. **跨平台**：虽然这里用于 macOS，但同一实现可用于其他平台

6.2 环形缓冲区工作原理



关键特性：

- 缓冲区大小必须是 2 的幂（利用位运算实现高效取模）
- 读写指针各自只被一个线程修改（无竞争）
- 使用原子操作和内存屏障保证可见性

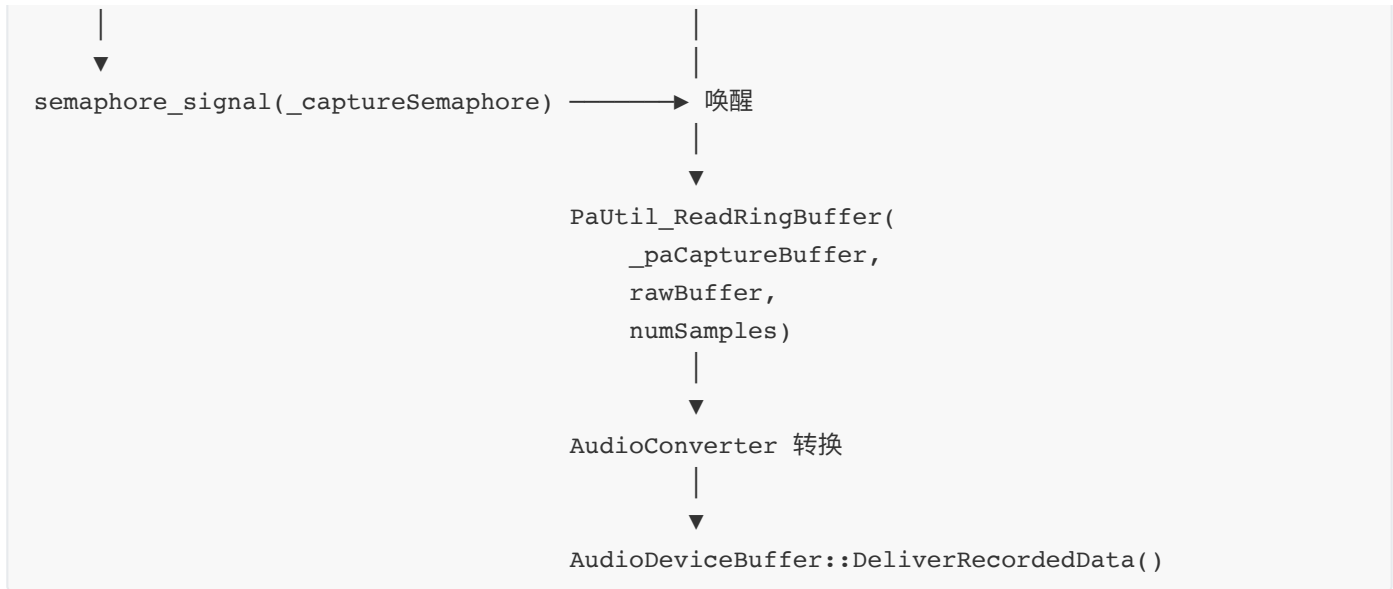
6.3 录音方向的 RingBuffer 使用

CoreAudio HAL 回调（实时线程）

```
inputData 到达
|
▼
PaUtil_WriteRingBuffer(
    _paCaptureBuffer,
    inputData->mBuffers[0].mData,
    numSamples)
```

CaptureWorkerThread（普通线程）

```
semaphore_wait(_captureSemaphore)
| [阻塞等待]
|
|
```



6.4 播放方向的 RingBuffer 使用



注意播放方向是反向的：

- 工作线程是生产者（预填充数据）
- HAL 回调是消费者（读取并送入硬件）

6.5 缓冲区大小与延迟的关系

```
录音延迟 = RingBuffer 中积累的数据量 / 采样率
          = N_BUFFERS_IN × kBufferSizeMs
          = 2 × 10ms = 20ms (最大)
```

```
播放延迟 = N_BUFFERS_OUT × kBufferSizeMs
          = 3 × 10ms = 30ms (最大)
```

```
总端到端延迟贡献 = 20ms + 30ms = 50ms (最大, 通常更低)
```

实际延迟通常低于最大值，因为工作线程会尽快处理数据，RingBuffer 不会总是满的。

6.6 Underrun 和 Overrun 处理

Underrun (播放缓冲区空)：

- HAL 回调读取时 RingBuffer 为空
- 结果：输出静音 (outputData 填零)
- 用户感知：音频断裂 (glitch)

Overrun (录音缓冲区满)：

- HAL 回调写入时 RingBuffer 已满
- 结果：新数据被丢弃
- 用户感知：录音数据丢失 (通常不明显)

设计上通过合理的缓冲块数量 (N_BUFFERS_IN/OUT) 来最小化这些情况的发生。

7. 设备变更监听机制

7.1 CoreAudio 属性监听器

macOS 的音频设备是动态的：用户可以随时插拔耳机、切换蓝牙设备、更改系统音频设置。

`AudioDeviceMac` 通过 CoreAudio 的属性监听机制来响应这些变化。

```
// modules/audio_device/mac/audio_device_mac.h

static OSStatus objectListenerProc(
    AudioObjectID objectId,
    UInt32 numberAddresses,
    const AudioObjectPropertyAddress addresses[],
    void* clientData);
```

7.2 监听的事件类型

```
// 注册监听的属性地址 (在 Init() 中设置)

// 1. 设备列表变化 (设备插拔)
```

```

AudioObjectPropertyAddress addr1 = {
    kAudioHardwarePropertyDevices,           // 属性：设备列表
    kAudioObjectPropertyScopeGlobal,         // 范围：全局
    kAudioObjectPropertyElementMaster        // 元素：主
};

// 2. 流格式变化（采样率/位深改变）
AudioObjectPropertyAddress addr2 = {
    kAudioDevicePropertyStreamFormat,
    kAudioDevicePropertyScopeInput, // 或 Output
    0
};

// 3. 数据源变化（如耳机/扬声器切换）
AudioObjectPropertyAddress addr3 = {
    kAudioDevicePropertyDataSource,
    kAudioDevicePropertyScopeOutput,
    0
};

// 4. 处理器过载（音频处理超时）
AudioObjectPropertyAddress addr4 = {
    kAudioDeviceProcessorOverload,
    kAudioObjectPropertyScopeGlobal,
    0
};

```

7.3 事件处理函数

```

// modules/audio_device/mac/audio_device_mac.cc

OSStatus AudioDeviceMac::objectListenerProc(
    AudioObjectID objectId,
    UInt32 numberAddresses,
    const AudioObjectPropertyAddress addresses[],
    void* clientData) {

    AudioDeviceMac* ptrThis = (AudioDeviceMac*)clientData;

    for (UInt32 i = 0; i < numberAddresses; i++) {
        switch (addresses[i].mSelector) {
            case kAudioHardwarePropertyDevices:
                ptrThis->HandleDeviceChange();
                break;
            case kAudioDevicePropertyStreamFormat:
                ptrThis->HandleStreamFormatChange();
                break;
            case kAudioDevicePropertyDataSource:
                ptrThis->HandleDataSourceChange();
                break;
            case kAudioDeviceProcessorOverload:

```

```
ptrThis->HandleProcessorOverload();
break;
}
}
return noErr;
}
```

7.4 各事件的处理策略

HandleDeviceChange() — 设备插拔

触发场景：

- USB 麦克风/耳机插入或拔出
- 蓝牙音频设备连接或断开
- 聚合设备创建或销毁

处理逻辑：

1. 重新枚举可用设备列表
2. 检查当前使用的设备是否仍然可用
3. 如果当前设备消失，尝试切换到默认设备
4. 通知上层（通过 AudioDeviceObserver）设备变更

HandleStreamFormatChange() — 格式变化

触发场景：

- 用户在"音频 MIDI 设置"中更改采样率
- 独占模式应用改变了设备格式
- 蓝牙设备在 A2DP/HFP 模式间切换（采样率从 48kHz 变为 8/16kHz）

处理逻辑：

1. 查询新的流格式（AudioStreamBasicDescription）
2. 销毁旧的 AudioConverter
3. 根据新格式创建新的 AudioConverter
4. 可能需要调整 RingBuffer 大小

HandleDataSourceChange() — 数据源切换

触发场景：

- MacBook 合盖/开盖（内置扬声器 ↔ 外接显示器音频）
- 耳机插入（扬声器 → 耳机）

处理逻辑：

- 通常不需要重建音频管线

- 可能需要更新音量设置

HandleProcessorOverload() — 处理器过载

触发场景：

- HAL 回调执行时间超过了缓冲区时长
- 系统负载过高导致音频线程被抢占

处理逻辑：

- 记录警告日志
- 增加过载计数器
- 严重时可能触发音频重启

7.5 监听器的生命周期管理

```
// Init() 中注册
AudioObjectAddPropertyListener(deviceId, &propertyAddress,
                               objectListenerProc, this);

// Terminate() 中注销
AudioObjectRemovePropertyListener(deviceId, &propertyAddress,
                                   objectListenerProc, this);
```

关键注意事项：

- 必须在 Terminate() 中移除所有监听器，否则回调可能访问已释放的对象
- 监听器回调可能在任意线程上被调用（通常是 CoreAudio 的内部线程）
- 处理函数中需要注意线程安全

8. _twoDevices 模式详解

8.1 模式定义

```
// modules/audio_device/mac/audio_device_mac.h

bool _twoDevices; // 输入输出设备不同时为 true
bool _doStop;     // 停止播放（或全双工停止信号）
bool _doStopRec;  // 停止录音（仅 _twoDevices=true 时使用）
```

8.2 为什么需要区分单设备和双设备？

CoreAudio HAL 的 IOProc 机制有一个重要特性：

单设备模式（_twoDevices = false）：

- 输入和输出是同一个物理设备（如 USB 音频接口、内置音频）

- 一个 IOProc 同时处理输入和输出
- 输入和输出共享同一个时钟源，天然同步
- 只需注册一个 `deviceIOProc`

双设备模式 (`_twoDevices = true`) :

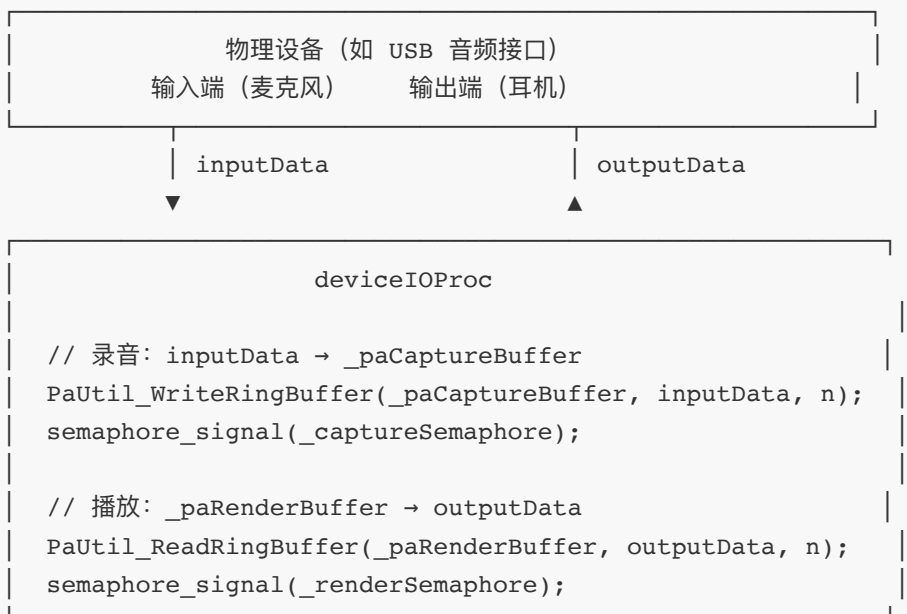
- 输入和输出是不同的物理设备（如内置麦克风 + USB 耳机）
- 两个设备有独立的时钟源，可能存在时钟漂移
- 需要分别注册 IOProc
- 输出设备使用 `deviceIOProc`，输入设备使用 `inDeviceIOProc`

8.3 模式判断逻辑

```
// modules/audio_device/mac/audio_device_mac.cc
// 在 InitRecording() 或 InitPlayout() 中判断

if (_inputDeviceID == _outputDeviceID) {
    _twoDevices = false;
    // 使用单个 deviceIOProc 处理全双工
} else {
    _twoDevices = true;
    // 输出设备注册 deviceIOProc
    // 输入设备注册 inDeviceIOProc
}
```

8.4 单设备模式的数据流

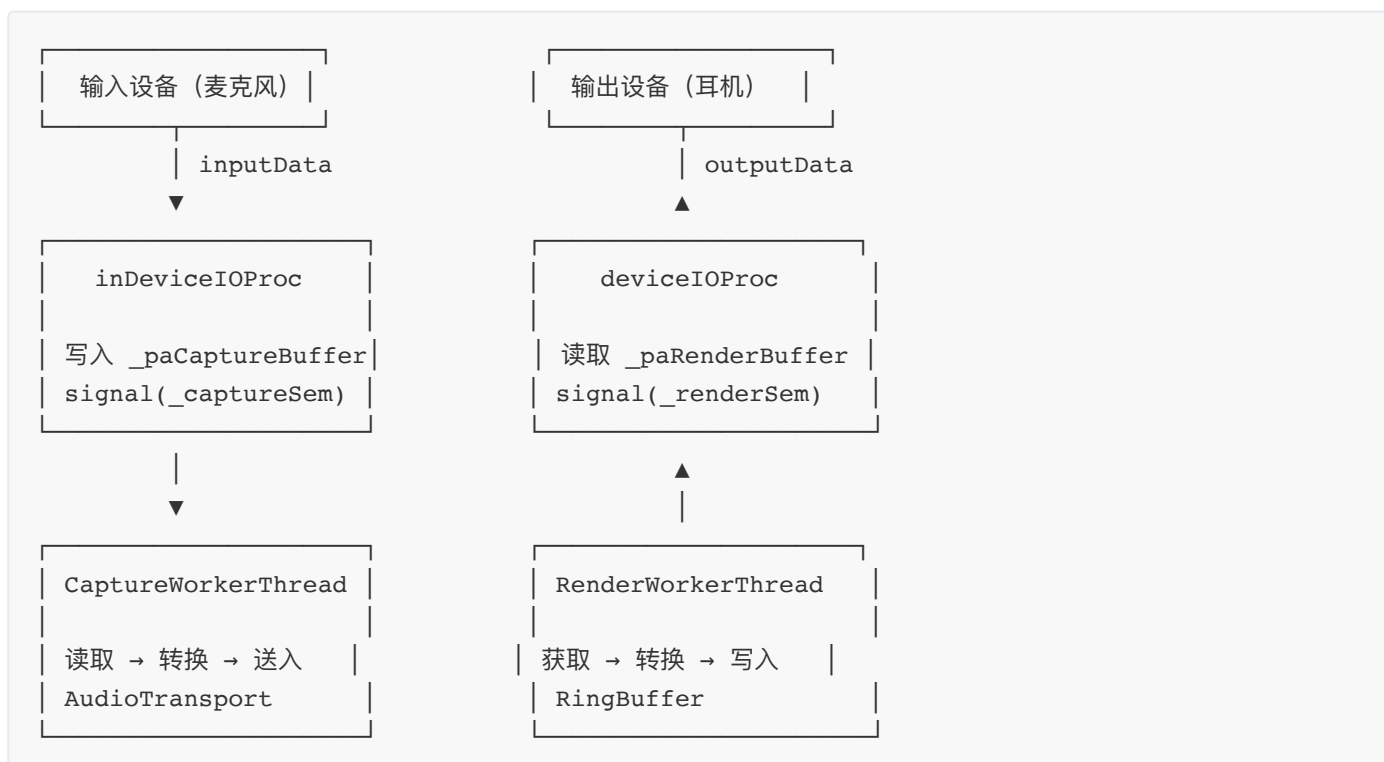


优势:

- 输入输出在同一回调中处理，时间戳完全对齐

- AEC（回声消除）可以精确计算延迟
- 只有一个 IOProc 的开销

8.5 双设备模式的数据流



挑战：时钟漂移

两个独立设备有各自的晶振时钟，长时间运行后会产生漂移：

- 设备 A 的 48000 Hz 实际可能是 47999.5 Hz
- 设备 B 的 48000 Hz 实际可能是 48000.3 Hz
- 每小时漂移约 $0.8 \text{ ppm} \times 3600\text{s} \approx$ 几个样本

WebRTC 通过 RingBuffer 的弹性来吸收少量漂移，严重漂移时可能触发重同步。

8.6 停止控制的差异

```

// 单设备模式停止
_doStop = true;
semaphore_signal(_renderSemaphore); // 唤醒播放线程使其退出
semaphore_signal(_captureSemaphore); // 唤醒录音线程使其退出
// 两个线程都检查 _doStop

// 双设备模式停止播放
_doStop = true;
semaphore_signal(_renderSemaphore);
// 只停止播放，录音继续

// 双设备模式停止录音
_doStopRec = true;

```



```
semaphore_signal(_captureSemaphore);  
// 只停止录音，播放继续
```

为什么单设备模式不需要 `_doStopRec` ?

在单设备模式下，`deviceIOProc` 同时处理输入和输出。如果只停止录音而不停止播放，`IOProc` 仍然在运行（为了播放），只是不再将 `inputData` 写入 `RingBuffer`。
控制逻辑通过 `_doStop` 统一管理。

8.7 IOProc 注册与启停

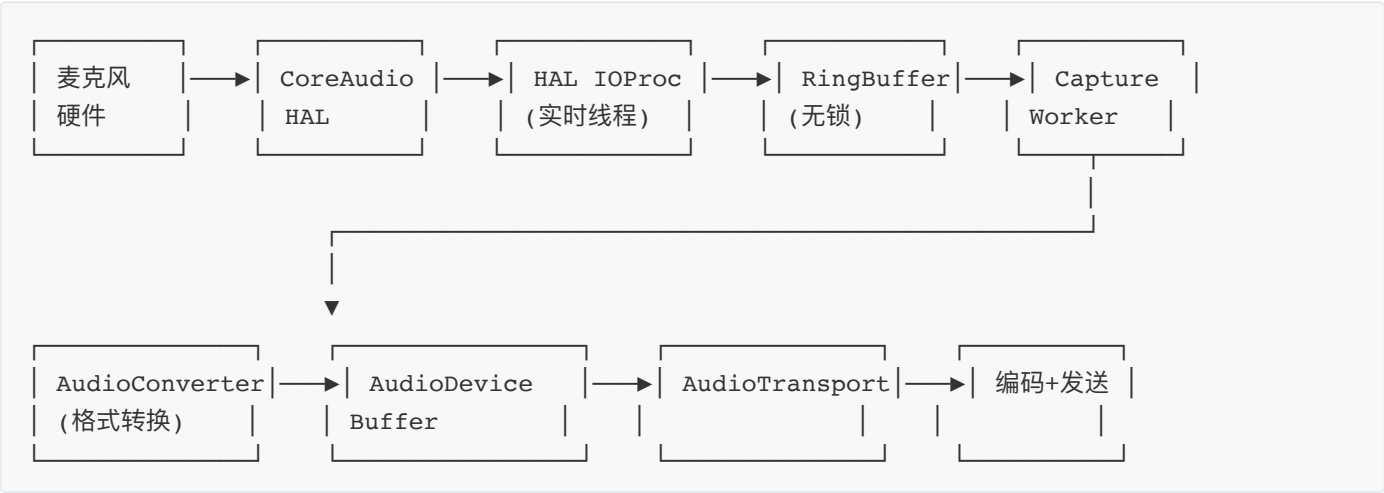
```
// 单设备模式  
AudioDeviceCreateIOProcID(_outputDeviceID, deviceIOProc, this, &_deviceIOProcID);  
AudioDeviceStart(_outputDeviceID, _deviceIOProcID);  
  
// 双设备模式 - 额外注册输入设备的 IOProc  
AudioDeviceCreateIOProcID(_inputDeviceID, inDeviceIOProc, this, &_inDeviceIOProcID);  
AudioDeviceStart(_inputDeviceID, _inDeviceIOProcID);  
// 输出设备同样注册 deviceIOProc  
AudioDeviceCreateIOProcID(_outputDeviceID, deviceIOProc, this, &_deviceIOProcID);  
AudioDeviceStart(_outputDeviceID, _deviceIOProcID);
```

停止时的顺序：

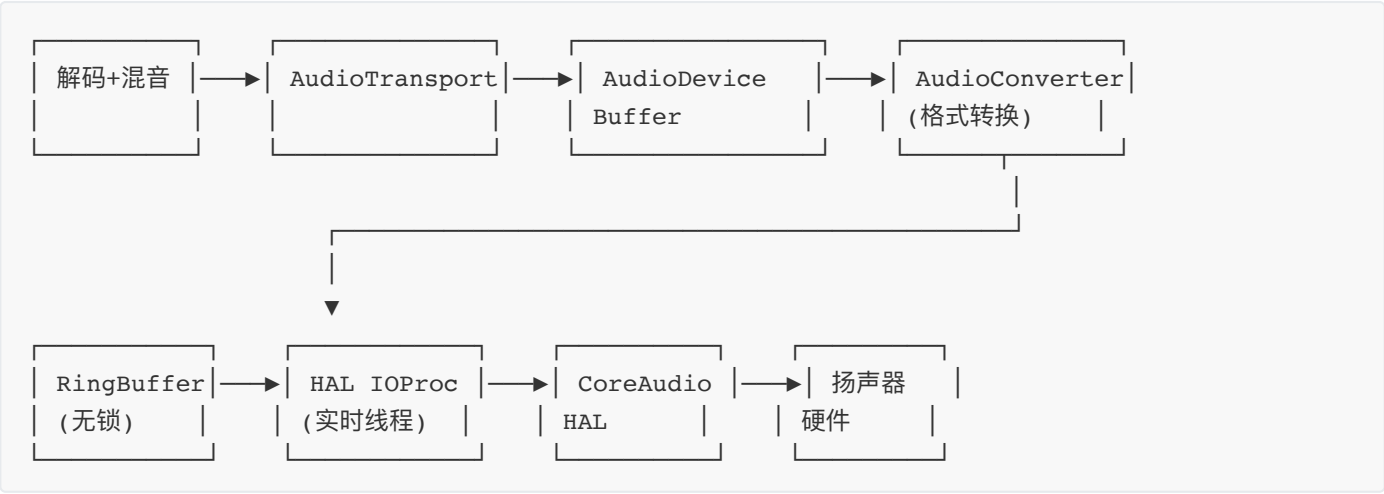
```
// 必须先停止 IOProc，再销毁  
AudioDeviceStop(_outputDeviceID, _deviceIOProcID);  
AudioDeviceDestroyIOProcID(_outputDeviceID, _deviceIOProcID);  
  
if (_twoDevices) {  
    AudioDeviceStop(_inputDeviceID, _inDeviceIOProcID);  
    AudioDeviceDestroyIOProcID(_inputDeviceID, _inDeviceIOProcID);  
}
```

9. 完整数据流总结

9.1 录音完整路径（从硬件到网络）



9.2 播放完整路径（从网络到硬件）



9.3 关键设计决策总结

设计决策	原因	源码依据
48kHz 固定采样率	Opus 兼容 + 10ms 整除	<code>N_REC_SAMPLES_PER_SEC = 48000</code>
录音单声道	语音通信 + AEC 效率	<code>N_REC_CHANNELS = 1</code>
播放立体声	匹配输出设备	<code>N_PLAY_CHANNELS = 2</code>
10ms 缓冲块	WebRTC 标准帧长	<code>kBufferSizeMs = 10</code>
播放 3 块缓冲	防止 underrun	<code>N_BUFFERS_OUT = 3</code>
无锁 RingBuffer	实时线程安全	<code>PaUtilRingBuffer*</code>
Mach Semaphore	低延迟内核级同步	<code>semaphore_t</code>
AudioConverter	高质量格式转换	<code>AudioConverterRef</code>
双回调模式	支持异构设备组合	<code>_twoDevices</code>
属性监听器	动态设备管理	<code>objectListenerProc</code>

9.4 与其他平台的对比

特性	macOS (AudioDeviceMac)	iOS (AudioUnitIOS)	Android (AAudio)
API 层级	HAL (最底层)	AudioUnit (中层)	AAudio (中层)
回调模型	IOProc	Render Callback	数据回调
格式转换	AudioConverter	AudioUnit 内置	自行实现
设备管理	属性监听器	Route Change	AudioManager
缓冲策略	RingBuffer	FineAudioBuffer	内置缓冲
线程模型	HAL 实时线程 + 工作线程	AudioUnit 线程	AAudio 线程

macOS 使用最底层的 HAL API，获得了最大的控制力，但也承担了最多的复杂性。
iOS 使用更高层的 AudioUnit API，系统帮助处理了更多细节。

10. 源码文件索引

文件	职责
modules/audio_device/mac/audio_device_mac.h	类声明、常量定义、成员变量
modules/audio_device/mac/audio_device_mac.cc	完整实现
modules/audio_device/mac/portaudio/pa_ringbuffer.h	RingBuffer 接口
modules/audio_device/mac/portaudio/pa_ringbuffer.c	RingBuffer 实现
modules/audio_device/audio_device_buffer.h	AudioDeviceBuffer 接口
modules/audio_device/audio_device_buffer.cc	AudioDeviceBuffer 实现
modules/audio_device/include/audio_device.h	AudioDeviceModule 接口定义
modules/audio_device/audio_device_generic.h	平台实现基类

文档生成时间：2026-05-07
基于 WebRTC M7680 源码分析

Windows 平台音频实现深度分析

源码路径：modules/audio_device/win/

一、两套实现概览

Windows 上 WebRTC 提供了两套独立的 WASAPI 实现，通过 AudioLayer 枚举区分：

AudioDeviceModule::kWindowsCoreAudio	→ AudioDeviceWindowsCore (旧)
AudioDeviceModule::kWindowsCoreAudio2	→ AudioDeviceModuleWin (新, 推荐)

特性	AudioDeviceWindowsCore (旧)	AudioDeviceModuleWin (新)
源码位置	win/audio_device_core_win.h	win/audio_device_module_win.h
架构	单类实现全部接口	Input/Output 分离
自动重启	不支持	支持 (automatic_restart)
工厂方法	AudioDeviceModuleImpl::Create	CreateWindowsCoreAudioAudioDeviceModule
COM 要求	调用方初始化	IsSupported() 检测
推荐场景	兼容旧代码	新项目

二、旧实现：AudioDeviceWindowsCore

文件：modules/audio_device/win/audio_device_core_win.h

2.1 依赖的 Windows API

```
#include <audioclient.h>    // IAudioClient, IAudioRenderClient, IAudioCaptureClient
#include <audiopolicy.h>    // IAudioSessionControl
#include <mmdeviceapi.h>    // IMMDeviceEnumerator, IMMDevice
#include <avrt.h>           // MMCSS (Multimedia Class Scheduler Service)
#pragma comment(lib, "avrt.lib")
```

关键接口：

- IMMDeviceEnumerator：枚举音频端点设备
- IAudioClient：WASAPI 核心，管理音频流
- IAudioRenderClient：写入播放数据
- IAudioCaptureClient：读取录音数据
- MMCSS：将线程提升为 Pro Audio 优先级，减少调度延迟

2.2 静态检测方法

```
// audio_device_core_win.cc
static bool AudioDeviceWindowsCore::CoreAudioIsSupported() {
    // 检测 Vista+ 的 WASAPI 支持
    // 通过尝试创建 IMMDeviceEnumerator 来验证
    // COM 必须已初始化，否则返回 false
}
```

重要：CoreAudioIsSupported() 要求 COM 已初始化。如果在未初始化 COM 的线程上调用，会返回 false 导致 ADM 创建失败。

2.3 事件驱动线程模型

旧实现使用主动等待模式（与 macOS 的被动回调相反）：

```
[render 线程]
```

```

loop:
    WaitForSingleObject(render_event_, INFINITE) ← 等待 WASAPI 事件
    IAudioRenderClient::GetBuffer() ← 获取写入缓冲区
    AudioDeviceBuffer::RequestPlayOutData() ← 向上层拉取 PCM
    IAudioRenderClient::ReleaseBuffer() ← 提交数据给硬件

[capture 线程]
loop:
    WaitForSingleObject(capture_event_, INFINITE) ← 等待 WASAPI 事件
    IAudioCaptureClient::GetBuffer() ← 获取录音数据
    AudioDeviceBuffer::SetRecordedBuffer() ← 存入缓冲
    AudioDeviceBuffer::DeliverRecordedData() ← 推给 AudioTransport
    IAudioCaptureClient::ReleaseBuffer() ← 释放缓冲区

```

MMCSS 提升线程优先级:

```

// 将线程注册为 "Pro Audio" 任务, 获得更高调度优先级
DWORD task_index = 0;
HANDLE mmcsh_handle = AvSetMmThreadCharacteristicsA("Pro Audio", &task_index);
// 完成后恢复
AvRevertMmThreadCharacteristics(mmcsh_handle);

```

三、新实现: AudioDeviceModuleWin

文件: `modules/audio_device/win/audio_device_module_win.h`

3.1 Input/Output 分离架构

```

// 输入接口 (录音)
class AudioInput {
    virtual int Init() = 0;
    virtual int Terminate() = 0;
    virtual int NumDevices() const = 0;
    virtual int SetDevice(int index) = 0;
    virtual int SetDevice(AudioDeviceModule::WindowsDeviceType device) = 0;
    virtual int DeviceName(int index, std::string* name, std::string* guid) = 0;
    virtual void AttachAudioBuffer(AudioDeviceBuffer* audio_buffer) = 0;
    virtual bool RecordingIsInitialized() const = 0;
    virtual int InitRecording() = 0;
    virtual int StartRecording() = 0;
    virtual int StopRecording() = 0;
    virtual bool Recording() = 0;
    virtual int VolumeIsAvailable(bool* available) = 0;
    virtual int RestartRecording() = 0; // ← 新增: 自动重启
    virtual bool Restarting() const = 0; // ← 新增: 重启状态查询
    virtual int SetSampleRate(uint32_t sample_rate) = 0; // ← 新增
};

// 输出接口 (播放), 对称设计

```

```
class AudioOutput {
    // ... 同上, 播放方向
    virtual int RestartPlayout() = 0;
    virtual bool Restarting() const = 0;
    virtual int SetSampleRate(uint32_t sample_rate) = 0;
};
```

3.2 工厂方法与 COM 要求

文件: `modules/audio_device/include/audio_device_factory.cc`

```
scoped_refptr<AudioDeviceModule>
CreateWindowsCoreAudioAudioDeviceModule(const Environment& env,
                                          bool automatic_restart) {
    // 检测 Core Audio 支持 (同时验证 COM 是否已初始化)
    if (!webrtc_win::core_audio_utility::IsSupported()) {
        RTC_LOG(LS_ERROR)
            << "Unable to create ADM since Core Audio is not supported";
        return nullptr; // ← COM 未初始化时返回 nullptr
    }
    // 创建分离的 Input/Output 实现
    return CreateWindowsCoreAudioAudioDeviceModuleFromInputAndOutput(
        env,
        std::make_unique<webrtc_win::CoreAudioInput>(env, automatic_restart),
        std::make_unique<webrtc_win::CoreAudioOutput>(env, automatic_restart));
}
```

COM 初始化要求:

```
// 调用方必须在同一线程上先初始化 COM
// WebRTC 提供了 ScopedCOMInitializer 工具类
#include "rtc_base/win/scoped_com_initializer.h"

// 正确用法 (在 worker_thread 上):
ScopedCOMInitializer com_initializer(ScopedCOMInitializer::kMTA);
if (!com_initializer.Succeeded()) {
    // COM 初始化失败, 无法创建 ADM
}
auto adm = CreateWindowsCoreAudioAudioDeviceModule(env, true);
```

为什么 COM 必须在 `worker_thread` 上初始化?

WASAPI 的 COM 接口使用 MTA (多线程套间), 所有对 COM 对象的调用必须在同一套间内。WebRTC 的 `Init()`、`StartPlayout()` 等方法都在 `worker_thread` 上调用, 因此 COM 必须在 `worker_thread` 上初始化。

3.3 CoreAudioInput/Output 自动重启机制

文件: `modules/audio_device/win/core_audio_input_win.h`

```
class CoreAudioInput : public AudioInput {
public:
    // automatic_restart=true: 设备断开时自动重新初始化
    explicit CoreAudioInput(const Environment& env, bool automatic_restart);

    // 重启相关
    int RestartRecording() override; // 触发重启
    bool Restarting() const override; // 是否正在重启中
};
```

自动重启触发条件：

- 音频设备被拔出（AUDCLNT_E_DEVICE_INVALIDATED）
- 设备格式变化
- 独占模式被其他应用抢占

重启流程：

```
设备错误事件
→ 录音/播放线程检测到 AUDCLNT_E_DEVICE_INVALIDATED
→ 设置 restarting_ = true
→ StopRecording() / StopPlayout()
→ 重新枚举设备
→ InitRecording() / InitPlayout()
→ StartRecording() / StartPlayout()
→ restarting_ = false
```

四、WASAPI Shared Mode vs Exclusive Mode

WebRTC 使用 **Shared Mode**（共享模式）：

```
// IAudioClient::Initialize 调用
hr = audio_client->Initialize(
    AUDCLNT_SHAREMODE_SHARED, // ← 共享模式，与其他应用共享设备
    AUDCLNT_STREAMFLAGS_EVENTCALLBACK, // ← 事件驱动
    buffer_duration,
    0,
    wave_format,
    nullptr);
```

模式	延迟	格式	适用场景
Shared	较高（~20ms）	由 Windows 混音器决定	WebRTC（默认）
Exclusive	极低（<5ms）	应用自定义	专业音频软件

Shared Mode 下 Windows 音频引擎（AudioEngine）负责混音，WebRTC 不需要自己处理多应用共存。

五、设备选择：kDefaultCommunicationDevice

文件：media/engine/adm_helpers.cc

```
#if defined(WEBRTC_WIN)
#define AUDIO_DEVICE_ID \
    (AudioDeviceModule::WindowsDeviceType::kDefaultCommunicationDevice)
#else
#define AUDIO_DEVICE_ID (0u)
#endif

void Init(AudioDeviceModule* adm) {
    // Windows 上使用 kDefaultCommunicationDevice (通信设备)
    // 而不是 kDefaultDevice (默认多媒体设备)
    adm->SetPlayoutDevice(AUDIO_DEVICE_ID);
    adm->SetRecordingDevice(AUDIO_DEVICE_ID);
}
```

kDefaultCommunicationDevice 是 Vista+ 引入的"默认通信设备"概念：

- 用户可以在控制面板中单独设置通信设备（如耳机）和多媒体设备（如音箱）
- WebRTC 使用通信设备，确保在 VoIP 场景下使用正确的设备
- 对应 IMMDeviceEnumerator::GetDefaultAudioEndpoint(eConsole, eCommunications, ...)

六、线程模型

```
[signaling_thread]
    ConnectionContext 构造
        → CreateMediaEngine()
            → new WebRtcVoiceEngine(adm=nullptr) ← ADM 对象在此创建
            → EnsureAudioDeviceModule()
                → CreateWindowsCoreAudioAudioDeviceModule() ← 需要 COM 已初始化!

[worker_thread]
    AddRefMediaEngine()
        → WebRtcVoiceEngine::Init()
        → adm_helpers::Init(adm)
            → adm->Init() ← WASAPI IAudioClient 初始化
            → adm->SetPlayoutDevice(kDefaultCommunicationDevice)
            → adm->InitSpeaker()
            → adm->SetRecordingDevice(kDefaultCommunicationDevice)
            → adm->InitMicrophone()

    AudioState::AddReceivingStream()
        → adm->InitPlayout() ← 创建 render 线程, 注册 WASAPI 事件
        → adm->StartPlayout() ← 启动 render 线程循环

[render 线程 (WASAPI 事件驱动)]
```



```
WaitForSingleObject(render_event_)
    → IAudioRenderClient::GetBuffer()
    → AudioDeviceBuffer::RequestPlayOutData()
    → IAudioRenderClient::ReleaseBuffer()

[ capture 线程 (WASAPI 事件驱动) ]
WaitForSingleObject(capture_event_)
    → IAudioCaptureClient::GetBuffer()
    → AudioDeviceBuffer::SetRecordedBuffer() + DeliverRecordedData()
    → IAudioCaptureClient::ReleaseBuffer()
```

七、常见问题

Q：为什么 `CreateWindowsCoreAudioAudioDeviceModule` 返回 `nullptr`？

最常见原因是 COM 未初始化。检查：

- 1. 调用线程是否已调用 `CoInitializeEx` 或使用 `ScopedCOMInitializer`
- 2. `core_audio_utility::IsSupported()` 返回 `false` 时会打印 `LS_ERROR` 日志

Q：旧实现和新实现可以混用吗？

不能。`kWindowsCoreAudio` 和 `kWindowsCoreAudio2` 是完全独立的代码路径，通过 `AudioLayer` 枚举在 `AudioDeviceModuleImpl::Create` 中区分：

```
// audio_device_impl.cc
if (audio_layer == AudioDeviceModule::kWindowsCoreAudio2) {
    RTC_LOG(LS_ERROR) << "Use the CreateWindowsCoreAudioAudioDeviceModule() "
                          "factory method instead for this option.";
    return nullptr; // ← 明确拒绝通过旧工厂创建新实现
}
```

八、源码文件索引

文件	内容
<code>modules/audio_device/win/audio_device_core_win.h/.cc</code>	旧实现 <code>AudioDeviceWindowsCore</code>
<code>modules/audio_device/win/audio_device_module_win.h/.cc</code>	新实现框架， <code>AudioInput/AudioOutput</code> 接口
<code>modules/audio_device/win/core_audio_input_win.h/.cc</code>	新实现录音端
<code>modules/audio_device/win/core_audio_output_win.h/.cc</code>	新实现播放端
<code>modules/audio_device/win/core_audio_base_win.h/.cc</code>	新实现公共基类
<code>modules/audio_device/win/core_audio_utility_win.h/.cc</code>	WASAPI 工具函数，含 <code>IsSupported()</code>
<code>modules/audio_device/include/audio_device_factory.h/.cc</code>	<code>CreateWindowsCoreAudioAudioDeviceModule</code> 工厂
<code>rtc_base/win/scoped_com_initializer.h</code>	COM 初始化 RAII 工具

Linux 平台音频实现深度分析

源码路径: `modules/audio_device/linux/`

一、两套实现概览

Linux 上 WebRTC 提供两套实现, 通过构建标志和 `AudioLayer` 枚举选择:

```
kLinuxPulseAudio    → AudioDeviceLinuxPulse (默认, 推荐)
kLinuxAlsaAudio     → AudioDeviceLinuxALSA (备选)
```

选择逻辑来自 `audio_device_impl.cc` 的 `CreatePlatformSpecificObjects`:

```
#if !defined(WEBRTC_ENABLE_LINUX_PULSE)
// 构建标志 rtc_include_pulse_audio=false: 只有 ALSA
RTC_LOG(LS_WARNING) << "PulseAudio is disabled using build flag.";
if ((audio_layer == kLinuxAlsaAudio) ||
    (audio_layer == kPlatformDefaultAudio)) {
    audio_device_.reset(new AudioDeviceLinuxALSA());
}
#else
// 默认: PulseAudio 优先
RTC_LOG(LS_INFO) << "PulseAudio support is enabled.";
if ((audio_layer == kLinuxPulseAudio) ||
    (audio_layer == kPlatformDefaultAudio)) {
    audio_device_.reset(new AudioDeviceLinuxPulse()); // ← 默认选择
} else if (audio_layer == kLinuxAlsaAudio) {
    audio_device_.reset(new AudioDeviceLinuxALSA());
}
#endif
```

为什么 **PulseAudio** 是默认选择?

- PulseAudio 是现代 Linux 桌面的标准音频服务器 (Ubuntu/Fedora/Debian 默认)
- 支持多应用共享设备、热插拔、蓝牙音频路由
- ALSA 是底层驱动接口, 直接访问硬件, 不支持多应用共享

二、PulseAudio 实现: `AudioDeviceLinuxPulse`

文件: `modules/audio_device/linux/audio_device_pulse_linux.h`

2.1 延迟常量设计

```
// 播放延迟: 最小 20ms, 每次 underflow 增加 20ms
const uint32_t WEBRTC_PA_PLAYBACK_LATENCY_MINIMUM_MSECS = 20;
const uint32_t WEBRTC_PA_PLAYBACK_LATENCY_INCREMENT_MSECS = 20;
// 请求大小 = buffer_size / 2 (平衡 CPU 开销和 underflow 风险)
const uint32_t WEBRTC_PA_PLAYBACK_REQUEST_FACTOR = 2;

// 录音延迟: 推荐 10ms (1ms 约 6-7% CPU, 10ms 约 4%)
const uint32_t WEBRTC_PA_LOW_CAPTURE_LATENCY_MSECS = 10;
// 服务端额外缓冲 (防止网络往返导致 overflow)
const uint32_t WEBRTC_PA_CAPTURE_BUFFER_EXTRA_MSECS = 750;
```

这些常量揭示了 PulseAudio 的延迟特性: 播放最低 20ms, 比 iOS (~10ms) 和 macOS (~10ms) 高, 原因是 PulseAudio 的 C/S 架构引入了额外的网络往返延迟。

2.2 异步回调架构

PulseAudio 使用异步事件循环 (pa_mainloop), 所有操作通过回调完成:

```
// 静态回调函数 (PulseAudio C API 要求) → 转发给实例方法
static void PaContextStateCallback(pa_context* c, void* pThis);
static void PaSinkInfoCallback(pa_context* c, const pa_sink_info* i,
                               int eol, void* pThis);
static void PaSourceInfoCallback(pa_context* c, const pa_source_info* i,
                                 int eol, void* pThis);
static void PaStreamStateCallback(pa_stream* p, void* pThis);

// 数据流回调
static void PaStreamWriteCallback(pa_stream* unused,
                                  size_t buffer_space, void* pThis);
static void PaStreamReadCallback(pa_stream* unused1,
                                 size_t unused2, void* pThis);
static void PaStreamUnderflowCallback(pa_stream* unused, void* pThis);
static void PaStreamOverflowCallback(pa_stream* unused, void* pThis);
```

回调转发模式 (所有静态回调都这样做) :

```
void AudioDeviceLinuxPulse::PaStreamWriteCallback(
    pa_stream* unused, size_t buffer_space, void* pThis) {
    // 将 void* 转回实例指针, 调用实例方法
    static_cast<AudioDeviceLinuxPulse*>(pThis)
        ->PaStreamWriteCallbackHandler(buffer_space);
}
```

2.3 双工作线程

```
PlatformThread _ptrThreadPlay; // 播放线程
PlatformThread _ptrThreadRec; // 录音线程

static void PlayThreadFunc(void*); // → PlayThreadProcess()
static void RecThreadFunc(void*); // → RecThreadProcess()
```

线程同步使用 Event:

```
Event _timeEventRec; // 定时触发录音处理
Event _timeEventPlay; // 定时触发播放处理
Event _recStartEvent; // 录音启动确认
Event _playStartEvent; // 播放启动确认
```

2.4 播放数据流

```
[播放线程 PlayThreadProcess()]
_timeEventPlay.Wait(1000) ← 等待定时事件 (10ms 周期)
↓
PaLock() ← 锁定 PulseAudio mainloop
↓
pa_stream_writable_size() ← 查询可写空间
↓
AudioDeviceBuffer::RequestPayoutData() ← 向上层拉取 PCM
↓
pa_stream_write() ← 写入 PulseAudio stream
↓
PaUnlock()

[PaStreamWriteCallbackHandler]
← PulseAudio 通知有可写空间时触发
EnableWriteCallback() / DisableWriteCallback() 控制回调开关
```

2.5 录音数据流

```
[PaStreamReadCallbackHandler]
← PulseAudio 通知有可读数据时触发 (在 PA mainloop 线程)
↓
pa_stream_peek() ← 获取录音数据指针 (零拷贝)
↓
ReadRecordedData()
↓
ProcessRecordedData()
↓
AudioDeviceBuffer::SetRecordedBuffer()
AudioDeviceBuffer::DeliverRecordedData() ← 推给 AudioTransport
↓
pa_stream_drop() ← 释放 peek 的数据
```

2.6 Underflow 自适应延迟

```
void AudioDeviceLinuxPulse::PaStreamUnderflowCallbackHandler() {
    // 每次 underflow, 增加目标延迟
    _configuredLatencyPlay += WEBRTC_PA_PLAYBACK_LATENCY_INCREMENT_MSECS;
    // 重新配置 stream 的缓冲属性
    pa_buffer_attr new_attr;
    new_attr.tlength = pa_usec_to_bytes(
        _configuredLatencyPlay * WEBRTC_PA_MSECS_PER_SEC, &_amp;_playbackBufferAttr);
    pa_stream_set_buffer_attr(_playStream, &new_attr, nullptr, nullptr);
}
```

这是 PulseAudio 实现的一个重要设计：**延迟自适应**。初始延迟 20ms，如果发生 underflow 就增加 20ms，直到稳定。

2.7 动态符号加载

```
// pulseaudiosymboltable_linux.h
// PulseAudio 通过 dlopen 动态加载, 避免硬依赖
typedef webrtc::adm_linux_pulse::PulseAudioSymbolTable WebRTCPulseSymbolTable;
WebRTCPulseSymbolTable* GetPulseSymbolTable();
```

这意味着即使系统没有安装 PulseAudio，WebRTC 也能编译和运行（只是 PulseAudio ADM 初始化会失败）。

三、ALSA 实现：AudioDeviceLinuxALSA

文件：`modules/audio_device/linux/audio_device_alsa_linux.h`

3.1 同步 PCM 接口

ALSA 使用**同步阻塞**接口，与 PulseAudio 的异步回调完全不同：

```
// ALSA 核心句柄
snd_pcm_t* _handlePlayout;    // 播放 PCM 句柄
snd_pcm_t* _handleRecord;    // 录音 PCM 句柄

// 同步读写
snd_pcm_writei(_handlePlayout, buffer, frames); // 阻塞写入
snd_pcm_readi(_handleRecord, buffer, frames);   // 阻塞读取
```

3.2 双工作线程（与 PulseAudio 对称）

```
PlatformThread _ptrThreadPlay; // 播放线程（循环调用 snd_pcm_writei）
PlatformThread _ptrThreadRec;  // 录音线程（循环调用 snd_pcm_readi）
```

3.3 动态符号加载

```
// alsasymboltable_linux.h
typedef webrtc::adm_linux_alsa::AlsaSymbolTable WebRTCAlsaSymbolTable;
WebRTCAlsaSymbolTable* GetAlsaSymbolTable();
// 同样通过 dlopen 动态加载 libasound.so
```

四、两套实现对比

特性	PulseAudio	ALSA
API 风格	异步回调	同步阻塞
多应用共享	支持 (PA 服务器混音)	不支持 (独占设备)
延迟	较高 (≥20ms)	较低 (可达 5ms)
热插拔	支持	有限支持
蓝牙	支持 (通过 PA 路由)	不支持
线程模型	PA mainloop + 工作线程	纯工作线程
动态加载	是 (dlopen)	是 (dlopen)
构建标志	<code>rtc_include_pulse_audio=true</code> (默认)	<code>WEBRTC_ENABLE_LINUX_ALSA</code> (默认开启)
适用场景	桌面 Linux	嵌入式/服务器

五、线程模型

```
[worker_thread]
WebRtcVoiceEngine::Init()
→ adm->Init()
→ AudioDeviceLinuxPulse::Init()
→ InitPulseAudio() ← 连接 PA 服务器, 创建 mainloop
→ CheckPulseAudioVersion()
→ InitSamplingFrequency() ← 查询服务器默认采样率

AudioState::AddReceivingStream()
→ adm->StartPlayout()
→ 创建 _ptrThreadPlay (播放工作线程)
→ 创建 pa_stream (播放流)

[播放工作线程 _ptrThreadPlay]
PlayThreadProcess() 循环:
    _timeEventPlay.Wait(1000)
    PaLock()
    pa_stream_write(PCM data)
    PaUnlock()

[PulseAudio mainloop 线程 (内部)]
PaStreamWriteCallback → PaStreamWriteCallbackHandler
PaStreamUnderflowCallback → 自适应延迟调整
```

```
[录音工作线程 _ptrThreadRec]
RecThreadProcess() 循环:
    _timeEventRec.Wait(1000)
    PaLock()
    pa_stream_peek() → 处理数据 → pa_stream_drop()
    PaUnLock()
```

六、PaLock / PaUnLock

PulseAudio 的 mainloop 不是线程安全的，所有对 PA API 的调用必须在持有 mainloop 锁时进行：

```
void AudioDeviceLinuxPulse::PaLock() {
    pa_threaded_mainloop_lock(&_amp;Mainloop);
}
void AudioDeviceLinuxPulse::PaUnLock() {
    pa_threaded_mainloop_unlock(&_amp;Mainloop);
}
```

`WaitForOperationCompletion` 用于等待异步操作完成：

```
void AudioDeviceLinuxPulse::WaitForOperationCompletion(
    pa_operation* paOperation) const {
    while (pa_operation_get_state(paOperation) == PA_OPERATION_RUNNING) {
        pa_threaded_mainloop_wait(&_amp;Mainloop); // 释放锁并等待
    }
    pa_operation_unref(paOperation);
}
```

七、源码文件索引

文件	内容
<code>modules/audio_device/linux/audio_device_pulse_linux.h/.cc</code>	PulseAudio 实现
<code>modules/audio_device/linux/audio_device_alsa_linux.h/.cc</code>	ALSA 实现
<code>modules/audio_device/linux/pulseaudiosymboltable_linux.h/.cc</code>	PulseAudio 动态符号表
<code>modules/audio_device/linux/alsasymboltable_linux.h/.cc</code>	ALSA 动态符号表
<code>modules/audio_device/linux/audio_mixer_manager_pulse_linux.h/.cc</code>	PulseAudio 音量控制
<code>modules/audio_device/linux/audio_mixer_manager_alsa_linux.h/.cc</code>	ALSA 音量控制
<code>modules/audio_device/linux/latebindingsymboltable_linux.h/.cc</code>	dlopen 动态加载基础设施

Android 平台音频实现深度分析

源码路径：`sdk/android/src/jni/audio_device/`

一、三套实现概览

Android 上 WebRTC 提供三套实现，通过 `AudioLayer` 枚举选择：

<code>kAndroidJavaAudio</code>	→ Java <code>AudioRecord</code> + Java <code>AudioTrack</code>
<code>kAndroidOpenSLESAudio</code>	→ OpenSL ES 录音 + OpenSL ES 播放
<code>kAndroidJavaInputAndOpenSLESAudio</code>	→ Java 录音 + OpenSL ES 播放（混合）
<code>kAndroidAAudioAudio</code>	→ AAudio 录音 + AAudio 播放
<code>kAndroidJavaInputAndAAudioOutputAudio</code>	→ Java 录音 + AAudio 播放（混合）

选择逻辑：Android ADM 不通过 `AudioDeviceModuleImpl::Create` 创建，而是有独立工厂：

```
// audio_device_impl.cc
} else if (audio_layer == AudioDeviceModule::kAndroidJavaAudio || ...) {
    RTC_LOG(LS_ERROR) << "Use the CreateAndroidAudioDeviceModule() "
                          "factory method instead for this option.";
    return nullptr; // ← 明确拒绝，必须用专用工厂
}
```

文件：`sdk/android/src/jni/audio_device/audio_device_module.h`

```
// Android 专用工厂函数
scoped_refptr<AudioDeviceModule> CreateAudioDeviceModuleFromInputAndOutput(
    const Environment& env,
    AudioDeviceModule::AudioLayer audio_layer,
    bool is_stereo_playout_supported,
    bool is_stereo_record_supported,
    uint16_t playout_delay_ms,
    std::unique_ptr<AudioInput> audio_input,
    std::unique_ptr<AudioOutput> audio_output);
```

Android ADM 同样采用 **Input/Output 分离架构**（与 Windows 新实现相同），通过组合不同的 `AudioInput` 和 `AudioOutput` 实现来支持混合模式。

二、架构：Input/Output 分离

```
// Android 的 AudioInput 接口 (jni/audio_device/audio_device_module.h)
class AudioInput {
    virtual int32_t Init() = 0;
    virtual int32_t Terminate() = 0;
    virtual int32_t InitRecording() = 0;
    virtual bool RecordingIsInitialized() const = 0;
    virtual int32_t StartRecording() = 0;
    virtual int32_t StopRecording() = 0;
    virtual bool Recording() const = 0;
    virtual void AttachAudioBuffer(AudioDeviceBuffer* audioBuffer) = 0;
    // Android 特有：内置 AEC/NS 支持查询
    virtual bool IsAcousticEchoCancellerSupported() const = 0;
```



```

virtual bool IsNoiseSuppressorSupported() const = 0;
virtual int32_t EnableBuiltInAEC(bool enable) = 0;
virtual int32_t EnableBuiltInNS(bool enable) = 0;
};

class AudioOutput {
    virtual int32_t Init() = 0;
    virtual int32_t InitPlayout() = 0;
    virtual int32_t StartPlayout() = 0;
    virtual int32_t StopPlayout() = 0;
    virtual bool Playing() const = 0;
    virtual bool SpeakerVolumeIsAvailable() = 0;
    virtual int SetSpeakerVolume(uint32_t volume) = 0;
    virtual void AttachAudioBuffer(AudioDeviceBuffer* audioBuffer) = 0;
    virtual int GetPlayoutUnderrunCount() = 0;
    virtual std::optional<AudioDeviceModule::Stats> GetStats() const;
};

```

实现组合：

AudioLayer	AudioInput 实现	AudioOutput 实现
kAndroidJavaAudio	AudioRecordJni	AudioTrackJni
kAndroidOpenSLESAudio	OpenSLESRecorder	OpenSLESPlayer
kAndroidJavaInputAndOpenSLESAudio	AudioRecordJni	OpenSLESPlayer
kAndroidAAudioAudio	AAudioRecorder	AAudioPlayer
kAndroidJavaInputAndAAudioOutputAudio	AudioRecordJni	AAudioPlayer

三、Java 实现：AudioRecordJni / AudioTrackJni

文件：`sdk/android/src/jni/audio_device/audio_record_jni.h`

3.1 Java/Native 边界

```

// AudioRecordJni 是 C++ 侧的代理，实际工作由 Java 的 WebRtcAudioRecord.java 完成
class AudioRecordJni : public AudioInput {
    // Java 对象引用
    jni_zero::ScopedJavaGlobalRef<jobject> j_audio_record_;

    // Java 侧分配的 DirectByteBuffer, C++ 直接读取（零拷贝）
    void* direct_buffer_address_;
    size_t direct_buffer_capacity_in_bytes_;
};

```

3.2 录音数据回调路径

```
[Java AudioRecordThread (高优先级线程) ]
WebRtcAudioRecord.java 的录音线程
↓ 调用 JNI
AudioRecordJni::DataIsRecorded(env, j_caller, length, capture_timestamp_ns)
↓
// direct_buffer_address_ 已包含录音数据 (Java 写入, C++ 读取)
audio_device_buffer_>SetRecordedBuffer(direct_buffer_address_, frames_per_buffer_)
audio_device_buffer_>SetVQEData(total_delay_ms_, 0)
audio_device_buffer_>DeliverRecordedData()
↓
AudioTransport::RecordedDataIsAvailable()
```

关键设计: `direct_buffer_address_` 是 Java 侧 `ByteBuffer.allocateDirect()` 分配的直接内存, C++ 可以直接访问, 避免了 JNI 数据拷贝。

3.3 线程检查器设计

```
// 两个独立的线程检查器
SequenceChecker thread_checker_;           // 绑定到创建线程 (worker_thread)
SequenceChecker thread_checker_java_;      // 绑定到 Java AudioRecordThread (kDetached)

// DataIsRecorded 在 Java 高优先级线程上调用
void AudioRecordJni::DataIsRecorded(...) {
    RTC_DCHECK_RUN_ON(&thread_checker_java_); // 验证在 Java 线程上
    // ...
}
```

3.4 内置 AEC/NS

```
bool AudioRecordJni::IsAcousticEchoCancellerSupported() const {
    // 查询 Java 侧 WebRtcAudioEffects.isAcousticEchoCancellerSupported()
    // Android 4.1+ (API 16) 开始支持硬件 AEC
}

int32_t AudioRecordJni::EnableBuiltInAEC(bool enable) {
    // 调用 Java 侧 WebRtcAudioRecord.enableBuiltInAEC()
    // 如果启用, WebRTC 软件 AEC 会被禁用 (与 iOS 类似)
}
```

四、OpenSL ES 实现

文件: `sdk/android/src/jni/audio_device/opensles_player.h`

4.1 Buffer Queue 机制

OpenSL ES 使用缓冲队列 (Buffer Queue) 模式, 与 iOS 的 AudioUnit 回调类似:

```

class OpenSLESPlayer : public AudioOutput {
    // OpenSL ES 对象层次
    SLObjectItf engine_object_;      // 引擎对象
    SLEngineItf engine_;             // 引擎接口
    SLObjectItf output_mix_object_;  // 输出混音对象
    SLObjectItf player_object_;      // 播放器对象
    SLPlayItf player_;               // 播放接口
    SLAndroidSimpleBufferQueueItf simple_buffer_queue_; // 缓冲队列

    // FineAudioBuffer: 适配 OpenSL ES 的 buffer size 与 WebRTC 10ms 的差异
    std::unique_ptr<FineAudioBuffer> fine_audio_buffer_;
};

```

4.2 播放数据流

```

[OpenSL ES 内部线程 (实时优先级)]
缓冲队列回调 (SimpleBufferQueueCallback)
↓
OpenSLESPlayer::SimpleBufferQueueCallback (静态)
↓
fine_audio_buffer_->GetPlayoutData(audio_buffer, playout_delay_ms)
↓
AudioDeviceBuffer::RequestPlayoutData()
↓
AudioTransport::NeedMorePlayData()
↓
AudioMixer::Mix() → 解码后的 PCM
↓
SLAndroidSimpleBufferQueueItf::Enqueue(buffer) ← 提交给硬件

```

4.3 低延迟条件

```

// API 17+ (Android 4.2): buffer count >= 2 可获得低延迟
// API 18+ (Android 4.3): buffer count = 1 即可低延迟
// 条件: buffer size 和 sample rate 必须与设备 native 参数匹配
// native 参数通过 AudioManager.getProperty() 查询

```

五、AAudio 实现

文件: `sdk/android/src/jni/audio_device/aaudio_wrapper.h`

5.1 AAudio 特性

```
// AAudio 是 Android O (API 26) 引入的新 C API
// 设计目标：低延迟、高性能
class AAudioWrapper {
    const AudioParameters audio_parameters_;
    const aaudio_direction_t direction_; // 输入或输出
    AAudioObserverInterface* observer_; // 数据回调接口
    AAudioStream* stream_; // AAudio 流
    int32_t frames_per_burst_; // 硬件 burst 大小（最小处理单元）
};
```

5.2 数据回调接口

```
class AAudioObserverInterface {
    // 实时线程回调：数据读写
    virtual aaudio_data_callback_result_t OnDataCallback(
        void* audio_data, int32_t num_frames) = 0;

    // 错误回调：设备断开等（不要在此重开流）
    virtual void OnErrorCallback(aaudio_result_t error) = 0;
};
```

5.3 低延迟性能模式

```
// AAudioWrapper::SetStreamConfiguration()
AAudioStreamBuilder_setPerformanceMode(
    builder,
    AAUDIO_PERFORMANCE_MODE_LOW_LATENCY); // ← 请求低延迟模式
// 系统会尽量满足，但不保证
// 实际延迟通过 EstimateLatencyMillis() 查询
```

5.4 自动重启

```
// AAudio 设备断开时触发 OnErrorCallback
// 正确处理方式：在另一个线程重新打开流
void AAudioPlayer::OnErrorCallback(aaudio_result_t error) {
    // 不能在回调线程中重开流
    // 通过 PostTask 到 worker_thread 处理
    thread_>PostTask([this] { HandleStreamDisconnected(); });
}
```

六、三套实现延迟对比

实现	典型延迟	最低延迟条件	适用场景
Java AudioRecord/Track	100-200ms	无法保证低延迟	兼容性最好
OpenSL ES	20-50ms	API 17+, native 参数匹配	Android 4.1-7.x
AAudio	5-20ms	API 26+, 低延迟模式	Android 8.0+

混合模式的意义（kAndroidJavaInputAndAAudioOutputAudio）：

- 录音用 Java：稳定性好，内置 AEC 支持完善
- 播放用 AAudio：低延迟，减少播放端延迟
- 适合对播放延迟敏感但录音延迟要求不高的场景

七、线程模型

```
[worker_thread]
WebRtcVoiceEngine::Init()
    → adm->Init() ← AudioRecordJni::Init() + AudioTrackJni::Init()
    → 初始化 Java 对象，缓存 DirectByteBuffer 地址

AudioState::AddReceivingStream()
    → adm->StartPlayout()
    → AudioTrackJni::StartPlayout()
    → Java: WebRtcAudioTrack.startPlayout()
    → 启动 Java AudioTrack 播放线程

[Java AudioTrack 播放线程（高优先级）]
WebRtcAudioTrack.java 内部线程
    → 调用 JNI AudioTrackJni::GetPlayoutData()
    → AudioDeviceBuffer::RequestPlayoutData()
    → AudioTransport::NeedMorePlayData()

[Java AudioRecord 录音线程（高优先级）]
WebRtcAudioRecord.java 内部线程
    → 调用 JNI AudioRecordJni::DataIsRecorded()
    → AudioDeviceBuffer::DeliverRecordedData()
    → AudioTransport::RecordedDataIsAvailable()

[AAudio 实时线程（系统管理）]
AAudioWrapper 数据回调
    → AAudioPlayer::OnDataCallback() / AAudioRecorder::OnDataCallback()
    → FineAudioBuffer → AudioDeviceBuffer → AudioTransport
```

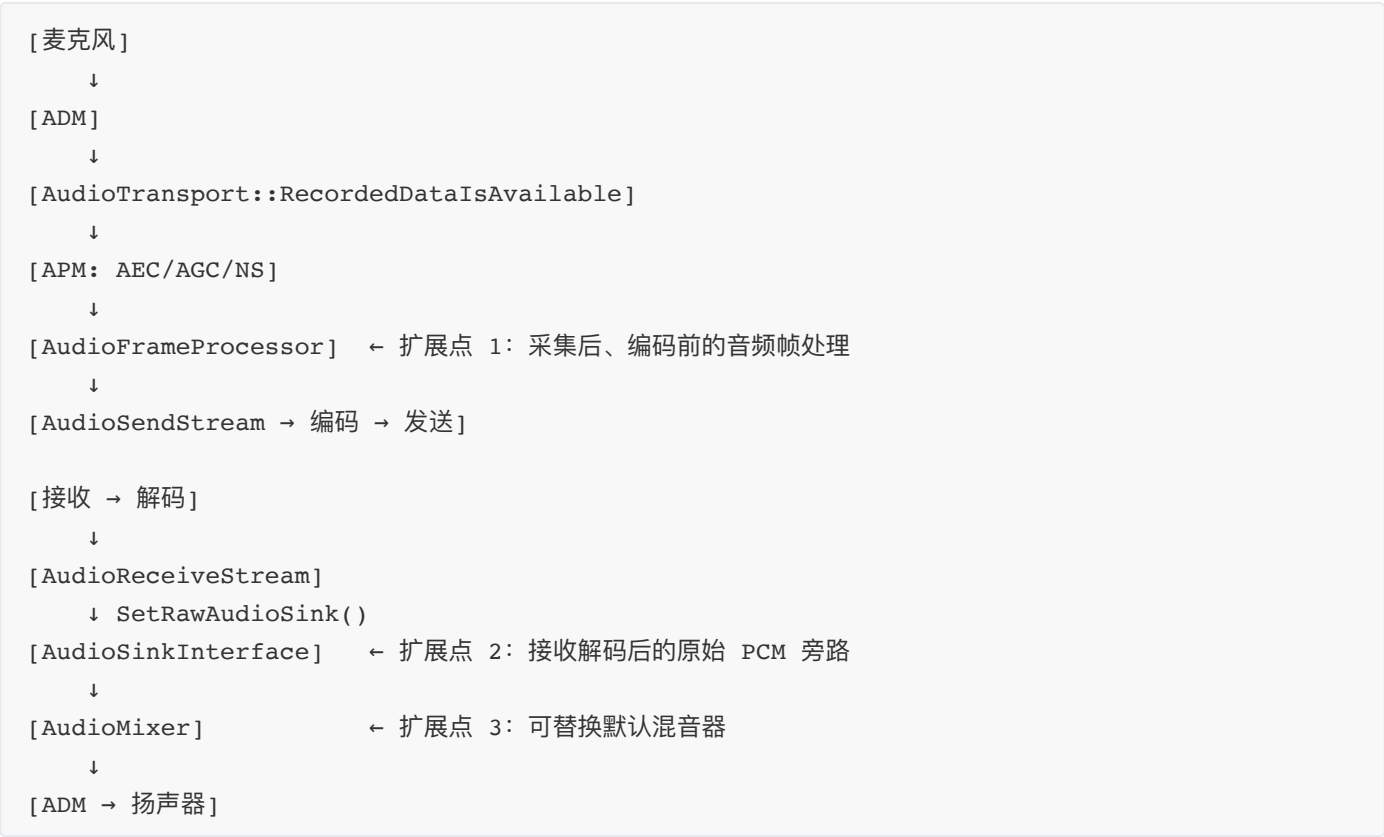
八、源码文件索引

文件	内容
sdk/android/src/jni/audio_device/audio_device_module.h/.cc	Android ADM 工厂和 Input/Output 接口
sdk/android/src/jni/audio_device/audio_record_jni.h/.cc	Java AudioRecord 录音实现
sdk/android/src/jni/audio_device/audio_track_jni.h/.cc	Java AudioTrack 播放实现
sdk/android/src/jni/audio_device/opensles_recorder.h/.cc	OpenSL ES 录音实现
sdk/android/src/jni/audio_device/opensles_player.h/.cc	OpenSL ES 播放实现
sdk/android/src/jni/audio_device/opensles_common.h/.cc	OpenSL ES 公共工具
sdk/android/src/jni/audio_device/aaudio_wrapper.h/.cc	AAudio 流封装
sdk/android/src/jni/audio_device/aaudio_recorder.h/.cc	AAudio 录音实现
sdk/android/src/jni/audio_device/aaudio_player.h/.cc	AAudio 播放实现
sdk/android/src/jni/audio_device/audio_common.h	公共常量（延迟估算等）

音频扩展点概览

本章介绍 WebRTC 音频管线中三个主要的扩展注入点，均有源码依据。

一、三个扩展点总览



二、扩展点 1: AudioFrameProcessor

文件: `api/audio/audio_frame_processor.h`

```
// 注释原文:
// "If passed into PeerConnectionFactory, will be used for additional
// processing of captured audio frames, performed before encoding.
// Implementations must be thread-safe."
class AudioFrameProcessor {
public:
    using OnAudioFrameCallback = std::function<void(std::unique_ptr<AudioFrame>)>;

    // 处理从 WebRTC 收到的音频帧
    // 在实时音频采集路径上调用 (非 worker_thread)
    // 必须通过 sink_callback 将处理后的帧传回
    virtual void Process(std::unique_ptr<AudioFrame> frame) = 0;

    // 原子替换当前 sink。未设置时, 处理后的帧被丢弃
    virtual void SetSink(OnAudioFrameCallback sink_callback) = 0;
};
```

注入方式

文件: `api/enable_media.cc`

```
auto audio_engine = std::make_unique<WebRtcVoiceEngine>(
    env,
    std::move(deps.adm),
    std::move(deps.audio_encoder_factory),
    std::move(deps.audio_decoder_factory),
    std::move(deps.audio_mixer),
    std::move(audio_processing),
    std::move(deps.audio_frame_processor)); // ← 注入点
```

通过 `PeerConnectionFactoryDependencies::audio_frame_processor` 注入:

```
PeerConnectionFactoryDependencies deps;
deps.audio_frame_processor = std::make_unique<MyAudioFrameProcessor>();
EnableMedia(deps);
auto factory = CreateModularPeerConnectionFactory(std::move(deps));
```

在管线中的位置

文件: `audio/audio_transport_impl.cc`

```
// WebRtcVoiceEngine 构造时, 如果有 audio_frame_processor,
// 会通过 AsyncAudioProcessing::Factory 包装
AudioTransportImpl::AudioTransportImpl(
    AudioMixer* mixer,
    AudioProcessing* audio_processing,
    AsyncAudioProcessing::Factory* async_audio_processing_factory)
: async_audio_processing_(
    async_audio_processing_factory
```

```

        ? async_audio_processing_factory->CreateAsyncAudioProcessing(
            [this](std::unique_ptr<AudioFrame> frame) {
                this->SendProcessedData(std::move(frame)); // ← 处理完后回调
            })
        : nullptr) {}

// RecordedDataIsAvailable 中:
if (async_audio_processing_)
    async_audio_processing_->Process(std::move(audio_frame)); // ← 异步处理
else
    SendProcessedData(std::move(audio_frame)); // ← 同步直接发送

```

调用线程：实时音频采集线程（平台 I/O 线程），实现必须线程安全。

适用场景：

- 自定义降噪/增益处理
- 音频数据录制/监听（采集侧）
- 语音活动检测（VAD）自定义实现

三、扩展点 2：AudioSinkInterface

文件：api/call/audio_sink.h

```

class AudioSinkInterface {
public:
    struct Data {
        const int16_t* data;           // 16-bit PCM 数据
        size_t samples_per_channel;    // 每通道采样数
        int sample_rate;               // 采样率 (Hz)
        size_t channels;               // 通道数
        uint32_t timestamp;            // RTP 时间戳（第一个采样）
    };

    virtual void OnData(const Data& audio) = 0;
};

```

注入方式

通过 VoiceMediaReceiveChannelInterface::SetRawAudioSink：

文件：media/engine/webrtc_voice_engine.h


```
class WebRtcVoiceReceiveChannel {
    // 为特定 SSRC 设置 sink (接收解码后的原始 PCM)
    void SetRawAudioSink(uint32_t ssrc,
                        std::unique_ptr<AudioSinkInterface> sink) override;

    // 为未信令流设置默认 sink
    void SetDefaultRawAudioSink(
        std::unique_ptr<AudioSinkInterface> sink) override;
};
```

实际使用（通过 RtpReceiver）：

```
// 通过 PeerConnection 获取 RtpReceiver 后设置
receiver->SetSink(my_audio_sink.get());
// 或通过 MediaChannel 直接设置 (内部 API)
```

在管线中的位置

AudioSink 在解码之后、混音之前接收数据，是接收侧的旁路出口：

```
[网络接收] → [RTP 解包] → [解码] → [AudioSink::OnData()] ← 旁路
                        ↓
                    [AudioMixer Source]
                        ↓
                    [混音 → ADM → 扬声器]
```

调用线程：worker_thread（解码在 worker_thread 上进行）。

适用场景：

- 录制特定参与者的音频
- 实时语音转文字（STT）
- 音频质量监控

四、扩展点 3：自定义 AudioMixer

文件：api/audio/audio_mixer.h

```
class AudioMixer : public RefCountInterface {
public:
    class Source {
    public:
        enum class AudioFrameInfo { kNormal, kMuted, kError };
        // 混音器调用此方法拉取每个 Source 的音频数据
        virtual AudioFrameInfo GetAudioFrameWithInfo(int sample_rate_hz,
                                                    AudioFrame* audio_frame) = 0;

        virtual int Ssrc() const = 0;
        virtual int PreferredSampleRate() const = 0;
    };
};
```

```
};

virtual bool AddSource(Source* audio_source) = 0;
virtual void RemoveSource(Source* audio_source) = 0;
// 混合所有 Source, 结果写入 audio_frame_for_mixing
virtual void Mix(size_t number_of_channels,
                 AudioFrame* audio_frame_for_mixing) = 0;
};
```

注入方式

通过 `PeerConnectionFactoryDependencies::audio_mixer` :

```
PeerConnectionFactoryDependencies deps;
deps.audio_mixer = my_custom_mixer; // scoped_refptr<AudioMixer>
EnableMedia(deps);
```

文件: `media/engine/webrtc_voice_engine.cc`

```
AudioState::Config config;
if (audio_mixer) {
    config.audio_mixer = std::move(audio_mixer); // ← 使用自定义
} else {
    config.audio_mixer = AudioMixerImpl::Create(); // ← 默认实现
}
```

适用场景:

- 自定义混音算法 (如空间音频)
- 混音前对每路音频做处理
- 自定义音量控制策略

五、三个扩展点对比

扩展点	位置	方向	调用线程	注入方式
AudioFrameProcessor	APM 之后, 编码之前	发送 (采集)	平台音频线程	deps.audio_frame_processor
AudioSinkInterface	解码之后, 混音之前	接收 (播放)	worker_thread	SetRawAudioSink(ssrc, sink)
AudioMixer	混音阶段	接收 (播放)	平台音频线程	deps.audio_mixer

六、源码文件索引

文件	内容
api/audio/audio_frame_processor.h	AudioFrameProcessor 接口
api/call/audio_sink.h	AudioSinkInterface 接口
api/audio/audio_mixer.h	AudioMixer 接口
modules/audio_mixer/audio_mixer_impl.h/.cc	默认 AudioMixer 实现
modules/async_audio_processing/async_audio_processing.h	AudioFrameProcessor 的异步包装
audio/audio_transport_impl.cc	AudioFrameProcessor 在管线中的调用位置
audio/channel_receive.cc	AudioSinkInterface 的调用位置
media/engine/webrtc_voice_engine.cc	AudioMixer 注入位置

自定义 ADM 实现指南

基于 modules/audio_device/include/audio_device_default.h 和 fake_audio_device.h

一、为什么需要自定义 ADM

默认 ADM 使用平台麦克风/扬声器。以下场景需要自定义：

- 从文件/网络读取音频（测试、录制回放）
- 自定义音频处理管线（绕过平台 API）
- 单元测试（不依赖真实硬件）
- 虚拟音频设备（屏幕共享时的系统音频采集）

二、最简实现：继承 AudioDeviceModuleDefault

文件：modules/audio_device/include/audio_device_default.h

WebRTC 提供了 AudioDeviceModuleDefault<T> 模板，为所有方法提供默认实现（大多数返回 0 或 false）：

```
#include "modules/audio_device/include/audio_device_default.h"
#include "api/make_ref_counted.h"

class MyCustomADM
    : public webrtc_impl::AudioDeviceModuleDefault<webrtc::AudioDeviceModule> {
public:
    // 必须实现：引用计数（继承自 RefCountInterface）
    // 使用 make_ref_counted<MyCustomADM>() 创建时自动处理

    // ① 必须实现：保存 AudioTransport 回调
    int32_t RegisterAudioCallback(webrtc::AudioTransport* transport) override {
        audio_transport_ = transport;
        return 0;
    }
}
```

```

// ② 必须实现：初始化（准备音频源）
int32_t Init() override {
    // 初始化你的音频源
    return 0;
}

// ③ 必须实现：开始录音（启动推送数据的线程/定时器）
int32_t StartRecording() override {
    recording_ = true;
    // 启动你的音频采集线程
    return 0;
}

int32_t StopRecording() override {
    recording_ = false;
    return 0;
}

bool Recording() const override { return recording_; }

// ④ 必须实现：开始播放（启动拉取数据的线程/定时器）
int32_t StartPlayout() override {
    playing_ = true;
    // 启动你的音频播放线程
    return 0;
}

int32_t StopPlayout() override {
    playing_ = false;
    return 0;
}

bool Playing() const override { return playing_; }

private:
    webrtc::AudioTransport* audio_transport_ = nullptr;
    bool recording_ = false;
    bool playing_ = false;
};

```

注入方式：

```

PeerConnectionFactoryDependencies deps;
deps.adm = webrtc::make_ref_counted<MyCustomADM>();
webrtc::EnableMedia(deps);
auto factory = webrtc::CreateModularPeerConnectionFactory(std::move(deps));

```

三、推送录音数据（采集侧）

在你的音频采集线程中，每 10ms 调用一次：

```

// 在音频采集线程中 (非 worker_thread)
void MyCustomADM::OnAudioCaptured(const int16_t* pcm_data,
                                   size_t samples_per_channel,
                                   int sample_rate_hz,
                                   size_t num_channels) {
    if (!audio_transport_ || !recording_) return;

    uint32_t new_mic_level = 0;
    // RecordedDataIsAvailable 参数说明:
    // - pcm_data: 16-bit PCM, 交错格式 (stereo: L R L R ...)
    // - samples_per_channel: 每通道采样数, 必须 = sample_rate_hz / 100 (10ms)
    // - bytes_per_sample: num_channels * 2
    // - num_channels: 1 (mono) 或 2 (stereo)
    // - sample_rate_hz: 8000/16000/32000/44100/48000
    // - total_delay_ms: 播放延迟 + 录音延迟, 给 AEC 用, 不知道填 0
    // - clock_drift: 已废弃, 填 0
    // - current_mic_level: 当前麦克风音量 0-255, 不需要填 0
    // - key_pressed: 是否有按键, 填 false
    // - new_mic_level: [out] AGC 建议的新音量, 忽略即可
    audio_transport_>RecordedDataIsAvailable(
        pcm_data,
        samples_per_channel,
        num_channels * sizeof(int16_t), // bytes_per_sample
        num_channels,
        sample_rate_hz,
        /*total_delay_ms=*/0,
        /*clock_drift=*/0,
        /*current_mic_level=*/0,
        /*key_pressed=*/false,
        new_mic_level);
}

```

关键约束:

- `samples_per_channel` 必须严格等于 `sample_rate_hz / 100` (10ms 对齐)
- 如果你的音频源 buffer 大小不是 10ms, 使用 `FineAudioBuffer` 适配

四、拉取播放数据（播放侧）

在你的音频播放线程中, 每 10ms 调用一次:

```

void MyCustomADM::OnNeedPlaybackData(int sample_rate_hz,
                                     size_t num_channels,
                                     size_t samples_per_channel) {
    if (!audio_transport_ || !playing_) return;

    // 分配输出缓冲区
    std::vector<int16_t> buffer(samples_per_channel * num_channels);
    size_t samples_out = 0;
}

```

```

int64_t elapsed_time_ms = -1;
int64_t ntp_time_ms = -1;

audio_transport_->NeedMorePlayData(
    samples_per_channel,
    num_channels * sizeof(int16_t), // bytes_per_sample
    num_channels,
    sample_rate_hz,
    buffer.data(),
    samples_out,
    &elapsed_time_ms,
    &ntp_time_ms);

// buffer.data() 现在包含混音后的 PCM 数据
// 将其写入你的播放设备
WriteToPlaybackDevice(buffer.data(), samples_out * num_channels);
}

```

五、FineAudioBuffer 适配非 10ms buffer

如果你的音频源/目标 buffer 大小不是 10ms（如 5ms 或 20ms），使用 `FineAudioBuffer`：

```

#include "modules/audio_device/fine_audio_buffer.h"
#include "modules/audio_device/audio_device_buffer.h"

// 注意: FineAudioBuffer 需要 AudioDeviceBuffer, 不能直接用于自定义 ADM
// 自定义 ADM 的正确做法是在推送前手动积累 10ms 数据:

class MyCustomADM : public ... {
    // 内部积累缓冲
    std::vector<int16_t> accumulation_buffer_;
    size_t samples_per_10ms_; // = sample_rate_hz_ / 100

    void OnAudioCaptured(const int16_t* data, size_t frames) {
        // 追加到积累缓冲
        accumulation_buffer_.insert(accumulation_buffer_.end(),
                                    data, data + frames * num_channels_);

        // 凑够 10ms 才推送
        while (accumulation_buffer_.size() >= samples_per_10ms_ * num_channels_) {
            uint32_t new_mic_level = 0;
            audio_transport_->RecordedDataIsAvailable(
                accumulation_buffer_.data(),
                samples_per_10ms_,
                num_channels_ * sizeof(int16_t),
                num_channels_,
                sample_rate_hz_,
                0, 0, 0, false, new_mic_level);
            // 移除已处理的数据
            accumulation_buffer_.erase(
                accumulation_buffer_.begin(),

```

```
        accumulation_buffer_.begin() + samples_per_10ms_ * num_channels_);
    }
}
};
```

六、必须实现 vs 可用默认值

方法	是否必须实现	说明
RegisterAudioCallback	必须	保存 AudioTransport 指针
Init	必须	初始化音频源
StartRecording / StopRecording	必须（如需录音）	启停采集
StartPlayout / StopPlayout	必须（如需播放）	启停播放
Recording / Playing	必须	AudioState 依赖此状态判断
InitRecording / InitPlayout	建议实现	AudioState 在 Start 前调用
Terminate	建议实现	清理资源
SetPlayoutDevice / SetRecordingDevice	可用默认	无多设备时忽略
InitSpeaker / InitMicrophone	可用默认	返回 0 即可
所有音量控制方法	可用默认	返回 0
PlayoutDelay	可用默认	返回 0（影响 AEC 效果）
BuiltInAECIsAvailable	可用默认	返回 false

七、常见陷阱

陷阱 1：在音频线程调用 RegisterAudioCallback

```
// 错误：RegisterAudioCallback 要求媒体未激活时调用
// audio_device_buffer.cc:
int32_t AudioDeviceBuffer::RegisterAudioCallback(AudioTransport* audio_callback) {
    if (playing_ || recording_) {
        RTC_LOG(LS_ERROR) << "Failed to set audio transport since media was active";
        return -1;
    }
    audio_transport_cb_ = audio_callback;
}

// 正确：在 Init() 之后、StartRecording/StartPlayout 之前注册
// WebRTC 框架会在 WebRtcVoiceEngine::Init() 中自动调用
```

陷阱 2：推送数据不是 10ms 对齐

```
// 错误: samples_per_channel = 240 (5ms @ 48kHz)
audio_transport->RecordedDataIsAvailable(data, 240, ...);
// audio_transport_impl.cc 会 DCHECK 失败:
// RTC_DCHECK_EQ(number_of_frames * 100, sample_rate);

// 正确: 必须是 480 (10ms @ 48kHz)
audio_transport->RecordedDataIsAvailable(data, 480, ...);
```

陷阱 3：Recording()/Playing() 返回值不正确

```
// AudioState 依赖这两个方法判断是否需要启动 ADM
// 如果 Recording() 始终返回 false, AudioState 会反复调用 StartRecording()
// 必须在 StartRecording() 后返回 true, StopRecording() 后返回 false
```

陷阱 4：忘记线程安全

```
// audio_transport_ 可能被 worker_thread 写 (RegisterAudioCallback)
// 同时被音频线程读 (RecordedDataIsAvailable)
// 解决: 在 RegisterAudioCallback 中加锁, 或确保调用时序正确
// WebRTC 的约定: RegisterAudioCallback 只在媒体未激活时调用,
// 所以实际上不需要加锁 (时序保证了安全)
```

八、参考实现

文件	用途
modules/audio_device/include/fake_audio_device.h	最简 fake 实现 (继承 Default, 什么都不做)
modules/audio_device/include/test_audio_device.h	测试用实现 (支持从文件读取/写入)
modules/audio_device/dummy/audio_device_dummy.h	Dummy 实现 (所有方法返回 0)
modules/audio_device/include/audio_device_default.h	Default 模板 (推荐继承基类)